

Course Overview

Course Overview

Hi, everyone. My name is Kevin. Welcome to my course, C# Design Patterns. I'm a Microsoft MVP and freelance solution architect currently focused on APIs and security. Every codebase can be improved. In this course, you will learn what design patterns are. After that, you will learn how to vastly improve your codebase by applying 23 different design patterns, covering all sorts of challenges. Each of the 23 patterns will be described in detail with a real-life example. The pattern structure will be covered as well so you can learn how to apply it to your own codebase. You will also learn about use cases for the pattern and about pattern consequences and related patterns. When you're finished with this course, you will have the skills and knowledge to use state-of-the-art design patterns in C# to improve your codebase. Before beginning the course, you should be familiar with C#. I hope you'll join me on this journey to learn how to improve your codebase by applying design patterns to it with the C# Design Patterns course, at Pluralsight.

Introduction to Design Patterns

Coming Up

Hi there, and welcome to the C# Design Patterns course, at Pluralsight. I'm Kevin, and I'll guide you through this course. Let's start with a quick version check. This course was created with C# 12 and .NET 8. The version of Visual Studio I use is Visual Studio 2022. The course is 100% applicable to C# 10 to 12 and .NET 6 to 8. I can't speak about future major versions yet, but chances are pretty good that what you will learn here will still apply. That being said, I try to keep my courses up to date and/or create new versions of my courses whenever a new major long-term supported version is released. The first thing to look for is the Course info on the right-hand side of the course page. You are looking

at one of my other courses right now. By having a look at the Course info, you can see when the course was last updated. This means that clips were changed. For some courses, you may see a reviewed date. This means that at that time I reviewed the course to check whether the content was still valid. The second thing to look for is my author page. There, all my courses are listed, so you can check whether one exists that targets the exact version you're targeting. Especially useful is the Follow button. If you want to be notified of new courses or course updates, simply click that, and you will get an email whenever a new course or course update is released. Let's have a look at what's coming up. We'll start by having a look at the course prerequisites, frameworks, and tooling and what you need to get going. After that, you will learn how the course is structured and where you can get the demo code. This is important because this is not a regular course like most here at Pluralsight. Make sure you watch this part so you know what you can expect and whether this is a course for you. With that out of the way, we will dive into design patterns. You will learn what they are, what the Gang of Four is, and which types of patterns exist. At the end of the module, you will learn about a few common object-oriented principles that will be useful for the rest of the course. As usual, the best place to pose questions is on the Discussion tab on the course page. Like that, the questions and answers can help other people out as well. If you want to know about my upcoming courses, user group events or are interested in the technology I'm interested in, you can follow me on X. I mainly post regarding ASP.NET Core, security, and APIs. Let's continue with the course prerequisites.

Course Prerequisites and Frameworks

This course is a best practices course in which I will cover a variety of design patterns. The focus is on the pattern itself, not on all the surrounding framework code. In other words, you should know C#, and that's about it. I'm using version 12 in this course, which is the latest version available at the time of recording. That being said, these patterns have been around long before C# 12 or even before C# itself existed, so the exact version really isn't that important. As far as .NET is concerned, .NET 8 is targeted. Let's have a look at the tooling we'll use.

Tooling and Setting up Your Environment

This course started with a quick version check. Let's elaborate on that a bit. As far as tooling is concerned, I will use Visual Studio 2022. If you haven't got a license, you can download the free Community Edition, but that doesn't mean you need to use Visual Studio 2022. .NET applications can be developed using other IDEs like Visual Studio Code. That one also runs on Mac and Linux. Feel free to use the IDE you feel most comfortable with. As mentioned, I'm using Visual Studio. If you want to use that as well and you haven't installed it yet, the first thing to do is install it. It could be that you or the company you work for has a license for using Visual Studio. In that case, you can use that license to install the Professional or Enterprise edition, depending on the type of your license. If you don't have a license, you can either download a trial version or, as mentioned before, the Community Edition. If you already have Visual Studio or another IDE installed, but you don't have .NET 8 support yet, you can download it at the URL on screen. Simply download and install the full .NET 8 SDK, and you're good to go.

Course Structure

You will learn about all 23 Gang of Four design patterns, more on that in a few minutes. Next to the Gang of Four design patterns, two additional often used enterprise patterns are also in the course, the Repository pattern and the Unit of Work pattern. They were added afterwards, after some viewers asked about them. You can find those modules near the end of the course. Each module covers one pattern, and each module also roughly adheres to the same structure. We'll first learn what the pattern in question is about, the pattern's intent, why it is used, and what it does. We'll clarify this with a real-life example to make it more tangible to understand which problem the pattern solves. After that, we'll build a diagram so you will learn how the different components of the pattern work together. This is called the pattern structure. That will always be a theoretical exercise, with class names like singleton A, factory B, etc. Having that theoretical view, allows you to easily apply it to day-to-day, real-life scenarios, and that is exactly what we will do next. We will implement the real-life use case used to describe the pattern in a demo. Expect lots of people, employees, shopping carts, and car classes in the upcoming hours. For each pattern, you will end up with a real-life example implementation. After that, and depending on the pattern, we will look into other real-life implementations, variations, points of interest, or alternatives. And then, at the end of each module, we will look into pattern use cases, pattern consequences, and related patterns. With pattern use case, I

mean that we're going to check out cases for which the pattern can help. If you notice one of the potential use cases in your code or your requirements, the pattern in question might be a good match. We will also look into different real-life use cases for each pattern in this part of the module. With pattern consequences, I mean the pros and cons of each pattern. In other words, pattern consequences can be both positive and negative. It's up to you to weigh the pros versus the cons for your specific situation. Don't just implement a pattern because you can, implement it when the advantages outweigh the disadvantages for your use case. You'll notice that patterns tend to solve related issues and often have comparable templates and implementations, that is why looking into related patterns is useful. Like that, you don't just learn how they are the same, but also how they are different, and how they can be combined. The patterns are grouped by type, creational, structural, or behavioral, more on that in a minute. Normally, after that, the Gang of Four ordered the patterns alphabetically by type. But, I decided not to do this. Instead, I ordered them by roughly the amount of times I used them in real-life projects. I realize that this is not a scientific method, but, as it is rooted in real life, I do think it carries some value. Okay, so that is what is in the course. But that begs the question, how should you watch this course? Well, you can choose whichever order you want, or simply watch those patterns that you need to learn about. Each module covers one pattern and can be regarded as a standalone module. The only module everyone should really watch first is the current one. Now, that does have some consequences. For one, seeing the order does not matter, I will often refer to other patterns than the one that's covered in the module you're watching, some of which you might not even know about yet. This will be especially true when I'm covering related patterns. But, seeing I don't know in which order you're watching these modules or what you know about other patterns, this may feel a bit confusing in the beginning. If that is the case, don't worry, know that once you've seen a few modules, this will all start to feel familiar and even out. Most patterns are related to each other in likewise manners. Also, each module is structured in the same way, that has the advantage of predictability and feeling at home in each module. Yet, it may feel somewhat repetitive if you watch all modules in a row. Here, too, I'd suggest you simply pick out the modules, clips, and demos you're interested in. The demo code for this course consists of one large solution with one project per pattern. All projects are console applications, and we're starting from an empty console app for each pattern, and that is exactly what's in the starter solution, which you are looking at right now. So, for each pattern, there's a console application with two files in it, a Program.cs file in which we will

write code to test the pattern, and an `Implementation.cs` file, in which we'll write code to implement the pattern. This is thus a very simple starter solution, but it will save us some boring steps, as we no longer have to add new projects for each pattern, they're already there. You can download the starter solution and finished solution via the Exercise File step, or you can get it from my GitHub, and that is probably the best place to get it from, as that's the place where I'll be able to commit updates and/or bug fixes the fastest. One final thing of importance. When creating this course, I tried to stick to what we get out of the box in Microsoft's templates regarding language features as much as possible. For example, I do use the `global using` feature, but I don't use the `file_scoped_namespace` feature. But, C# evolves fast, and it's not because something isn't currently in a default template that it's not useful. Therefore, I created a second branch on my GitHub, `latest-and-greatest`. In this branch, I do use language features that aren't necessarily commonly used yet. Most of this will probably make it into the main branch in a later version. But if you're interested in it already, have a look. Let's learn what design patterns are next.

Introducing Design Patterns

A design pattern is a general, reusable solution to a commonly occurring problem within a given context in software design. You can look at it as a template you can use to solve common issues you'll encounter during software design. They help with creating flexible, easily-maintainable applications. In that regard, you can consider them as best-practice approaches to common problems. Very important to remember is that you should see these patterns as a template to start from, not as a rule set in stone. Implementing a pattern can typically be done in multiple ways, depending on, for example, the language you're implementing them in. A good example of that is that you will notice that a lot of patterns talk about abstract classes. However, in C#, you can often use an interface instead, if all you need is a contract and not an implementation to start from. Throughout the course, you will learn how to implement patterns in various ways. That being said, each pattern has an intent. For example, the Adapter pattern, which we will encounter, intends to let classes work together that otherwise cannot. No matter how you implement this pattern, the intent should remain the same, and that is true for all patterns. A lot of these patterns are so common that you've undoubtedly used them before, maybe even without knowing that you were using them. Important is that you shouldn't learn the pattern implementation from the top of your head. Instead, it's more useful to learn which problem a pattern

solves. The patterns we will cover in this course are the well-known Gang of Four design patterns. Next to those, two additional enterprise patterns were added together with an updated version of this course, the Repository pattern and the Unit of Work pattern. You will find those modules at the end of the course. The Gang of Four are Eric Gamma, Richard Helm, Ralph Johnson, and John Vlissides. In 1994, these guys published a book that, to this day, is the de-facto standard work on software design patterns. It's named, Design Patterns, Elements of Reusable Object-Oriented software. It consists of 23 design patterns, and those are the ones we will cover in this course. Even though the book was written in 1994, these patterns are still commonly used today. They are often built into the frameworks we use. So, we even use them without knowing it. They all solve common issues you might encounter when designing, in this case, object-oriented software, like how do I ensure that only a single instance of this class exists, how do I make two objects with a different interface work together, how can I extend an object's interface without changing the underlying object, how do I enable support for undue functionality, and many, many more. That being said, the book was written 1994. Were you to read through it, you will quickly notice that it is not always easy to map the implementation you can see in the book, which is C++ 2.0, to make commonly-used language from 2023, like C++ 12. It wouldn't even be easy to map it onto the current version of C++, which is version 20, or at least you wouldn't be using modern features of that language. So, even though I will cover all 23 Gang of Four design patterns in this course, and I am basing myself on the book, it's definitely not just going to be a copy of the examples from the book. The pattern implementations will all be adjusted to modern-day possibilities, using current, real-life examples you might encounter in your day-to-day life as a developer. The intent of the pattern will, of course, remain the same. As far as some of the use cases, consequences, and related patterns are concerned, not everything from the book is current today, and some consequences or use cases that are common today weren't in the book, where applicable, these will be adjusted. This is very much a current course and not a 1994 course. I mentioned before that it's not that important to learn the exact implementation of these patterns from the top of your head, instead learn which problem they solve, and the problem they solve that relates to the type of the pattern. The Gang of Four defines three types of patterns, let's check those out next.

Gang of Four Pattern Types

The first pattern type is the creational pattern type. It consists of five different patterns, which all deal with object creation. The idea behind these patterns is to abstract the object instantiation process. In other words, these patterns help with making your system independent of how its objects are created, composed, and represented. If you notice you've got a lot of new statements in your client code and classes that are tightly coupled using concrete implementations, these patterns will help with that. The second pattern type is the structural pattern type. It consists of seven patterns. They deal with ways to define relations between classes or objects. In other words, they are concerned with how classes and objects are composed to form larger structures, hence the name. Lastly, the third pattern type is the behavioral pattern type. There's 11 of them. These types of patterns define ways to communicate between classes or objects. They characterize complex control flow that's difficult to follow at runtime, and they let you concentrate on the way objects are interconnected. All right, that's it for the pattern types. Now let's have a look at two important object-oriented principles you will encounter throughout the course.

Object-oriented Principles Refresher

There's two important object-oriented principles that are followed by the Gang of Four in their design patterns. The first one is program to an interface, not an implementation. The second one is favor object composition over class inheritance. You will see these two principles coming back throughout the course in a lot of patterns. Let's start with the first one, program to an interface, not an implementation. Doing this has two main benefits. One, clients remain unaware of the specific types of objects they use, at least as long as the objects adhere to the interface that the client expects. And two, clients remain unaware of the classes that implement these objects. Clients only know about the interface, not about the implementation. Let's clarify this with an example. What you see on screen is a very typical example of this. We've got an interface, `IDiscountService`, and two implementations, `BelgiumDiscountService` and `FranceDiscountService`. And then there's a class, `Client`, that requires a `discountService`. In our example, it gets that through constructor injection, but that's not really that important. Important is that the client doesn't rely on `BelgiumDiscountService` or `FranceDiscountService`. It relies on the `IDiscountService` interface. This is what enables loose coupling. What you have to keep in mind though is that the book was written in 1994. Languages like C# didn't even exist at that time. When we, these days, think about program to an interface and not an

implementation, we will quite quickly think about the interface language feature, just as I've shown you in the code sample. However, what the Gang of Four is talking about is not a technical interface we would define with the interface keyword. They are talking about the set of requests that an object can respond to, in other words, the object's type. Now, of course, there is a close relation to the interface language feature we know about, as it can be implemented as such, but you can also implement the interface they're talking about with an abstract class. It is important to remember that distinction. I mentioned before that these patterns can be implemented in various ways. Using the language feature interface versus the language feature abstract class is one of the main ways that leads to different implementations. On screen, you can see the same example we've just detailed, but this time implemented with an abstract class. Most patterns that talk about the object interface or an interface can, in C#, be implemented with abstract classes or interfaces. Throughout the course, I will use both. To decide on which one to use, a good rule of thumb is use an abstract class to create functionality that subclasses can implement or override. Use an interface when all you need to do is define functionality and not implement it. And where as a class can extend only one abstract class, it can take advantage of multiple interfaces. When a pattern starts working with interfaces, as defined by this principle, you will notice that we will often start adhering to the open/closed principle from the SOLID coding principles. That's the principle that states that software entities like classes, modules, functions and so on should be open for extension, but closed for modification. When that's the case for a certain pattern, we will look into it. Onto the second principle, that was favor object composition over class inheritance. Let's clarify these two terms. Both are techniques fully using functionality in object-oriented systems. Class inheritance lets you define the implementation of one class in terms of another's. You simply create a subclass of a base or parent class, both terms are used interchangeably, and you do that by inheriting the parent class. Like that, you are reusing functionality of the parent class in your subclass or derived class, however you prefer to call it. This is often referred to as white-box reuse, meaning that the internals of the parent class are often visible to the subclasses. And then, there's object composition. Here, new functionality is obtained by assembling or composing objects to get more complex functionality. This requires that objects being composed have well-defined interfaces. This is referred to as black-box reuse because internal details of objects are not visible to the composing class. Both have their advantages and disadvantages, and the debate can get quite heated. This isn't an object-oriented primer course, so I'm not going to get into that

discussion. Moreover, in almost any system, both techniques are commonly used. What is important for this course is that these design patterns will, whenever possible, favor object composition over class inheritance, as the Gang of Four experiences that inheritance as a reuse technique is overused. And when you see some systems that regularly define classes that inherit from each other six or seven levels deep, I tend to agree. That is not a great design. Designs are often made more reusable and simpler by relying on object composition. You will encounter that throughout the course. When a pattern starts working with composition, as is defined by this principle, you will notice that we will often start adhering to the single responsibility principle from the SOLID coding principles as well. That's the principle that states that a class should have one and only one reason to change. In other words, it should only have one responsibility. When that's the case for a certain pattern, we will look into it. And with that, it is time for this module's summary.

Summary

A design pattern is a general, reusable solution to a commonly occurring problem within a given context in software design. You can look at it as a template you can use to solve common issues you will encounter during software design. They help with creating flexible, easily maintainable applications. In that regard, you can consider them as best practice approaches to common problems. The patterns we will cover in this course are the Gang of Four patterns, 23 object-oriented patterns that are still commonly used today. There's three categories. Creational patterns help with making your system independent of how its objects are created, composed, and represented. Structural patterns are concerned with how classes and objects are composed to form larger structures. And behavioral patterns characterize complex control flow that's difficult to follow at runtime and lets you concentrate on the way objects are interconnected. Two principles you'll have to keep in the back of your mind throughout the course as the patterns are built with and around them are one, to program to an interface and not to an implementation, and two, to favor object composition over class inheritance. We just looked into those in detail. Up next, the singleton pattern, a very common one and an easy one, ideal to get used to what a design pattern is.

Creational Pattern: Singleton

Coming Up

In this module, the singleton pattern is covered, a very common and pretty simple one to start with. By starting with this simple pattern, you can already get used to how the modules in this course are structured. First, we'll describe the pattern. We'll do that with the help of a real-life example, a very typical use case for a singleton. We'll do that with the help of a real-life example, a very typical use case for a singleton, a logger. After that, we'll describe the pattern structure and implement it in a demo. Then, we will look into a variation to make the pattern thread-safe. We'll continue with learning about use cases for this pattern and pattern consequences, which can be positive or negative. We'll end the module by looking into related patterns. Let's continue with describing the singleton pattern.

Describing the Singleton Pattern

The singleton pattern is a creational pattern from the well-known Gang of Four design patterns. Creational patterns deal with object creation. The intent of the singleton pattern is to ensure that a class only has one instance and to provide a global point of access to it. If you've got a few years of experience in our sector and have worked with an inversion of control container before, you've undoubtedly encountered it, as registering a type with a singleton lifetime is functionality offered by almost all of these containers. A very common example is a logger. You typically only want one logger instance for your whole application to avoid multiple instances interfering with each other, which could result in messages being logged multiple times or not at all. Now, we could simply hold the instance of the class in a global variable, but there's an issue with this approach. It doesn't prevent clients from creating other instances of the class. That's a responsibility that's too widely distributed to be reliable. Therefore, making the class itself responsible for ensuring that only one instance of the class exists is a better option. Another option is, as mentioned before, relaying the responsibility to another component like an inversion of control container. To make our Logger class responsible for managing its only instance, we should make the constructor private. Like that, other parts of the code cannot directly instantiate our logger. If you want to allow subclassing the logger, a protected constructor is also acceptable. Next to that, the logger will contain a method or a property in a modern language like

C# 12 that will return the instance of itself, for example, an instance property with a private backing field. This property is static, which ensures that when multiple instances of the lower class exists, there will still only be one instance of the instance property. When the property getter is accessed the first time, the instance gets created and stored. When it gets accessed after that, the previously created instance is returned. When it gets accessed after that, the previously created instance is returned, thus ensuring that only one instance exists at any given time. Next to that, a Logger class will typically contain methods like Log. These are defined as regular instance methods, so they're not static. They are accessed via the instance property. Let's have a look at the pattern structure.

Singleton Pattern Structure

Mapping our logger example to the singleton pattern structure is pretty straightforward. We're starting from the diagram we ended up with in the previous clip. The Logger class itself is called the singleton. A singleton defines an instance operation that lets clients access its unique instance. On it, a public static property instance exists. A private backing field contains the stored instance value. And the Log method, that one maps to what is called a singleton operation in the pattern structure. And with that, we're ready to implement the singleton pattern in the upcoming demo.

Demo - Implementing the Singleton Pattern

So, in this demo, we're going to implement the singleton pattern. As mentioned in the introduction module, project files for all patterns have been added to the starter solution. All of these contain a Program.cs file in which we will write the code to test the pattern and an Implementation.cs file in which we will write the pattern code itself. For the singleton pattern, we want to start with a Logger class. That's our singleton. To this class, we add a protected constructor so clients cannot instantiate it, but it can be subclassed. The next thing we want to add is a static Instance property and a private backing field for that property. This must be nullable. In the property getter, we can then check whether it's still null. If it is, we instantiate and store a Logger instance. And afterwards, we always return that instance. That takes care of ensuring that only one instance of the singleton class, our Logger in this case, exists at any given time. We also add one method, Log, for testing purposes. This is called a singleton operation. Every instance of the logger that's returned when getting the instance property

has access to it, and it's always exactly the same instance that is being returned. Let's write some code to test this. First, let's see if we can new this up. `Logger` is defined in the singleton namespace, so let's import that, and apparently we can't. The constructor is inaccessible due to its protection level. So far, so good. What we can do, however, is get an instance via `Logger.Instance`. So, let's get two of those and compare them. If these two are the same instance and they match the `Logger.Instance` property, we write that out to the console output window, and then we log a few messages as an example. Each of these calls should do exactly the same. Let's test this, and there we go. It seems that the instances are indeed the same instance, which means that these three messages written out from the `instance1`, `instance2`, and `Logger.Instance` variables resulted in exactly the same. So this seems to work, but there's a potential issue with this. Let's learn about that next.

Demo - Making the Implementation Thread-safe with `Lazy<T>`

We're looking at the implementation of the singleton class. What we've done here is a naive form of lazy initialization. Lazy initialization is the principle that states that we're only going to create an instance of a class once we need it and not when it's constructed. The problem is that this code is not guaranteed to be thread-safe. We could go and create our own logging mechanism, but there's no need for that anymore. .NET includes a way to deal with lazy initialization in a thread-safe manner using `Lazy<T>`. When we use that, the compiler takes care of making this thread-safe, so let's do that. We add a new static field to replace our static private logger instance. This is of type `Lazy<T>`, in this case, `Lazy<Logger>`. And we state that it's read-only because we won't need to change this object during the singleton lifetime. We immediately initialize it, but we initialize it as a `Lazy<Logger>`. And that does not mean that the logger instance itself will be constructed. Instead, we pass through a method that will be used for constructing the logger when we first access the lazy logger's value property. So we no longer need this old private static logger instance, and we don't need it in the property getter either. Instead, we return the value of our `_lazyLogger`. The first time this is accessed, the logger will be constructed using the action we passed through when initializing the lazy logger. Every time after that, the same instance will be returned, and all of this happens in a thread-safe manner. There's no need to change our client code. Normally, when we try this, we should get the exact same output as in the previous demo. And that is indeed the case, and this time we are working in a thread-safe manner. That's it for this demo. Let's cover the use cases for this pattern next.

Use Cases for the Singleton Pattern

Let's start with a few high-level abstract use cases for this pattern. You can use this pattern when there must be exactly one instance of a class, and it must be accessible to clients from a well-known access point. Our logger is a prime example of that. The pattern is also useful when the sole instance should be extensible by subclassing, and clients should be able to use an extended instance without modifying their code. That is what we enabled by using a protected constructor. Now let's map these rather abstract use cases to some real-world examples. A good example would be managing a database connection pool. In a multi-threaded application, you might want to manage a pool of database connections, and you can use a singleton for that. In that case, you choose the singleton to create and manage the pool. Like that, you ensure that only one instance of the connection pool exists, which prevents resource wastage and ensures that all threads share the same pool. Another example could be caching frequently accessed data. Often, you'll need some sort of caching mechanism in an application. Well, you can implement such a caching mechanism for frequently accessed data by using a singleton pattern. This ensures that there is only one cache instance, which prevents redundant caching and makes it easier to manage cache eviction policies and updates. Another common use case would be managing application configuration settings. When you have a need to deal with application configuration settings, which a lot of applications do, like reading values from a configuration file or from environment variables, well you can also use the singleton pattern. This ensures that the configuration settings are only loaded once and can be accessed globally by different parts of the application. And for the final example, I'd like to take this a bit broader and refer to general resource management in your applications. You can use the singleton pattern to manage resources in general like references to hardware, connections to external devices, connections to external services, and much more. That prevents overuse or conflicts related to those resources. Let's have a look at the pattern consequences.

Pattern Consequences

There's quite a few consequences to this pattern. Because this singleton class encapsulates its sole instance, it has strict control over how and when clients access it. It's also better than global variables because it avoids polluting the namespace with global variables used for storing sole instances. As the class can be subclassed, which is enabled by a protected constructor, you can configure the

application with an instance of the class you need at runtime. And lastly, in case you change your mind and want to allow multiple instances, you can achieve that without having to alter the client because instance management is contained within the singleton itself. But there's also a disadvantage, the singleton pattern violates the single responsibility principle because objects not only control how they are created, they also manage their own lifecycle, and that's not a single responsibility anymore. That's two different responsibilities. As mentioned before, in modern languages, you don't often write code like this anymore, as all of this is mostly dealt with via an inversion of control container. And in that case, it is no longer the responsibility of the singleton class itself to manage its own lifetime. It's the IoC container that takes on that responsibility. Let's have a look at the related patterns.

Related Patterns

Quite a few creational patterns are related to the singleton pattern if only for the reason that they can be implemented as singletons. There's three creational patterns like that, the abstract factory pattern, the builder pattern, and the prototype pattern. Abstract factories, builders, and prototypes can all be implemented as singletons. Next to that, the singleton pattern is also related to the state pattern, as state objects are often implemented as singletons as well.

Summary

In this module, we learned about the singleton pattern. Its intent is to ensure that a class only has one instance and to provide a global point of access to it. A naive implementation can be achieved with a static instance property and backing field combined with a private or protected constructor. However, that is not guaranteed to be thread-safe. A better implementation can be achieved with lazy initialization via `Lazy<T>`. We implemented both as an example. Up next is one of two factory patterns, the factory method pattern. Let's have a look.

Checkpoint 01 - Implementing the Singleton Pattern

Checkpoint 02 - Making the Implementation Thread-safe with `Lazy<T>`

Creational Pattern: Factory Method

Coming Up

In this module, we will cover the factory method pattern. The idea is that this pattern lets a class defer instantiation to one or more subclasses. I'm Kevin, and I'll guide you through it. We'll start by describing the pattern. As a real-life example, we will use a shopping cart checkout context. We'll create a factory with the factory method to provide us with the correct discount service implementations because discounts might be different per country, per code and so on. We will use that example to learn about the pattern structure, and we will implement it. After that, we will continue with learning about use cases for this pattern and pattern consequences. We will end the module by looking into related patterns. Let's dive in.

Describing the Factory Method Pattern

The factory method pattern is a creational pattern from the well-known Gang of Four design patterns. Creational patterns deal with object creation, and using a factory method is a very common way of doing that. It's also known as a virtual constructor. These days, we don't really talk about the factory method pattern a lot anymore. We simply talk about the factory pattern. But if we want to be correct, the factory pattern can refer to either the factory method pattern or the abstract factory pattern, which we'll cover in the next module. Once we've done that, we will compare both, also in the next module. For now, let's focus on the factory method pattern. Its intent is to define an interface for creating an object, but to let subclasses decide which class to instantiate. In other words, factory method lets a class defer instantiation to subclasses. Let's clarify this with an example. Imagine that you're creating part of a shopping cart system. At a certain moment, a discount can be applied. One of the services that can provide a discount is a service that provides a discount depending on the country the user is in. Another one provides a discount depending on the discount code. So, both services are not constructed in the same way nor is the logic to calculate the actual discount the same, yet they do have something in common, the `DiscountPercentage` that needs to be exposed. We could write some

code that news these classes up and gets the DiscountPercentage. That's what you see on screen, yet this creates tight coupling between the Client class, which is instantiating them, and the different discount service implementations. The factory method pattern helps avoid that. We do that by adding a base class or interface named DiscountService. It's this class that any part of the application that needs to know the discount can interact with. The concrete implementation isn't important for the client. Now, the client, which is part of the shopping cart system that needs to know the discount, does not need to know which actual subclass it needs to instantiate. It doesn't care whether it's the CodeDiscount or CountryDiscountService. All it needs to know is when it needs such an instance. But how do we instantiate it then? For that, we create a base DiscountFactory with one abstract method, CreateDiscountService. This is called the factory method. Concrete implementations of this factory are then a CodeDiscountFactory and a CountryDiscountFactory. The client works on the DiscountFactory base class, so it uses whichever implementation is provided to get a hold of the concrete implementation of the DiscountService via the concrete implementation of the DiscountFactory with the concrete implementation for the DiscountFactory method. Via the factory method, we encapsulated the logic to create the actual subclass, so the client doesn't need to know about that anymore. And because we're working on an interface instead of a concrete implementation, we prevent tight coupling. Let's map this to the pattern structure.

Factory Method Pattern Structure

When we map this to the pattern structure, we start with the product. That could be an abstract base class or interface for the thing that needs to be created. It just defines the interface of objects that the factory method creates. In our case, that's the DiscountService class. Then, we have ConcreteProducts, the implementations of that product interface. If we map that to our example, that's the CodeDiscountService and CountryDiscountService. The creator then declares the factory method. This factory method must return an object of type Product. In our case, that's the DiscountFactory with the abstract CreateDiscountService method on it. We can map that to factory method. The creator itself may also call the factory method to create a Product object. And lastly, we have the ConcreteCreator. These are implementations of Creator, which override the factory method to return an instance of ConcreteProduct. Map to our example, these are the CodeDiscountFactory and

CountryDiscountFactory. And with that, we've got our pattern structure. Let's implement our example in a demo.

Demo - Implementing the Factory Method Pattern

Time to implement the factory method pattern. Let's start with our product, the DiscountService. This is an abstract class, as we want it to serve as an interface for a concrete implementation. On it, we can add properties or methods like the DiscountPercentage that can be used by clients. This too is marked with the abstract keyword, as we want the concrete products to implement the logic behind this. Note that as with most pattern implementations that require an abstract base class, you can use an interface as an alternative. As an example, I will use an interface in the next module when we'll look into the abstract factory pattern, a pattern closely related to this one. We also override the ToString method so it returns the actual type name. That'll make it easier to see what's going on when we test this, then let's implement this. We add a ConcreteDiscountService, which implements our DiscountService abstract class. This is a ConcreteProduct. Depending on the passed-in countryIdentifier, a different percentage must be returned. So, we store the identifier in a read-only field. We do not want it changed after the object has been constructed. Then we override the DiscountPercentage, as it's up to the concrete implementation to decide on this. Let's say that for this demo, when you're from Belgium, you get a better discount. Purely coincidental, of course, the fact that I happen to live in Belgium has got nothing to do with this at all. Our next implementation then is a CodeDiscountService. We start in a likewise manner. The DiscountService is implemented, and we override the DiscountPercentage property. Let's say that each code results in the same DiscountPercentage, but a code can only be used once. In real life, this is where you could add that check. We're not going to implement that here because what is important for us is that we ended up with two different concrete products. Up next, the factories. As we learned, we start with an abstract DiscountFactory class with one abstract method, CreateDiscountService, which returns a DiscountService, the base class for our concrete discount services. And then we need to implement the concrete implementations. CountryDiscountFactory news up a CountryDiscountService for which it needs the _countryIdentifier. So it's this factory that's responsible for creating a CountryDiscountService. And in a likewise manner, our CodeDiscountFactory is responsible for creating and managing the lifetime of the CodeDiscountService, passing through the code it gets from

the CodeDiscountFactory constructor. And with that, we've got all the moving parts of our pattern in place. We can test this. Let's switch to the Program.cs file. And first, let's create our two factories. For that, we need to import the Factory Method namespace. We store these in a list, which makes it easy to run through them to see what's going on. It's through these factories that the actual services can be created. So that's what we do next. We run to our list of factories, and on each factory, we call CreateDiscountService. That means that this client is doing the call on the DiscountFactory base class. And then we write out the percentage and where it's coming from. Like that, we can see what was actually created. Important to remember here is that the client does not know about any concrete implementations. The factories it works with are abstract and so are the discount services. Let's give this a try. And there we go. Two concrete products were created by our two concrete factories without the client having to know about concrete implementations. That's it for the demo. Let's cover the use cases for this pattern next.

Use Case for the Factory Method Pattern

Good use cases for this pattern are when a class can't anticipate the class of objects it must create, in other words, when you don't know in advance which types of objects your code should work with. When that's the case, the concrete creators can take on this responsibility. Another good use case is when a class wants its subclasses to specify the objects it creates. This enables the scenario in which standard components or classes can be extended via subclasses. Here too, this means the concrete creator takes on the responsibility for creating the objects. Up next, when classes delegate responsibility for creation to one of several helper subclasses and you want to localize the knowledge of which helper subclass is the delegate, localization of that knowledge is now at the level of whichever part of your application is responsible for providing the concrete implementation of the creator that's needed. And you can also use it as a way to enable reusing of existing objects. The logic inside of the factory method can take care of either creating an object or reusing an existing one as it sees fit. If you're newing up instances yourself, you'll either have to write code like that wherever you new it up or put it in a reusable method. And that reusable method, that's the factory method. Let's cover the consequences next.

Pattern Consequences

A main consequence of this pattern is that factory methods eliminate the need to bind application-specific classes to your code, to the client, for example. And that's because the code only deals with the interface, the product, and not with the concrete implementation. That means it can work with any concrete product implementation. In other words, you're avoiding tight coupling. It's this that enables adding new types of products without breaking client code, and that's the open/closed principle at work. The single responsibility principle is also adhered to. Creating the products is moved to one specific place in your code, the creator. That's only responsible for that. A potential disadvantage is that clients might need to create subclasses of the Creator class just to create a particular ConcreteProduct object. If only one subclass, so one concrete creator is expected to be needed, you would've just added unnecessary complexity. Let's have a look at the related patterns next.

Related Patterns

Closely related to this pattern is the abstract factory pattern. It's often implemented with factory methods, as we will see very soon in the next module. There's also a relation to another creational pattern, the prototype pattern. Prototypes don't require subclassing creators. They are not based on inheritance, which is a plus when you favor composition over inheritance, which, as we learned, the Gang of Four do. However, they often require an initialize operation on the Product class, which the factory method doesn't. So, factory method is based on inheritance, but it doesn't require the initialization step. Lastly, there's also a relation to a behavioral pattern. Factory methods are usually called within template methods. On to the module summary.

Summary

In this module, we learned about the factory method pattern. Its intent is to define an interface for creating an object, but to let subclasses decide which class to instantiate. Factory method lets a class defer instantiation to its subclasses. We implemented this with a DiscountService example. Important here is that this pattern eliminates the need to bind application-specific classes to your code, to your client, for example. You will notice throughout this course that a lot of patterns will also help with this. As far as implementation goes, in C#, the product can be implemented as an interface or abstract

class. Same goes for the creator itself. On its concrete implementation, a method must be implemented to create a ConcreteProduct. That's the factory method. Up next is a closely related pattern, the abstract factory pattern.

Checkpoint 03 - Implementing the Factory Method Pattern

Creational Pattern: Abstract Factory

Coming Up

The intent of the abstract factory pattern is to provide an interface for creating families of related or dependent objects without specifying their concrete classes. In this module, you will learn what that means. I'm Kevin, and I'll guide you through it. We'll start by describing the pattern, when to use it, and why. After that, we'll dive into the pattern structure. As a real-life example and implementation, we will continue with our shopping cart example from the factory method module. We will also compare the abstract factory pattern to the factory method pattern. After that, we'll continue with learning about use cases for this pattern, pattern consequences, and the related patterns. Let's dive in.

Describing the Abstract Factory Pattern

The abstract factory pattern is a creational pattern from the well-known Gang of Four design patterns. It's closely related to the factory method pattern, which we covered in the previous module. It's even often implemented by using factory methods. The abstract factory pattern is sometimes also referred to as the factory pattern, and so is the factory method pattern, which we previously covered, so this might be a bit confusing. Because of that, we will look into how they compare further on in this module. For now, let's focus what this module is all about, which is the abstract factory pattern. We will do so by working with the likewise context, as we did when we learned about the factory method pattern, the context of a shopping cart application. The intent of the abstract factory pattern is to provide an

interface for creating families of related or dependent objects without specifying their concrete classes. When we looked into the factory method pattern, we used a `DiscountService` example to clarify it. We can extend that example to get to a good use case for the abstract factory pattern. In the example for the factory method pattern, we stated that everyone from Belgium got a 20% discount while everyone else got a 10% discount. This is something typically used in a shopping cart scenario. But if you want to make that scenario a bit closer to real life, you need more than just a discount service that's dependent on the country. Shipping costs will also differ by country, and these two belong together. We could implement that by simply newing this up. But then we do run into the same issue as we had before implementing the factory method pattern, tight coupling. Moreover, we don't convey the intent that these objects belong together. So, the code example you see on screen now can definitely be improved. In fact, this would be a really good use case for a factory that creates a family of related objects, `IShoppingCartPurchaseFactory`. The interface it confirms to is the `IShoppingCartPurchaseFactory` interface. It contains methods to create an `IDiscountService` `CreateDiscountService`, and to create an `IShippingCostsService` `CreateShippingCostsService`. This is the family of services required for one purchase. It also means we need to define these interfaces, an `IDiscountService`, which exposes a `DiscountPercentage` and an `IShippingCostsService`, which exposes the `ShippingCosts`. Note that this time, we're working on interfaces by which I mean the language feature and not on abstract base classes. If you prefer an abstract base class, you can use that as well. I already used that approach when explaining the factory method pattern, so this time I'm going to go for an interface-based approach. Like that, you can get a taste of both. From my experience, most of the time these days interfaces are used. If you need to provide some basic functionality that can potentially be overridden, you could go for the abstract base class route, otherwise interfaces will do. They already allow you to specify the expected functionality of a class. Anyway, concrete implementations could then be a `BelgiumShoppingCartPurchaseFactory` with related `BelgiumDiscountService` and `BelgiumShippingCostsService`. The responsibility for creating these concrete service implementations lies with the `BelgiumShoppingCartPurchaseFactory`. It exposes this functionality via `CreateDiscountService` and `CreateShippingCostsService` respectively. Note that these still return the interfaces, so `IShippingCostsService` and `IDiscountService` and not the concrete implementations. Another implementation could be a `FranceShoppingCartPurchaseFactory` with related `FranceDiscountService` and `FranceShippingCostsService`, which follow a likewise logic.

On top of that, we then have a client, the ShoppingCart, which uses one of these shopping cart factories to create the two services it needs. The shopping cart works with an `IShoppingCartPurchaseFactory`. Again, an interface and not an implementation. This allows it to work with any concrete implementation. A common way to provide the concrete factory implementation is via the constructor. As an example, we could add a method to this client, `calculate costs`, which calculates the actual purchase costs using the two concrete service implementations. Seeing the client works on the interface using one of these factories or adding a new factory like the France purchase factory we covered can be done without having to change the client, and that is because the contracts, the interfaces, remain the same. The client doesn't know nor need to know about the concrete implementations, and that is one of the main reasons for using this pattern. The client is decoupled from the concrete implementation. All right, and with that, we've looked into the description of the abstract factory pattern. Before we implement this, let's map this to the pattern structure.

Abstract Factory Pattern Structure

The first player in the pattern structure is the `AbstractProduct`. It declares an interface for a type of Product object, maps to our example. `IDiscountService` and `IShippingCostsService` are `AbstractProducts`, and now we've got the `ConcreteProduct`. This defines the product that has to be created by a corresponding factory, and it implements the `AbstractProduct` interface. In our example, these are the implementations of `IDiscountService`, `BelgiumDiscountService`, and `IShippingCostsService`, `BelgiumShippingCostsService`. Note that we note these two down as belonging to the same family of products, family 1. Were we to work with another family of products, for example, France, as we've looked into before, we would map those to `ConcreteProducts` of family 2. Up next is the `AbstractFactory`, which lends its name from the pattern and declares an interface for operations that create `AbstractProduct` objects. In other words, this is our `IShoppingCartPurchaseFactory` interface. The methods on it are `CreateProductA` and `CreateProductB`, which return the respective abstract products. The implementations of this `AbstractFactory` are called `ConcreteFactory`. These are responsible for effectively creating the related `ConcreteProducts`. Note that these methods create `ConcreteProducts`, but return them as `AbstractProducts`, which it can because `ConcreteProducts` implement `AbstractProduct` interfaces. Mapped to our use case, the `BelgiumShoppingCartPurchaseFactory` maps to `ConcreteFactory1`, and the

FranceShoppingCartPurchaseFactory would map to ConcreteFactory2. And lastly, there's the client that's the part of our application, which requires an AbstractProduct implementation. Important is that it should only use the abstract implementations of product and factory and not a concrete implementation to keep it decoupled from them. It thus only uses interfaces. And like that, we have fully mapped our example to the pattern structure. Let's implement this in a demo.

Demo - Implementing the Abstract Factory Pattern

Let's start our implementation with adding our abstract factory. For our use case, that was the IShoppingCartPurchaseFactory interface. This interface describes how to create a family of related abstract products. For our use case, it exposes two methods, CreateDiscountService, which returns an IDiscountService interface, we still need to create that, and CreateShippingCostsService, returning an IShippingCostsService interface, which we also still need to create. Note that we are not working with any concrete implementations here. We won't implement this yet. Let's first add these abstract products so this resolves again. We start with the IDiscountService interface, which will have exposed a DiscountPercentage. Our IShippingCostsService then exposes a ShippingCosts decimal. As you notice, these are different contracts, then let's implement these. First, we need two implementations of IDiscountService, one ConcreteProduct, BelgiumDiscountService, which sets the DiscountPercentage at 20, and one ConcreteProduct, FranceDiscountService, which sets the DiscountPercentage at 10. So as you can see, we are starting to create families here. The BelgiumDiscountService will belong in one family, and the FranceDiscountService will belong to another. And we create likewise ConcreteProducts for our AbstractProduct, IShippingCostsService. The BelgiumShippingCostsService implementation of IShippingCostsService sets the ShippingCosts to 20, and the FranceShippingCostsService sets it to 25, and then we can start implementing our factories. As we remember, we need one factory per family. So, the first implementation of IShoppingCartPurchaseFactory becomes a BelgiumShoppingCartPurchaseFactory. This is responsible for creating the concrete BelgiumDiscountService and BelgiumShippingCostsService implementations. Yet, what they return are still the interfaces we work on, and we create a likewise concrete factory implementation for our France family, the FranceShoppingCartPurchaseFactory. As you can see, this is responsible for creating a family of concrete implementations of IDiscountService and IShippingCostsService, yet the methods themselves work on the underlying interfaces. And then

we need a client. Let's call that our shopping cart. We add two fields for our services to get to our family of products. These are our two abstract products. When we construct this shopping cart, we want to convey that one certain family of products should be used, and there's no mixing and matching. That's why we create the `DiscountService` and `ShippingCostsService` using the same factory. You can also let the client new up the factory to use, which is a common way of handling this. Important, however, is that this implementation should remain composition-based. In this context, that means that if you want to change the factory, it's a matter of changing the factory object by replacing it with another factory object. We'll also add a field to store the total costs of the order, and we're going to assume that the products we ordered have a total cost of €200. What will be added to this are the costs related to our discount and shipping cost services. And now we add one more method, `CalculateCosts`, which will make it easy for us to test this. In this method, we write out the `_orderCosts` minus the discount for which we use the `DiscountPercentage` from our `_discountService` plus the `ShippingCosts`. And that's it for our pattern implementation. Let's write some code to test this. The first thing we need to do is new up a factory because our shopping cart requires that. For that, we import the `AbstractFactory` namespace. And then our client application news up the shopping cart, passing through the factory that is responsible for creating the concrete implementations. And now we call `CalculateCosts` on our shopping cart, which will write out the actual costs to the console output window. And we do the same for another family of services, the France family. All right, let's give that a try. And there we go. The total costs are written out to our console output window, and they are different, meaning that the different factories that we used resulted in different concrete products, our services, being created and eventually used. But what are the use cases and advantages of all this code? Let's have a look.

Use Cases for the Abstract Factory Pattern

Let's start with a few high level, more abstract use cases for the abstract factory pattern. The first use case is when a system should be independent of how its products are created, composed, and represented. Our real-life sample nicely illustrated this. The client doesn't know about these concerns. Closely related to this is the case where you want to provide a class library of products and you only want to reveal their interfaces and not their implementations. The decoupling between client, and factory, and products is a win in this regard. Another one is when a system should be configured with

one of multiple families of a product. As we learned, the abstract factory deals with product families, products that belong together. In that way, it's different from the factory method pattern. And lastly, related to that, use it when a family of related product objects is designed to be used together and you want to enforce this constraint. All right, so now let's map these rather abstract use cases to a few additional real-life examples. A good use case would be when you want to support multiple languages. Often, when you're developing software for a global audience, you may need to support multiple languages. In that case, an abstract factory can be used to create language-specific resource loaders, string formatters, datetime formats and so on. That ensures that the application can adapt to different language and culture requirements. Another use case would be when you need to support converting documents to different formats. Imagine a document conversion tool that can convert various document formats like PDF, Word, HTML into different output formats like PDF, HTML, plain text and so on. You can use an abstract factory to create the document parser and exporter components for each input/output format combination. Like that, it becomes easier to add support for new formats. Another use case would be abstracting away your database access layer. In a lot of applications, you'll need to develop a database access layer. Sometimes, you may need to support multiple ORMs or multiple databases, for example, MySQL, SQL Server, and SQLite. An abstract factory can then provide a way to create database-specific connections, queries, and data access objects, which abstracts away the underlying database implementation. So, if you feel the need to write an ORM like Entity Framework Core or NHibernate from scratch, you will most definitely run into this. And then the last use case has to do with supporting different application themes or styles. Often in software applications with user interfaces, users want to customize the look and feel of the application or maybe you're working on some sort of multitenant-aware application in which each tenant will have its own look and feel. In that case, you can use an abstract factory to create theme-specific UI elements like buttons, colors, fonts and so on. Like that, users can switch between different styles easily, or you can have one style per tenant or in the case of multitenancy, switch depending on the tenant. That's it for the use cases. Let's look into the pattern consequences next.

Pattern Consequences

This pattern, as most, has a few benefits and liabilities. First, it isolates concrete classes because it encapsulates the responsibility and process of creating product objects. In other words, clients are

isolated from implementation classes. And that's mostly an advantage. It means we are avoiding tight coupling. You can thus easily introduce new products without breaking client code. That means we are adhering to the open/closed principle from the SOLID coding principles. Also, seeing the codes to create products is contained in one place, we are adhering to the single responsibility principle. This pattern also makes exchanging product families easy. Seeing the class of a concrete factory only appears once in an application where you instantiate it, it's easy to change that concrete factory. You can switch it out for another one by changing that one line of instantiating code, as the rest of the system works on the interface and not on the implementation. It also promotes consistency among products. This is because products in the same family are created by the same factory. Seeing an application only uses one abstract factory at a time, the products that are used are related and compatible with each other. A potential disadvantage is that supporting new kinds of products is rather difficult. If you want to extend the abstract factory, you have to change the factory interface, which also means changing the implementation and all of its subclasses. I've mentioned it a few times before, the abstract factory pattern is closely related to the factory method pattern, yet they are not the same. Let's compare them in the next clip.

Comparing the Abstract Factory Pattern to the Factory Method Pattern

Both the abstract factory pattern and the factory method pattern are sometimes referred to as the factory pattern. As they are creational patterns, they're both about creating objects. But there's more than one way of doing that. In fact, two more creational patterns are coming up. So why is it that these two patterns are so closely related? Let's start with the factory method pattern. In essence, this means we are creating an interface with a method on it, the factory method, to create an object of a certain type. Yet, what we work on, the object that is being returned, is never a concrete type. It's an interface or abstract base class. Next to that, it only produces one product, and it creates these objects through inheritance. It implements the interface or abstract base class with the factory method on it. And now we have the abstract factory pattern. The abstract factory pattern, on the other hand, is signified by exposing an interface to create related objects, families of objects. For that, multiple methods are exposed on the abstract factory. Here too, just like in the factory method pattern, the return types of these methods are interfaces or abstract classes. It thus produces families of products, and it does that via composition instead of via inheritance. The abstract factory contains the fields that hold the

concrete factories that are used for creating the concrete products. And the relation is even closer than this. Very often, the abstract factory pattern is implemented with factory methods. So, when I or most people talk about the factory pattern, that's exactly what is meant, a combination of the abstract factory with the factory method pattern. Let's have a look at the related patterns.

Related Patterns

So, abstract factory classes are often implemented with factory methods, but they can also be implemented by using the prototype pattern, which is another creational pattern that's coming up. Also, a concrete factory, so an abstract factory implementation, is often implemented as a singleton to ensure that only one instance exists throughout your application, and they cannot interfere with other instances. Let's have a look at the module summary.

Summary

The abstract factory pattern is a creational pattern from the well-known Gang of Four design patterns. It's closely related to the factory method pattern. It's even often implemented using factory methods. Its intent is to provide an interface for creating families of related or dependent objects without specifying their concrete classes. The main advantage here is that clients are isolated from implementation classes, in other words, decoupling. As far as implementation goes, this isn't the easiest pattern to implement. The most important part, however, is to keep working on interfaces or abstract base classes. When returning concrete products, return them via the underlying interface. Up next, we will look into another creational pattern, the builder pattern.

Checkpoint 04 - Implementing the Abstract Factory Pattern

Creational Pattern: Builder

Coming Up

In this module, we will cover the builder pattern. The intent of the builder pattern is to separate the construction of a complex object from its representation so the same construction process can create different representations. We'll start by describing the pattern, when to use it, and why. After that, we will look into the pattern structure. As a real-life example, we will create a vehicle builder, which can be used to create cars and motorcycles. We'll continue with learning about use cases for this pattern and pattern consequences, which can be both positive or negative. We'll end the module by looking into related patterns. Let's dive in.

Describing the Builder Pattern

The builder pattern is a common creational pattern from the Gang of Four. As it's a creational pattern, it deals with object creation, complex objects in this case, because the intent of the builder pattern is to separate construction of a complex object from its representation. In other words, it lets us construct complex objects step by step, and by doing so, the same construction process can create different representations. Let's explain that definition with a real-life example. A very typical example of the builder pattern in real life is a piece of software for a garage that at a certain point requires you to construct cars. We can consider such a car as a complex object consisting of different parts, say a frame and an engine in this example, but not all cars are constructed in the same manner. A three-door Mini has a different frame and engine than a five-door BMW, for example. So, the builder for these is different. This is a prime example of where the builder pattern shines. In this example, the BMW is a representation of a car, and the Mini can be seen as another one. So, we will have a builder for a car with on it methods to build the frame, build the engine and so on. It also needs to expose a way to return a car, for example, via a car property. Seeing we are returning a car, we will also have a Car class. On this class, we will have a method to add a part, for example, adding an engine, adding a frame and so on. How the Car class adds and stores these parts isn't of importance for our pattern implementation. And then we will have different implementations of this car builder, one for building a BMW, one for building a Mini and so on. Important here is that these different implementations still have the same methods. So creating the car, which is a complex object, is transparent for the consumer of the builder. No matter which builder implementation is used, it has to call into the same methods. This is possible because just as with other patterns in the creational pattern category, we are working on interfaces and not on implementations. We've again avoided tight coupling. The consumer,

in our case, would be a garage. It needs a builder to effectively construct a car. One way of passing that through is via the construct method, which accepts a car builder or via a constructor injection. And that's it as far as our real-life example goes. Now let's map this example to the pattern structure.

Builder Pattern Structure

The first part of the pattern structure is the builder itself. This specifies an abstract interface for creating parts of a Product object. In our case, this thus maps to our car builder. It's an interface for creating parts of a car. Then we have the ConcreteBuilder. These are implementations of that interface, our BMW builder and our Mini builder. They construct and assemble the product, which is a car, and they find and keep track of the representation it creates. That's that Product again, it also provides a way to return the Product. There we go. And then we have the Product itself. This represents the complex object under construction, a car in our case. It potentially includes classes that represent the constituent part. In our case, that could be a frame or an engine and so on. And lastly, we've got the Director. The Director is what constructs an object by using the Builder interface. In our example, we could map that to, say, a garage. In some implementations, the director allows setting the Builder interface to use via a property. In others, it allows passing it through via the Construct method. These are implementation details we can decide on ourselves. Let's do that in the demo.

Demo - Implementing the Builder Pattern

Before we start coding, remember that, as always with these design patterns, we should see them as a template. It's important to stick to the intent of the pattern, but implementations can vary, as you'll see. We're going to start with the product. That's the car. On it, we'll define a set of car parts as a `List<string>`. This is for demo purposes to avoid having to create separate classes for engine, for frame, etc. Each car also has a type, which we'll accept via a constructor. Like that, we can convey meaning to consumers, letting them know that we expect a non-null car type, and then we add one method, `AddPart`. This method will be used to add parts to our part list, in other words, to construct the car. And lastly, we overwrite the `ToString` method simply to make it a bit easier to test. In this `ToString` method, we write out the type of the car and the parts. Up next is the Builder class, which is the abstract base class for concrete builders. It has a `Car` property. That's the product it's building. We

initialize that in the constructor by passing through the carType. Important is that we also define two abstract methods on this car builder, BuildEngine and BuildFrame. These are the methods, which will have to be implemented by concrete builders to actually build the engine and build the frame. And that brings us to the concrete builders. Let's start with a builder to build a Mini. It implements the CarBuilder. Let's do that, and that means we have to implement the BuildEngine and BuildFrame methods. But let's start with the constructor. We construct this MiniBuilder by passing through the carType to the base constructor. That's something we can do thanks to the fact that CarBuilder is an abstract base class and not just an interface. The BuildEngine implementation is then responsible for adding the engine to the car. Car is defined on the CarBuilder base class, and BuildFrame is responsible for adding a frame. In our case, that's just a string in a more extensive implementation. You would effectively use classes for this. And I will do something likewise for a second concrete builder, a BMWBuilder. We invoke the base class constructor, and we add the implementations for BuildEngine and BuildFrame. Lastly, we have the director, a Garage. A director needs a builder to effectively construct something, a car in our case, so we store that in a field. Then we need to be able to inject that builder somehow. And for this example, we will do that via a construct method. The builder is thus injected, stored, and using the builder, the car is built. Seeing we're working on an interface here, the CarBuilder abstract base class, and not on a concrete implementation, we can inject any concrete builder. Now let's test this. First, we new up the director. That's our Garage. To be able to do that, we need to import the BuilderPattern namespace, and then we need instances of our builders because the construct method on the Garage expects that. So we create a MiniBuilder and a BMWBuilder instance. And to test this, we call the Construct method on the garage, passing through the miniBuilder and bmwBuilder respectively. A small variation. What's often done with this pattern is adding a slight extension. To avoid having to call into the car to visualize it, you can add a method on the Garage class to show it. So let's add a method, Show, on our Garage. I'll probably repeat it a few times throughout this course. The basics of the pattern are the basics. That's the template, but you can extend it by, for example, adding a Show method to the director in this case. And that allows us to call that Show method from our Client class. And there we go. Let's give this a try. The first time we're using the Car object directly to write this out, the second time it's garage.Show. And what we see is that even though we call Construct on the same director, our garage, it's thanks to the fact that we

inject a different builder that a different car is being built. Let's have a look at the use cases for this pattern.

Use Cases for the Builder Pattern

Let's start with some abstract high-level use cases. You can use the builder pattern when you want to make the algorithm for creating a complex object independent of the parts that make up the object and how they're assembled. A Mini or BMW is a complex object to build, and it is the builder that becomes responsible for constructing it, not the director. You can also use it when you want the construction process to allow different representations of the object that's constructed. In our case, we want to build a car. Different builder implementations allow for different representations of that car. These are rather abstract use cases, so let's map them to something you might use or some use cases you may encounter during application development. A good first example would be generating documents. In applications that generate documents such as PDFs or reports, you can use a builder pattern to construct the document step by step. You can define builders for headers, for footers, for content, and formatting, allowing you to create different types of documents with various structures. For those of you who have been following all the modules in order, you may notice that this use case in regards to generating documents, exporting documents, word processing, etc, these types of use cases tend to come back. That means that in one type of application, you often have multiple design patterns working together, or you could even replace one pattern with another one, depending on your requirements. Throughout the course, you will see that this becomes more obvious by looking into use cases for each pattern. Another use case would be building a database query. When constructing complex database queries, you can use the builder pattern to create a query builder class. That builder can then help construct SQL queries with various conditions, sorting options, and filtering criteria, making it easier to create dynamic and complex queries. Again, a use case you may have encountered before. And this also means that underneath the covers, ORMs like Entity Framework Core could use builder patterns for this. Another pretty cool example would be game character creation. In case you're lucky enough to be in video game development, you'll typically run into having to support character creation. And that often involves many customisable attributes such as appearance, skill, and equipment. A builder pattern can be used to create game characters with these specific attributes. So, instead of building BMWs and Minis, you would build druids, and assassins,

and maybe even a paladin. The last use case is constructing a user interface or a form. When you need to create a graphical user interface from code, especially in frameworks that allow for custom UI components, a builder pattern can be used to construct complex UI components. Or very much related to this, you could have a form builder to help create forms with various input fields and various validation rules. So as you can see, the builder pattern is a good fit for quite a few use cases. Let's have a look at the consequences of this pattern next.

Pattern Consequences

One of the basic consequences of the builder pattern is that it lets us vary a product's internal representation. As you know, the builder provides the director with an interface for constructing the product. Like that, the interface lets the builder hide the representation and internal structure of the product. This also makes it easy to change a product's representation. All you need is a new kind of builder. Next to that, it isolates code for construction and representation. This is a single responsibility principle from the SOLID coding principles at work. It thus improves modularity by encapsulating the way a complex object is constructed and represented. Clients don't need to know anything about the internal structure of the product. The pattern also gives us finer control over the construction process. The product is created in different steps instead of in one go, and each of these steps is under the director's control. A negative consequence is that the complexity of our code increases because we're often creating multiple new classes. There's nothing inherently wrong with complexity, but the main thing you want to avoid is complexity when it is not necessary. Unnecessary complexity, in my opinion, is one of the main issues you see in a lot of modern day architectures and application design. Let's have a look at the related patterns next.

Related Patterns

As you can guess, this pattern is related to a few other creational patterns, one being the abstract factory pattern, as builders and abstract factories can both be used to construct complex objects. The main difference is that the builder constructs the complex objects step by step. Depending on your use case, that may mean that you can use an abstract factory or a builder to achieve what you need. The builder is also related to the singleton pattern, as a builder can be implemented as a singleton. And

there's one structural pattern it's related to as well, the composite pattern, as composites are often built by builders. Time for the module summary.

Summary

The intent of the builder pattern is to separate the construction of a complex object from its representation, so the same construction process can create different representations. It is a common creational pattern from the Gang of Four that's useful when you want to make the algorithm for creating a complex object independent of the parts that make up the object and how they are assembled. We used a garage and cars as a real-life example. Important points from the implementation are define an abstract base class or interface as Builder and have the director work on the Builder and not on a concrete object and have the director work on the builder and not on ConcreteBuilder implementations. In other words, let it depend on an interface and not on an implementation. Up next is the last creational pattern, the prototype pattern.

Checkpoint 05 - Implementing the Builder Pattern

Creational Pattern: Prototype

Coming Up

In this module, we will cover the prototype pattern. Its intent is to specify the kinds of objects to create using a prototypical instance and create new objects by copying this prototype. We will start by describing the pattern and looking into the pattern structure. As a real-life example, we will use the creation of an employee and a manager, both deriving from a person, via a clone method. After that, we'll dive a bit deeper into shallow versus deep copies, which we will both implement. Manager has a list of employee, which allows us to use the Manager class as a deep clone example. We'll cover the ICloneable interface as well, and we will learn about use cases for this pattern, pattern consequences, and related patterns. Let's dive in.

Describing the Prototype Pattern

The prototype pattern is one of the Gang of Four design patterns, and it belongs to the creational patterns group. That means that it deals with object creation. It's the last one of these creational patterns. The intent of the pattern is to specify the kinds of objects to create by using a prototypical instance and create new objects by copying this prototype. Feels a lot like cloning, right? And that is indeed what it is often used for. So in other words, it lets us copy existing objects without making the client code dependent on their concrete classes. It's a pretty easy pattern, as you'll see. But as with most of these pattern definitions, they're easier to understand with an example. So let's dive a little bit deeper into that definition. Imagine that we have a system in which we work with people. We can simply create people by newing up instances of a Person class, or probably a bit more realistic, assuming that person is an abstract base class, an Employee or Manager class that derives from Person. Now imagine the use case where most property or field values of an existing object are required in a new object. Or maybe you even need all property values of the new object to match the existing object. In that case, we can simply copy them over, but that requires the client to have intrinsic knowledge of the concrete classes we're working with, Employee and Manager, and how to create them. We can go about this in another way as well. We could create a method on our Employee and Manager classes, Clone, that will return a copy of itself. Moreover, as we know that both Employee and Manager derive from Person, we can define this method on the Person class and override it in its subclasses. The Person class is an abstract base class, so in Gang of Four lingo, an interface. And the client application then only needs to know about the Person's class clone method to be able to clone subclasses like Employee and Manager. What you see here is the prototype pattern at work. In essence, it describes a way to create a new object by cloning an existing one. Let's map this example to the pattern structure.

Prototype Pattern Structure

The prototype pattern contains a Prototype class. That's a class that declares an interface for cloning itself. It maps to our Person class. A way to implement this would be as an abstract base class with an abstract clone method on it. But as is often the case, an interface is possible as well. Then we've got the ConcretePrototype. These are implementations of the prototype that implement the actual clone

operation, which returns a copy of itself. And lastly, we have the Client. This one creates a new object by asking the prototype to clone itself. Let's implement this pattern using our real-life example.

Demo - Implementing the Prototype Pattern

So, time to implement our pattern. Let's start with the prototype itself first. We're going to use an abstract base class for that, defining a Name property and a Clone method, which returns a Person object. That's the prototype, a clone of itself. And now we need to implement this. Let's start with a Manager class deriving from Person. That means we need to implement the abstract base class, so let's do that. And we see that we need to provide a name. What we can do for that is add a constructor that accepts that name, thus conveying that we expect this object, the Manager, to be constructed with a name. And there we go. We can now simply change this to getters and setters via auto properties syntax. In the constructor, we set the property name to the passed-through name. And now we also need to implement the Clone method. We could implement this by newing up a Person and manually copying over all the fields, but there's an easier, built-in way to do this by calling into MemberwiseClone. This is a method that's defined on object, so every instance you use in C# has access to this. It creates a shallow copy of the existing object. We'll get to what shallow copy means after the demo. For now, this is sufficient for our Manager class. Then let's add another ConcretePrototype, Employee. It's more or less the same as the Manager class with one difference, an employee has an additional property, its manager. So we construct this one by accepting a name and a Manager object. Let's test this. Let's new up a manager first. For that, we need to import the Prototype namespace. We'll pass through a name, Cindy, and then we'll try to clone this manager. For that, we can call Clone on the Manager object. Note that we do need to cast this back to a manager if we want to use it as a manager because the Clone method returns a Person object, the abstract base class. And then we do the same for Employee. The difference here is that we pass through the manager we just created, and let's give that a try. And that seems to work just fine. The manager clone's name was written out, and that is Cindy, which means cloning the manager seems to work. Employee was cloned, which worked out just fine, and the manager is Cindy. Makes sense because that's what we passed through. If we now pass through the managerClone instead, we should see the exact same result. Let's give that a try, and that is indeed the case. So this seems to work well, but let me make one small change. Right before the ReadKey statement, let's change the manager name

and write out the employee info again. Let's give this a try, and that's a bit odd. The manager name on the employee clone was changed, even though the clone was created before we changed the name. That may be what you want, but often it's not. You might expect to get an actual copy of the Manager object as well when you clone the Employee object. This is a limitation of MemberwiseClone. That statement always creates shallow copies. Getting an actual copy of the Manager object as well would mean we need support for deep copies or clones. Let's compare these.

Shallow Copy vs. Deep Copy

A shallow copy is a copy of primitive type values, so string, int, etc. will be copied. These are independent of the original object, but complex types that might be properties of the object you're cloning will be shared across the different clones. Because of that, changing a value from that complex object, for example, the manager name results in changes on both the original object and the clone. A deep copy, on the other hand, also results in copies of the complex types. In that case, we can modify the complex object properties of the original object without it affecting those of the cloned object and vice versa. Let's see how we can support this.

Demo - Supporting Deep Copies

In this demo, we'll add support for deep copies or clones. Let's start by adding a boolean, `deepClone`, as parameter to the clone method signature. We do that in the Person abstract base class, which means we also need to do this in the Manager class and the Employee class. Personally, I like passing through a default value for this, `false`, so clients don't have the obligation to be explicit about it. That's it for the manager, and we do the same for the employee. All right, all that's left is effectively creating such a deep clone. We could manually traverse the object tree, creating shallow copies of each complex type and assigning it as a property value. But a more common approach is making a binary copy using a binary formatter. The code for that would look like something you see on screen now. However, since that formatter was introduced in .NET, deserialization vulnerabilities were discovered. As a result, this does not follow modern best practices, and the `Deserialize` method can be used as a vector for attackers to perform denial-of-service attacks against consuming apps. This is not something that can be solved with this binary formatter. So what Microsoft currently recommends is to

simply no longer use the binary formatter anymore. If you want to read more about that, you can do that via the link on screen. So, what do we do now? Well, a common approach is to use a JSON serializer instead like the built-in serializer, which can be found in a `system.text.json` namespace. Let's do that. So, we implement the `Clone` method in the `Employee` class by serializing the object to JSON and then deserializing it back to an employee, and then we do the same for the `Manager` class. In the `Program` class, we change our clone call, this time requesting a deep clone by passing through `true`. And let's give this a try, and there we go. This time we do get back the expected result. We changed the manager name after having cloned the employee, yet cloning the employee also cloned the `Manager` object, which means that we see the manager name of the clone and not the name change we passed through. This is effectively a deep clone or copy. Let's have a look at something else next, the `IClonable` interface.

What About the `ICloneable` Interface?

.NET includes an interface, the `ICloneable` interface. We could use that. It enables us to provide a customized implementation that creates a copy of an existing object. The interface only contains one member, the `Clone` method, which we could implement. So, why didn't we use that? Well, we could, but there's a few disadvantages. One, the `ICloneable` interface simply requires that your implementation of the clone method returns a copy of the current object instance. It does not specify whether the cloning operation performs a deep copy, a shallow copy or something in between. Two, it also doesn't require all property values of the original instance to be copied to the new instance. And three, it returns an object, which would mean the client could need an additional cast. So, you can indeed use it, but I would advise not to. Let's have a look at the use cases for this pattern next.

Use Cases for the Prototype Pattern

Let's start with a few high-level more abstract use cases. The pattern is useful when a system should be independent of how its objects are created and to avoid building a set of factories that mimic the class hierarchy. For example, class `A` deriving from class `B` deriving from class `C`, potentially with all of them having properties of type class `D`, `E` or `F`. Creating such a hierarchy with factories would lead to a lot of boilerplate code. So, via a `Clone` method is much more concise. The other use case is when a

system should be independent of how its objects are created and when instances of a class can have one of only a few different combinations of states. In those cases, starting from a clone will often be more convenient than starting from an instance with default values. So, even though it's not used that often anymore, or so it seems, it still has a few really good use cases. Let's map these rather abstract use cases to a few real-life examples you might encounter in your own applications. The first real-life use case could be document cloning in word processors. Word processors often use the prototype pattern to clone documents. When a user wants to create a new document based on an existing one, the application can make a copy of the prototype document, preserving its formatting, style, and content. Like that, you can allow users to quickly create new documents with a consistent look and feel based on another existing document. So, if you need to implement functionality like that, the prototype pattern will be very helpful. Another use case is managing configuration instances. Some applications might require multiple instances of complex configurations. And in that case, you can benefit from the prototype pattern. For instance, if you're writing a network device manager, you can create and customize network profiles by cloning prototype configurations to ensure consistency and to reduce configuration errors. Another use case would be if you're creating an application in which you need to create reusable UI components. You could use the prototype pattern to create such a reusable UI component. And these components, these prototypes can then serve as templates for buttons, forms or other interface elements. Cloning these prototypes allows for consistent UI design across an application. A last use case, just like in the module on the builder pattern, would be creating characters in game development. The prototype pattern is useful for creating game entities or characters. Instead of creating new instances from scratch, you can clone prototypes with predefined attributes, behaviors, and textures. That might save you some processing power and memory. So like this, again, you can see that different patterns can work nicely together or can, in some cases, be used interchangeably. Let's have a look at the pattern consequences.

Pattern Consequences

Much of the consequences are related to the abstract factory and builder consequences. First of all, Prototype hides the ConcreteProduct classes from the client, which reduces what the client needs to know. It doesn't need to know about the concrete classes of the objects of which you want to copy. A positive prototype-specific consequence is reduced subclassing. Instead of having to create a factory

method to make a new object, you can simply clone an existing one. A potential liability is that each implementation of the prototype base class must implement its own clone method, and that might not always be convenient. Up next, two related patterns.

Related Patterns

Prototype and abstract factory can be used together. A factory might store a set of prototypes from which it clones when a new instance is requested. There's also a relation to another creational pattern, the factory method pattern. Prototypes don't require subclassing creators. They are not based on inheritance, which is a plus when you favor composition over inheritance. As you'll remember from the first module, the Gang of Four does that. However, they often require an initialize operation on the Product class, which the factory method doesn't. Factory method is based on inheritance, but it doesn't require the initialization step. Prototype is also related to the singleton pattern, as it can be implemented as one. And lastly, when using the composite and decorator patterns, which are both structural patterns, the prototype can be beneficial for convenient object creation. And with that, it's time for the module summary.

Summary

In this module, we covered the prototype pattern. Its intent is to specify the kinds of objects to create using a prototypical instance and then create new objects by copying this prototype. It essentially describes cloning. As far as the implementation is concerned, it's important to remember that subclasses of the Prototype implement the Clone method, and clients work on the Prototype. There's two types of clones, a shallow and a deep copy or clone. A shallow clone is a copy of primitive values, so strings, integers, etc. will be copied. These are independent of the original object, but complex types that might be properties of the object or cloning will be shared across different clones. A deep copy, on the other hand, also results in copies of the complex types, and that is it for the creational patterns. On to the next category, structural patterns, starting with the adapter pattern.

Checkpoint 06 - Supporting Deep Copies

Structural Pattern: Adapter

Coming Up

In this module, we will cover the adapter pattern, also known as the wrapper pattern. It's a very common one used to let classes work together that couldn't be before because of incompatible interfaces. We'll start by describing the pattern, which will quickly lead us to realize that there's two different variations of it, the class adapter and the object adapter. We will, after that, look into the pattern structure. And as a real-life example, we will use a travel agent scenario that needs to load cities from an external system. We'll continue with learning about the use cases for this pattern and pattern consequences, which can be both positive and negative. We will end the module by looking into related patterns. Let's dive in.

Describing the Object Adapter Pattern

The adapter pattern is a commonly used structural pattern from the well-known Gang of Four design patterns. The intent of the adapter pattern is to convert the interface of a class into another interface clients expect. Adapter thus lets classes work together that couldn't otherwise because of incompatible interfaces. There are two variations of this pattern, the first one being the object adapter pattern. So, we'll cover that one first. You could, for example, be creating an application that needs to talk to an API that exposes cities, or you could just be talking to another application layer that works on a different model than the one you are working on or the one you eventually need. For our use case, let's say we've got a `CityFromExternalSystem` class with the name `NickName` for the city and the amount of inhabitants as an integer property. This `CityFromExternalSystem` class is exposed by an `ExternalSystem` class, which contains one method, `GetCity`, which returns a `CityFromExternalSystem`, but you cannot work with this `CityFromExternalSystem` class. Your system needs a `City` class, which consists of a `FullName` property and an `Inhabitants` property, which is a long or `int64` instead of a regular `int32`. So, you somehow need to convert that `CityFromExternalSystem` into a `City`. Worded slightly differently, you need to adapt the `ExternalSystem`. One way of doing that is via a wrapper. You can create a new class, which adheres to a certain interface and which calls into the `ExternalSystem`. It gets the result of the `GetCity` call at level of the `ExternalSystem`, converts it to a `City` object and

returns it. And that wrapper, that is what we could call a CityAdapter. The contract or interface it must adhere to is what we could call ICityAdapter. The client then calls GetCity on ICityAdapter, and that's it as far as the description goes. I'm pretty sure a lot of you will already have used something like this, maybe without knowing it. Let's map this to the object adapter pattern structure.

Object Adapter Pattern Structure

Our two classes, CityFromExternalSystem and City, are simple helper classes that help us describe the pattern. They are not part of the pattern structure. Our ExternalSystem, which contains the GetCity method that returns a city from ExternalSystem, that is what we call an adaptee. An adaptee is the thing that needs to be adapted. This thus defines an existing interface that needs to be adapted, the specific request. The ICityAdapter interface is named Target, in other words, the thing we need to adapt the adaptee to. It defines the domain-specific interface the client works with. The client works with objects conforming to the Target interface. The implementation of that target, which is our CityAdapter, is actually named the Adapter, or if you want, the wrapper that adapts the adaptee to the target. And with that, we've got our pattern structure. Let's implement this in a demo.

Demo - Implementing the Object Adapter Pattern

Time to implement the object adapter pattern. Seeing that in this module we're going to implement two different implementations from the adapter pattern, the object and the class adapter, I didn't create an implementation class in advance. So, let's add one for the object adapter pattern, and let's name that ObjectAdapterImplementation. That takes care of that, and let's get rid of what's in there. And now we can start. We'll start with the CityFromExternalSystem class. This is a simple class with three properties. We accept three matching fields in the constructor. Like that, we convey that this object must be created with these three field values. With that, we've got the helper class that our adaptee needs, so we can add that adaptee. The ExternalSystem class, it exposes one method, GetCity, which returns a new city from ExternalSystem. As city name, I'll use Antwerp, which is where I live. The nickname for this city is Stad, and it has about half a million of inhabitants. Note that there's more cities named Antwerp in the world, but as far as I know, only one has the nickname Stad. Then, on to the City class. This is another helper class we need, and it looks a bit like the CityFromExternalSystem

class. But instead of a name and a nickname, it only has one string property, a `FullName`. The `Inhabitants` property is of type `long`. That's an `int64`. The `CityFromExternalSystems` `Inhabitants` property is an `int`, which means `int32`. So these classes are alike, yet different. The next thing we do now that we've got this helper class is create or define the target. We use an interface for this. An abstract class is also a possibility, but I prefer to use an interface in this case, as we don't need base implementation in this example. What the interface does is define a `GetCity` method, which returns a `City` instead of a `CityFromExternalSystem`. In other words, we've now got two incompatible systems. We can make them compatible by implementing this `ICityAdapter`, and that's what we name the adapter. When we implement the `ICityAdapter` interface, we see that we need to implement one method, `GetCity`. But to make this compatible, we also want to store an `ExternalSystem` property. In other words, we're going to be using composition to make these two different interfaces or systems compatible. For the demo, I'm newing it up ad hoc, but you can inject it if needed or otherwise ensure that the `CityAdapter` gets a hold of this. Again, as already mentioned, the design patterns should be seen as a template to start from. You can change the template if it fits your needs, as long as you stick to the intent of the pattern. And now we only need to implement the `GetCity` method. We do that by calling into the `ExternalSystem` and then adapting the result, which is a `CityFromExternalSystem`. In this method, we call into the `ExternalSystem`. And then we adapt the result, which is a `CityFromExternalSystem`, to a `City` object, and that's what we return. With that, we've created our pattern. Now let's write some sample code to try it out. First, we new up our adapter. For that, we need to import the adapter namespace, and then we try to get a city from it. This should call into the `ExternalSystem` and return a `City` object. To check whether that's really the case, we simply write out the `FullName` and the `Inhabitants`. Note that what should happen now is that the `ExternalSystem`'s `GetCity` call, which would result in Antwerp being returned, will be transformed into a `City` object. So, we should still see the string Antwerp being written out, including the nickname this time, Stad. Let's try that, and that is indeed the case. So far for our object adapter. Let's have a look at the variation of this pattern, the class adapter.

Describing the Class Adapter Pattern

The object adapter pattern we looked into works via object composition. What you see on screen now is the diagram we ended up with when describing the object adapter. The class adapter pattern is a

variation, which relies on multiple inheritance to adapt one interface to another one. In this case, our CityAdapter would derive from the ExternalSystem and from the target, assuming that both of them are abstract base classes. There's only one little problem with this. C# is a language that prefers composition over inheritance, as do the Gang of Four, and moreover, it doesn't support multiple inheritance. We could do this in Java, but we cannot do this in C#. We can let our CityAdapter derive from the ExternalSystem and implement the ICityAdapter interface. Like that, we don't need multiple inheritance. We're implementing a base class, the ExternalSystem, and an interface, and that is supported by C#. Let's have a look at the pattern structure.

Class Adapter Pattern Structure

We're looking at the object adapter structure with a slight change in which we state that the Adapter implements Adaptee, we get the class pattern variation we can work with. The adapter no longer has to store an Adaptee object to call into the specific request on the Adaptee object because the adapter derives from the adaptee, therefore it can now call into the specific request method directly from its parent class, which is the Adaptee. All right, let's implement this.

Demo - Implementing the Class Adapter Pattern

In this demo, we're going to implement the class adapter. For that, let's add a new class to our adapter project. We'll name it ClassAdapterImplementation. I'm going to change the namespace to ClassAdapter and copy over the code from our ObjectAdapterImplementation, as it's about 90% the same code. There we go. Nothing new here. There's only one change we have to make, and that's at level of the CityAdapter class. We let it derive from the ExternalSystem class and implement the ICityAdapter. That means we need to implement GetCity a little bit differently. We no longer store the ExternalSystem as a property here, so we're no longer working with composition. Instead, we call GetCity on the base class, the ExternalSystem class. We can do that as we're deriving from it, and that should be it. Let's give this a try. To do so, we simply change the namespace we import. We're going to change it to ClassAdapter, which means that now the classes from our ClassAdapterImplementation are used. And let's give that a try, and that indeed results in the exact same thing. Let's have a look at the use cases for this pattern.

Use Cases for the Adapter Pattern

Let's start with a high-level view of some use cases. This pattern can be used when you want to use an existing class, but the interface does not match the one you need. Adapting a City class from one system to another one would be a good example of that. Another use case is when you want to create a reusable class, which is the adapter, that works with classes that don't have compatible interfaces. And lastly, you can also use this pattern when you need to use several existing subclasses, but you don't want to create additional subclasses for each of them, yet you still need to adapt their interface. If that's the case, you can use the object adapter pattern to adapt the interface of its parent class.

Okay, so these were high level, rather abstract use cases. What about we make it a bit more tangible by looking into some real-life use cases? A first good use case would be integrating with a third-party library. Often, you'll want to use third-party libraries, but they may not conform to your application's interface standards. Instead of modifying the third-party library's code, which you may not even be allowed to do, you can create an adapter that translates calls between your application and the library, and like that, you're making them compatible. Another typical use case is integrating with a web service. Much like the previous example, when you need to integrate with an external web service, you may run into the issue that what they expect as a request to return as a response doesn't conform to a structure your application can work with. If that is the case, you can use adapters to normalize this. Yet another example comes up during testing, mocking objects. So, imagine you're writing unit tests or integration tests. It's very common that you want to replace some or all components with mocks. Adapters can be used to wrap the real components, which allows you to switch between real and mock implementations easily without having to change the test code itself. And lastly, another example would be when you're integrating with the logging or monitoring framework. Often, you may be dealing with various logging or monitoring frameworks, and you might find that they have different interfaces for logging messages or tracking metrics. You can use an adapter to wrap these interfaces, which allows your application to log or monitor using a standardized interface. On to the pattern consequences.

Pattern Consequences

As every pattern, this pattern too has a few consequences. I'm focusing on the object adapter pattern here, as a true class adapter with multiple inheritance isn't possible in C#, as we learned. One of the

consequences is that it lets a single adapter work with many adaptees, the adaptee itself, and all of its subclasses, as you can cast to those. The adapter can also easily add functionality to all adaptees at once, as calls to the ExternalSystem, the adaptee, pass through the adapter. The single responsibility principle from the SOLID coding principles is adhered to in the way that you're separating the interface, which is the adapter code, out from the rest of the code. The adapter thus has a single responsibility. As we're working on interfaces, new types of adapters can easily be introduced without breaking client code. That's the open/closed principle at work. But there's also some potential disadvantages. For one, the object adapter makes it harder to override adaptee behavior. If you want to do that, you will have to subclass the adaptee and make the adapter refer to the subclass rather than the adaptee itself. Another potential disadvantage, as is often the case with these patterns, is that you're introducing additional complexity because you're introducing additional interfaces and classes. Mind you, complexity isn't bad, but only if you really need it. Unnecessary complexity should be avoided. On to the related patterns.

Related Patterns

As usual, we've got a few patterns related to this one. From the family of structural patterns, there's a relation to the bridge pattern. Both patterns have a similar structure, but a different intent. So the relation is in the structure. A bridge is meant to separate an interface from its implementation so they can be varied easily and independently. An adapter, on the other hand, is meant to change the interface of an existing object. It's also related to the decorator pattern. The decorator pattern changes an object without changing its interface. It's just a bit more transparent than the adapter, which changes the interface of an existing object. Now, as you can grasp from this, even if you haven't used a bridge or decorator yet, you can feel that you can solve likewise or related use cases with different patterns. The adapter pattern is also related to the facade pattern. With the facade, you're defining a new interface for an entire subsystem of objects. With the adapter, you're making an existing interface usable by wrapping one object. And there's also a relation to the proxy pattern. The difference is that the proxy pattern defines a surrogate for another object. It doesn't change its interface. The adapter does provide a different interface. And with that, we are ready for the module summary.

Summary

The adapter pattern lets classes work together that couldn't before because of incompatible interfaces. Two variations exist, object adapter, which relies on composition, and the class adapter, which relies on multiple inheritance. That is not supported in C#, but we can support a slight variation by combining a base class with an interface. That being said, as a language, C# was designed to favor composition over inheritance, and that's one of the reasons why it doesn't support multiple inheritance. In reality, you will spend most of your time with an object adapter and not with class adapters. We implemented an example in which we made two incompatible interfaces work together with an adapter or wrapper class. Up next is a closely related pattern, the bridge pattern.

Checkpoint 07 - Implementing the Object Adapter Pattern

Structural Pattern: Bridge

Coming Up

In this module, we will cover the bridge pattern, another common one. The intent of this pattern is to decouple an abstraction from its implementation so the two can vary independently. We'll start the module by describing the bridge pattern. We will learn what it is and which problem it solves. For that, we will look into a real-life scenario in the context of a cash register. And from that real-life scenario, we will learn about the pattern structure, and we will implement it. We'll continue with learning about use cases for this pattern and pattern consequences, positive consequences and negative consequences. We will end the module by looking into related patterns. Let's dive in.

Describing the Bridge Pattern

The bridge pattern is a commonly used structural pattern from the well-known Gang of Four design patterns. They describe the intent of this pattern as to decouple an abstraction from its implementation so the two can vary independently, in other words, so the two can be developed independently. As with all these intent definitions, this one requires a bit more explanation. You could rephrase this as this pattern aims to separate an abstraction of the class from the implementation. As a result, this

provides the means to replace an implementation with another implementation without modifying the abstraction. Abstraction in this context can be seen as a way to simplify something complex. Take, for example, a console app that wants to write something. `Console.WriteLine` is an abstraction to whatever is contained within the `WriteLine` method. So every action, function, method, etc. are abstractions. We are thus constantly using and creating them. Abstractions handle complexity by hiding the parts that we don't need to know about, and it's this abstraction that we're trying to decouple from its implementation. When I hear things like separate abstraction from implementation, I'm immediately thinking about interface and abstract base classes, yet it's a little bit more complicated than that. Let's clarify this with a real-world example. Say that we are creating software for managing a restaurant register. There's two menus available, one meat-based menu named `MeatBasedMenu` and one vegetarian menu represented by the `VegetarianMenu` class. Both menu implementations expose a method, `CalculatePrice`. Depending on the menu, other fields are available, but those are not of interest for us right now. So, the class that needs to add up the total amount owed can simply call into `CalculatePrice` on each menu. We could implement this with an abstract base class menu or with an `IMenu` interface. Now imagine that we're introducing something new, a coupon, one that offers a €1 discount, another one that offers a €2 discount and so on. These affect the price of both our menus, obviously. How do we implement that? Well, one way of doing that might be to create additional menu specializations. We could create `MeatBasedMenuWithOneEuroCoupon` and `MeatBasedMenuWithTwoEuroCoupon`, both deriving from `MeatBasedMenu`, and we could do something likewise for `VegetarianMenu`. I don't know about you, but this does not look ideal to me. Imagine that another menu is added, `VeganMenu`. We'll have to create additional subclasses for that as well. Or imagine another coupon value being added. We'll need a new subclass for each menu. So, as you can imagine, approaching it like this leads to many additional classes and complexity, which is unnecessary, and unnecessary complexity is something you always want to avoid. That is where the bridge pattern comes in. We're going to put the abstraction, which is the `MeatBasedMenu` and `VegetarianMenu`, in other words, the one based on the menu abstract base class or the `IMenu` interface in one hierarchy. The abstract base class contains the definition of `CalculatePrice`, and `MeatBasedMenu` and `VegetarianMenu` implement it. But to be able to calculate the price, our menus need to know the coupon value. These are named implementors, which we can put in a separate hierarchy, as we could have a coupon abstract base class or an `ICoupon` interface, as you can see on

screen, which states that a coupon value must be exposed. Implementations of it are then `OneEuroCoupon` and `TwoEuroCoupon`. They expose the actual coupon value. Each menu then needs access to a coupon on which it can calculate the coupon amount while calculating its price. We can do that with composition. The abstract base menu class will require an abstract coupon object or an `ICoupon`. And that means that our concrete menu implementations also have access to the coupon value. The relationship between the menu and the coupon, that is named the bridge, as it bridges the abstraction and implementation, letting them vary independently. From this real-world example, we can now extract the structure of this pattern.

Bridge Pattern Structure

The abstract `Menu` class structurally is called an abstraction, as it hides the complexity of getting the coupon value. The abstraction thus defines the abstraction's interface. It will hold a reference to an implementor. We will get to that. A concrete implementation like our `VegetarianMenu` and `MeatBasedMenu` classes is named `RefinedAbstraction`. This extends the interface defined by `Abstraction`. We can implement getting the coupon value as a property we can get on a base coupon object or interface or as a method. But no matter how we do that, the fact remains that the implementation itself is hidden. The coupon base is thus the implementor. This defines the interface for concrete implementations. The interface doesn't need to correspond to the `Abstraction` interface. In fact, most of the time it's very different. You can gather that from our example. `Menu` and `coupon` are definitely different. Classes like `OneEuroCoupon` and `TwoEuroCoupon` are concrete implementations of that `Implementor` interface. A `ConcreteImplementor` implements the `Implementor` interface and defines its concrete implementation. And like that, we know how the bridge pattern is structured. Let's implement our real-world example.

Demo - Implementing the Bridge Pattern

Let's learn how we can implement the bridge pattern. We're going to start by adding our abstraction base, the `Menu`. We're going to use an abstract class for that, which we will use as a base class for the refined abstractions. We give it one method, `CalculatePrice`. That's an abstract method, as it will have to be implemented in the classes that implement this `Menu` class. As we know, this `Menu` class

requires access to an operation implemented by an implementor. We're going to expose that via a Coupon object. This can be a simple abstract class, exposing one property getter to get the value. These are the implementations of the operation that will effectively be implemented on the classes deriving from this abstract Coupon class. Note that this can also be an interface. In fact, let's make it an interface, as it doesn't contain any implementation details. It's just a contract. So, we change it to interface. Let's name it ICoupon, and we will require one integer, a CouponValue. Then let's add some concrete implementations. We'll start with NoCoupon. That exists as well. And now we add two more coupons, one that will give a €1 discount, one that will give a €2 discount. Back to our abstraction. Here, we can now add that coupon as a field. We say that it's readonly. We don't want to see this changed after the menu has been constructed. We also add a constructor here that accepts such a coupon and sets the field. Combined with a readonly modifier, this ensures that our menu class always has a non-null coupon value. And that's why I've written null! here. If we don't do that, we will see a warning in classes that derive from the Menu class. That's because our compiler cannot be sure of the fact that this will not be null, even though we are. By using this operator, we can convey that. There we go. And now let's create our two menu types. These are named RefinedAbstractions. We start with the VegetarianMenu. Via the constructor, we call the base constructor, ensuring that the coupon value is set. So like this, we ensure that coupon is not null. In the CalculatePrice method, we use the CouponValue getter, which is the actual operation implementation from the pattern structure diagram. We use that to get the actual coupon value, which we deduct from the standard menu price. And then we do the same for the MeatBasedMenu. There we go. That's it as far as the pattern implementation goes. We can now test it. We first new up a few coupons, and then we new up a MeatBasedMenu, passing through one of our coupons, the noCoupon coupon. We write out the price. Then we pass through a oneEuroCoupon. For that, we have to new it up again, and we again write out the price, and then we do the same for the VegetarianMenu. Note that we can safely pass through the same coupon instances we used for the MeatBasedMenu. Let's give that a try. And there we go. Our coupons are nicely applied to our different menus. Thanks to the bridge pattern, it is now very easy to add a new menu or a new coupon. It will no longer result in an explosion of classes. Let's have a look at the pattern use cases.

Use Cases for the Bridge Pattern

There's quite a few use cases for the bridge pattern. There's a reason this pattern is very common. Let's start with some more abstract generalized use cases. First good use case is when you want to avoid a permanent binding between an abstraction and its implementation, for example, when the implementation must be selected at runtime, our coupon example showed that. In other words, what a bridge pattern enables is switching implementations at runtime. Another use case is when abstraction and implementation should be extensible by subclassing. The bridge pattern allows combining the different abstractions and implementations, and it allows extending them independently. Another good use case is when you don't want changes in the implementation of an abstraction have an impact on the client. As far as the client is concerned, this should be completely transparent thanks to loose coupling. If you notice a requirement like this in your application, then you may want to consider the bridge pattern. Now let's make this a bit more tangible by mapping these rather abstract use cases to real-life application scenarios like you might encounter in your day-to-day job. Let's start with notification mechanisms. Imagine that you're working on a notification system and you need to separate different notification mechanisms. You can use the bridge pattern for that. You can use the bridge pattern to separate the notification mechanism, which could be email, SMS, a push notification and so on. Separate that from the user notification logic. Like this, you can add new notification methods without modifying the core notification handling code. A second example would be separating streaming protocols in audio or video streaming. If you're building a multimedia streaming application, you could use the bridge pattern to separate the streaming protocol, HTTP, RTSP, from the media player logic. This allows you to support various streaming protocols and media implementations and let them evolve independently from each other. Another good use case would be separating UI components from the code that renders them. In modern UI frameworks, the bridge pattern is often used to separate the UI components like buttons and text fields from the platform-specific rendering code, which will be different on Windows versus Android versus iOS. It's this approach that enables cross-platform development while maintaining a consistent user experience. And lastly, imagine that you're writing an app to, for example, manage a universal remote-control system. In that case, you can use the bridge pattern to separate the remote control's functionalities like the volume control and channel selection from the specific devices like the TV or DVD player the remote works on. This way, you can easily add support for new devices or features without altering the core remote-control code.

Pattern Consequences

As every pattern, this pattern also has a few consequences. The first and foremost consequence, a very important one, is decoupling. The implementation isn't permanently bound to the abstraction anymore. It can be configured and potentially even changed at runtime. Because of this decoupling, layering is encouraged, which typically results in a system that's better structured. A second consequence is that extensibility is improved. As the abstraction and implementation hierarchies are separate, they can evolve and be extended independently. You can thus introduce new abstractions and implementations independently of each other. That is the open/closed principle at work. Three, you can hide implementation details away from clients, which as we know is a consequence of abstraction. Pattern also allows focusing on high-level logic in the abstraction and on the details in the implementation. By doing so, we can adhere to the single responsibility principle. Up next are the related patterns.

Related Patterns

A few patterns are related to the bridge pattern. From the family of creational patterns, there's a relation to the abstract factory pattern. Such a factory can create and configure a bridge, which hides complexity away from client code. From the family of structural patterns, there's a relation to the adapter pattern. That pattern is used for making unrelated classes work together. The adapter adapts the adaptee for that. The bridge pattern, as we learned, is used to let abstractions and implementations vary independently. Strategy, bridge, and state are all related and look similar because they are based on composition, even though they solve different problems. And with that, it's time for the module summary.

Summary

The intent of the bridge pattern is to decouple an abstraction from its implementation so the two can vary independently. We looked at an example scenario from a cash register application in which the bridge pattern helped avoid an explosion of classes. That's a very visible consequence. But more importantly, the main consequences of the bridge pattern are decoupling, improved extensibility, and hidden implementation details. Up next is the decorator pattern.

Structural Pattern: Decorator

Coming Up

We're going to learn about the decorator pattern in this module. This pattern allows adding behavior dynamically to an individual object without it affecting the behavior of other instances of the same class. It's sometimes referred to as a wrapper. I'm Kevin, and I'll guide you through this module. We'll start the module by describing the decorator pattern. We will learn what it is and which problem it solves. The real-life scenario we'll cover while doing so is related to collecting statistics and storing a list of sent email messages. From that real-life example, we will learn about the pattern structure, and we will implement it. After that, we will cover the pattern use cases and pattern consequences, and we'll have a look at related patterns. Let's dive in.

Describing the Decorator Pattern

The decorator pattern is a commonly used structural pattern from the well-known Gang of Four design patterns. The intent of the pattern is to attach additional responsibilities to an object dynamically. A decorator thus provides a flexible alternative to subclassing for extending functionality. Just like the adapter pattern, this pattern is sometimes called a wrapper. Attaching additional responsibility or behavior to a class can easily be done by simply adding an additional method to the class. In this case, however, we need a way to attach additional responsibility to an object and not to a class. We want to allow this dynamically, so at runtime. That is a bit more complicated, but not that much, really. The fact that this pattern is sometimes called a wrapper should already give you a good idea of how something like that can be implemented. Let's clarify this with a real-life example. In one of my other courses, I use a `CloudMailService` and a regular `OnPremiseMailService` to send mail. Both of these implement a simple `IMailService` interface, which exposes one method, `SendMail`. Now imagine I want to add some functionality to the service. Say that I want to start collecting some statistics, for example, how long does sending mail take or how often is the `SendMail` method accessed? I could add

functionality like that to the different mail service implementations, but that kind of violates the single responsibility principle for that class. Moreover, imagine that we want to add even more responsibilities like logging when mail is being sent or storing all the sent mail messages in a database somewhere. Our classes would get littered with code that doesn't necessarily belong there. That's where the decorator pattern comes in. Let's start with our `IMailService` and both implementing classes. We're going to create a wrapper around our `IMailService` to add functionality we need by overriding the `SendMail` method in that wrapper. We can call that wrapper `MailServiceDecoratorBase`, for example. It's an abstract class, which also implements the `IMailService` interface, thus exposing a `SendMail` method. Next to that, we also store the `IMailService` we want to decorate in this `MailServiceDecorator`. It's now up to concrete implementations of this decorator to effectively add functionality. We could have a `StatisticsDecorator` that derives from `MailServiceDecoratorBase`, calculates some statistics, and then calls into the `SendMail` method. We could have a `MessageDatabaseDecorator`, which stores all the messages that are being sent in a database and so on. Instead of directly calling, for example, `CloudMailService.SendMail`, we can now call `StatisticsDecorator.SendMail` or `MessageDatabaseDecorator.SendMail`. With a few simple interventions, we can easily extend the functionality of the `MailService` objects without having to resort to an explosion of subclasses or large inheritance trees and all of that while sticking to the single responsibility principle. From this real-world example, we can now extract the structure of this pattern.

Decorator Pattern Structure

The `IMailService` interface structurally is called `Component`. The `Component` defines the interface for objects that can have responsibilities added to them dynamically. Concrete implementations like our `CloudMailService` and our `OnPremiseMailService` classes are called `ConcreteComponents`. They define an object to which additional responsibilities can be attached. The other implementation of that `IMailService` interface is structurally called the `Decorator`. That is our abstract `MailServiceDecoratorBase` class. It intercepts if you want the `SendMail` operation, thus allowing us to extend the functionality of the object. It maintains a reference to a `Component` object, and it defines an interface that conforms to the `Component`'s interface. Concrete implementations like our `StatisticsDecorator` and `MessageDatabaseDecorator`, which contain added behavior like storing statistics or state like a message store, these are called `ConcreteDecorators`, for example,

ConcreteDecorator1 and ConcreteDecorator2. They thus add responsibility to the component. And like that, we know how the decorator pattern is structured. Let's implement our real-world example.

Demo - Implementing the Decorator Pattern

In this demo, we will implement the decorator pattern. Let's start by adding our Component, the `IMailService` interface. Note that as with many of these patterns, this can also be implemented with an abstract class instead of an interface. Our interface contains one method, `SendMail`, accepting the message to send. It returns a boolean so we know whether sending the mail was successful or not, then we can create our concrete components. We do that by implementing the interface. In the first concrete component implementation named `CloudMailService`, we fake sending the mail via a cloud service. And we do something likewise with an `OnPremiseMailService`, another concrete component implementation. So far, so good. Up next is the decorator itself. Let's add a new class, `MailServiceDecoratorBase`. This will become our decorator. As we learned on the slides, it should implement the `IMailService` interface. Like that, we ensure that the same method, `SendMail`, must be implemented. We'll get to the implementation of this method immediately. First, we ensure that this class can only be constructed with the `IMailService` implementing object being passed in. That is the object we want to add functionality to. We store that in a private, readonly field. We do still need to call `SendMail` on it, and that's what we do in the `SendMail` method on our `MailServiceDecoratorBase`. And we return the result, but this isn't done yet. This class doesn't really do anything yet. It wraps the `IMailService`, but it doesn't add functionality to the `MailService`. This serves as an abstract class to store the `IMailService` instance. You could name this a base wrapper. We don't want this to be used on its own, so we mark it as abstract. One thing left here. We want to make sure we mark the `SendMail` method as virtual. So, implementing classes can effectively overwrite this and add functionality. And that's it for this class. Now let's add concrete decorator implementations. The first one is a `StatisticsDecorator`. It implements the `MailServiceDecoratorBase` abstract class, which ensures that we get a constructor generated. We call into the base constructor from the `StatisticsDecorator` constructor. Like that, our `mailService` is nicely stored in the private field. We also override the `SendMail` method. We collect some statistics, and we send the mail. We have to call `base.SendMail` for that, as the actual implementation of sending that mail is at that level. On to our second concrete decorator. Our `MessageDatabaseDecorator`, likewise it implements the `MailServiceDecoratorBase`. It

stores a list of sent messages, acting as a data store for messages sent through this decorator. In the `SendMail` method, we store the message if it's been successfully sent, and that's it for our pattern implementation. Let's test this. The first thing we want to do is instantiate our mail services. For that, we need to import the `Decorator` namespace. So, we new up a `cloudMailService` and an `onPremiseMailService`. And for testing purposes, we've sent a mail through them. That's the default behavior of these mail services. What we want to do now is add behavior to it, to collect statistics. To do that, we decorate the `cloudMailService`, for example, with the `StatisticsDecorator`. And then we send the mail through that `StatisticsDecorator`. This should show us some added behavior, in other words, some added functionality. And then we also add behavior to the `OnPremiseMailService`. As you can see, this works in a likewise manner. The decorators work on the `MailService` interfaces and not on the concrete implementations. So, we send a few messages through the `MessageDatabaseDecorator`. And then we run through the stored messages to see whether or not our decorator successfully executed. Let me add a `ReadKey` statement here, and let's give this a try, and there we go. The first thing we see are the two messages we sent via the `SendMail` method on the services themselves. Then we decorated one of them with a `StatisticsDecorator`, and that's what we see next, `Collecting statistics in StatisticsDecorator`. So we pass through that. The next line is the message passed through to the `CloudMailService` from the `StatisticsDecorator`, `Hi there via StatisticsDecorator wrapper`. Up next, we used the `MessageDatabaseDecorator` to send messages via the `OnPremiseMailService`. We did that twice, which is what you can see on screen. After that, we ran through the stored messages because the `MessageDatabaseDecorator` stores the messages that are sent through it. And we indeed see our two messages being sent. With that, we successfully implemented a decorator. Let's have a look at the use cases for this pattern next.

Use Cases for the Decorator Pattern

Let's start with a few rather abstract, generic, high-level use cases. You can use this pattern when you have a need to add responsibilities to individual objects dynamically at runtime without affecting other objects. The example we just covered was thus a good use case. Another good use case for this pattern is when you need to be able to withdraw responsibilities you attached to an object. Decorators can be applied dynamically and thus also removed dynamically. That is an advantage. For example, when you want to switch out storing mail messages in a database to another user store, simply

remove the `MessageDatabaseDecorator`, and if needed, replace it. And lastly, you can use this pattern when extension by subclassing is impractical or impossible, for example, if the class is sealed. Now let's try mapping these rather abstract use cases to some real-life examples, some real-life scenarios you might encounter during your day-to-day job. A first good use case like that would be adding logging and monitoring capabilities. You can use decorators to add logging and monitoring capabilities to various components of your application without modifying their core functionality. For example, you can use a logging decorator that logs method calls, input parameters, and results, which helps with debugging and performance analysis, and you can do that without having to modify the method itself. Formatting text in text editors is another often used use case for the decorator pattern. You can use it to add formatting features to text. For example, you can have a basic text editor component and then apply decorators to add features like font formatting, color highlighting or spell checking. Users can then combine these decorators to customize the appearance and functionality of their text. Remember, decorators can be applied at runtime, so you can do this dynamically when needed. Another potential use case would be adding authentication and/or authorization layers. You can use decorators for that. Let me clarify that a bit. You could start with a base authentication module and then add a decorator for multi-factor authentication, for role-based access control, and other security features. These decorators help extend your application's security while keeping the core authentication interface unchanged. The last use case I want to cover would be enhancing the appearance or behavior of your graphical user interface elements, in other words, styling or theming an application. You can use decorators to enhance the appearance or behavior of your buttons, for example. For instance, you could have a base graphic object representing a shape like a rectangle or a button and then apply decorators to add features like borders, shadows or patterns. These decorators can be stacked or combined in any order to achieve different visual effects. As you've realized by now, these use cases are often related to use cases from other modules. That's because you can often solve problems in multiple ways using multiple patterns or even replacing one pattern with another one. Let's have a look at the pattern consequences.

Pattern Consequences

As every pattern, this one too has a few consequences. It's more flexible than using static inheritance via subclassing. Responsibilities can be added and removed at runtime when needed. If you compare

that to subclassing, which leads to increased complexity of your system when many different behaviors must be added, the decorator pattern is definitely preferable. Too, it avoids feature-loaded classes that don't respect the single responsibility principle. With this pattern, you can split up that class until there's just one responsibility left, thus adhering to that single responsibility principle from the SOLID coding principles. The third consequence is that this pattern does tend to litter your system with lots of small, simple classes, and that does potentially increase the effort required to learn and debug the system. Let's have a look at the related patterns.

Related Patterns

As usual, a few patterns are related to the decorator pattern. From the family of structural patterns, there's a relation to the adapter pattern, but the adapter pattern gives a completely new interface to an object, while the decorator pattern only changes its responsibilities. Another related structural pattern is the composite pattern. A decorator can be viewed as a composite with only one component. That being said, the decorator is not intended for object aggregation, while the composite pattern is. And lastly, a behavioral pattern, the strategy pattern. While the decorator pattern lets you change the skin, which would be the outer side of an object, the strategy pattern lets you change the inner workings. So, both allow you to change an object, but in a different way. Let's check out the module summary.

Summary

The intent of the decorator pattern is to attach additional responsibilities to an object dynamically. We implemented this pattern to add functionality like collecting statistics and storing sent messages to mail service implementations. The pattern offers more flexibility than static inheritance through subclassing. Up next, the composite pattern, one that's related to the decorator pattern.

Checkpoint 09 - Implementing the Decorator Pattern

Structural Pattern: Composite

Coming Up

In this module, we will cover the composite pattern. That's a pattern that takes advantage of recursion, and it is often used when your data resembles a tree-like structure. We'll start by describing the pattern using a real-life example. As a real-life example, we'll use a file system scenario. After that, we will map that to a structural example, and we will implement the real-life example in a demo. We'll continue with learning about use cases for this pattern and pattern consequences. We will end the module by looking into related patterns. Let's dive in.

Describing the Composite Pattern

The composite pattern is a structural pattern from the well-known Gang of Four design patterns. Its intent is to compose objects into tree-like structures to represent part-whole hierarchies. The composite pattern lets clients treat individual objects and compositions of objects uniformly, as if they were all individual objects. Okay, that sounds good, but what would that translate to in real life? Well, composing objects into tree-like structures is something we're all very much used to. Imagine the structure of an XML document where each element, in turn, consists of other elements, which can again consist of elements. Another example might be an application in which a user can draw. You can represent such a drawing as something that consists of one or more lines and potentially one or more other pictures, which in turn consists of lines and pictures and so on. The example we're going to work with is another one you can undoubtedly immediately picture, the file system. We've essentially got items in there. Let's name them file system items that are either a file or a directory. Such a directory can consist of one or more files and directories and so on. So there's our tree structure already. We've got two classes on that structure, a File and a Directory. Now, imagine we want to get the size of all the items in our tree structure. We would have to run through these objects. We need to know the exact classes, File and Directory. We need to know the method to call on them to get their size. We need to know how deep nesting goes to be able to access those levels of files and directories and so on. This becomes very cumbersome very fast. That's what the composite pattern helps with. What if instead of working with Directory and File classes directly, we'd work with the base FileSystemItem class that represents a common interface through which we can calculate the total size. Let's put that into practice. On the FileSystemItem base class, we define a method to calculate its size, GetSize. This is implemented by our File class and by our Directory class. The Directory class next to that also

stores its children, and it includes methods to add additional file system items so the tree can be built. On the File class itself, we don't need that, as such a file cannot have children. Only a directory can. The other common thing we want to do is define an operation on the FileSystemItem, a method to get the size of the FileSystemItem. In case of a single file, it's simply a matter of returning the actual size. In case of a directory, that method would count the file size of the directory itself and then run over all file system items in its FileSystemItem collection, get the size for each, and add them up. That is a very nice example of recursiveness. All of this is transparent for the client. It's the FileSystemItem itself that will handle counting the size of all the items in the tree structure. This is a lot more convenient to work with. So that's our real-life example. Let's map this to the pattern structure.

Composite Pattern Structure

Our base object, that's FileSystemItem, is what maps to a component. This can be implemented as an interface or an abstract base class. This Component thus declares the interface for objects in the composition and potentially implements default behavior, which is possible if you implement it as a base class. It also contains a common operation. That common operation in our example is the method to get the FileSystemItem size. It can potentially define an interface for accessing and managing its children. This was obligatory in the initial design pattern template, but in more current iterations of the pattern, this type of behavior is often defined and implemented on the composite itself instead of on the base class. We'll get to that. One item would be a Leaf, in our case, file. This one does not have children. It derives from the component, and it implements the common operation. So, behavior for the primitive objects in the composition is the responsibility of the Leaf. Another item would be a Composite. That one also derives from the component base and stores a set of components, children. It implements the common operation. It thus defines behavior for components having children. And lastly, there's the client, of course. This one manipulates objects in the composition through the composition interface. In the structure, as it's detailed in the Gang of Four design patterns book, the component abstract class also defines an operation to get a specific child by index. But as mentioned before, you shouldn't see that as a literal must-have to have such an operation. Instead, see it as a reminder that you can add that type of functionality if you need it. In that same spirit, we have varied this a bit. In the original implementation, the add and remove operations were defined on the abstract base class and not overridden on the Leaf class. We simply didn't define

them on the base class and instead add them to the Composite class. So this just goes to show that there are multiple ways of implementing a pattern. Just make sure you keep the intent in mind. And with that, we can implement this example.

Demo - Implementing the Composite Pattern

Time to implement our example. In this demo, we will implement the composite pattern using the real-life example we just described. We'll start with the abstract base class, the `FileSystemItem`. This is the component. We give it a name and define an abstract method on it to get its size. We also add a constructor that accepts the name, signifying we expect consumers to pass through a name when constructing the item. This is an abstract class, so it cannot be used on its own. The next step thus is implementing this class. Our first implementation is the Leaf. In our case, that's a file. Next to a name, we also expect a size this time, so we add a constructor that accepts that size and stores it in a private field. Implementing the `GetSize` method is then simply a matter of returning the size variable. The directory from our example is the Composite, and it too derives from `FileSystemItem` and requires a size. So we have a likewise constructor as we had for the file, yet we haven't implemented `FileSystemItem` yet. When we do that, we see that we need to provide an implementation for the `GetSize` method, and that's a bit more complicated in this case. As the directory is a composite, it can contain other components, `FileSystemItem` in our case. So we need a variable to store these in. We make sure this one is initialized to avoid `NullReferenceExceptions`. We also wanted to add and remove functionality. There's multiple ways of doing this. We could simply expose our private list of `FileSystemItems`, but I'm not a big fan of that, as it breaks encapsulation. It would allow other classes to directly manipulate this list. Whether or not you want to do that depends on how much your use case requires you to shield this list. For our case, we're going to make encapsulation as strict as possible. We actually don't need this to have a property getter and setter. So, we add the two methods to add and remove `FileSystemItems` by manipulating the `FileSystemItems` list. Lastly, we can now implement the `GetSize` operation. This shouldn't just return the size value. It should return the sum of the current size value and that of each size of each `FileSystemItem`. In turn, one of those could be a directory, which would result in the size getting calculated by running through all of its children and so on. So that's a very nice example of a recursive method at work. From inside of the `GetSize` method, we call `GetSize` on each `FileSystemItem`. And that's it for our implementation. Let's write a bit of code

to test this. The first thing we need to do is create our tree structure. So, let's already create a few directories and files. We start with a root, which we give no specific value for size. To that, we want to add one file and two directories, so we new up a file and two directories. We give the file and these directories a size, and we add them to the root. We're not going to stop there. We're going to create a tree that's a bit bigger than this, so we're going to create two sublevel files as well. We new those up, giving them a size, and we add them to one of the top-level directories. Like that, we have something that is starting to resemble a nice tree hierarchy, and then let's test whether our GetSize methods actually work. So we write out to the Console window a GetSize statement on the topLevelDirectory1, one on topLevelDirectory2, and one on the root itself, which should result in the full size of our tree. Interesting to notice here is that at level of the client, we are getting the size. But as far as the client is concerned, interacting with a Leaf is exactly the same as interacting with a Component. We're calling GetSize on the directories here, but we might as well call GetSize on one of the files as well. This is completely transparent to the client. Let's give that a try, and there we go. Our three GetSize statements checked out nicely. And if we do the math, it seems that it indeed checks out as well. Let's have a look at the use cases for this pattern next.

Use Cases for the Composite Pattern

Let's start with a few high level, rather abstract use cases for this pattern. You can use this pattern when you want to represent part-whole hierarchies of objects, in other words, when you want to represent a tree-like object structure, and/or you can use it when you want clients to be able to ignore the difference between compositions of objects and individual objects. As we've noticed in the examples, as far as the client is concerned, it doesn't matter if the operation, GetSize in our example, is called on a Leaf or on a Composite. So, if you find one of these two requirements in your codebase, then the composite pattern is something to look into. Now let's map these rather abstract use cases to a few real-life scenarios and examples you might encounter in your day job. A first good example would be investment portfolio software. These types of portfolios can consist of individual assets such as stocks, bonds, mutual funds, and so on. You can use the composite pattern to represent the portfolio as a whole with the ability to calculate its overall value, even if it contains these various asset types. Another pretty typical example would be a menu system. In a menu system, menus can contain menu items, submenus, separators, and so on. With the composite pattern, you can build a

hierarchical structure of menus, making it easy to add, remove, and manipulate menu items and submenus. And now we have an organization chart. In such a chart, you typically have employees at various levels within the hierarchy. You can use the composite pattern to represent employees and their positions within the organization. That allows for consistent treatment of both individual employees and entire departments. And lastly, imagine you're creating a drawing application. And lastly, a use case which I referred to in the beginning of the module already, a drawing application. These types of applications often involve creating complex drawings by combining simple lines, shapes, objects. What you could do is use the composite pattern to represent the drawing structure, allowing users to manipulate individual elements or groups of elements. And with that, you should have a good idea of use cases for this pattern. Let's have a look at its consequences.

Pattern Consequences

The first consequence of this pattern is that it makes the client simple, as clients don't know whether they're dealing with a Leaf or a Composite component. That's an advantage. It's also easy to add new kinds of components. And that is because newly defined subclasses automatically work with existing structures and client code. From that, it follows that clients don't have to change when new component classes are added. Imagine you'd have another type of `FileSystemItem`. You could simply implement it and work with it. The client doesn't care. It simply calls `GetSize` on the common `FileSystemItem` interface and not on the subclass. This is the open/closed principle from the SOLID coding principles at work. But there's also a disadvantage. All of this can make the overall system too generic. By making it easy to add and remove items to the composite, we also make it a bit harder to restrict the components of a composite. And that is because it's no longer the compiler that checks for this, as `add` and `remove` expect the abstract base type or interface and not the subclass. Take our `FileSystemItem` example. If we want to restrict the directory component to only contain files and not subdirectories, we cannot rely on the compiler for that anymore because `add` or `remove` accepts the base `FileSystemItem`. We have to write additional code to check for the actual type and run type instead. Let's have a look at the related patterns.

Related Patterns

There's quite a few related patterns to this one. From the behavioral family, there's a chain of responsibility. The component parent link is often used for that. When a Leaf component gets a request, it can potentially pass it through the chain of all parent components down to the root of the object tree. It's also related to the iterator pattern because that one can be used to traverse composites. And lastly, the visitor pattern can localize operations and behavior that would otherwise be distributed across composite and Leaf classes. You can thus use the visitor pattern to execute an operation over the full composite tree. Time for the module summary.

Summary

The composite pattern provides a transparent, easy way to work with tree-like structures. Whether the client is interacting with a Leaf or Composite isn't of importance to the client. The recursiveness of the operation makes sure of that. Up next, we will learn about the facade pattern.

Checkpoint 10 - Implementing the Composite Pattern

Structural Pattern: Facade

Coming Up

In this module, we will cover the facade pattern. It's intended to provide a higher-level interface over the subsystem to make the subsystem easier to use. We'll start by describing the pattern using a real-life example. As a real-life example, we're going to use a discount calculator service. After that, we will map that to a structural example and implement the real-life example in a demo. We'll continue with learning about use cases for this pattern and pattern consequences, which can be positive or negative. We will end the module by looking into related patterns. Let's dive in.

Describing the Facade Pattern

The facade pattern is a commonly used structural pattern from the well-known Gang of Four design patterns. Its intent is to provide a unified interface to a set of interfaces in a subsystem. It defines a

higher-level interface that makes the subsystem easier to use. Let's think about that by studying a real-life example. You're building a system in which you want to know the discount a customer will receive. However, calculating this can be quite complicated. First of all, we need to call into a service that checks whether the customer has enough orders, and then we need to calculate the discount base percentage, which depends on the customer. For example, the percentage can be different, depending on the country the customer is from. Lastly, a factor must be taken into account, depending on the day of the week. This is something you sometimes see at airfare sites. The calculation thus isn't trivial, and it has to be done in different parts of the application. Putting a facade in front of it hides away the complexity. So, it's an abstraction, and it encourages reuse. We could name that facade `DiscountFacade`. It contains one method, `CalculateDiscount`, which is responsible for calling into the various services and calculating the actual discount percentage. So that's our real-life example. Let's map this to the pattern structure.

Facade Pattern Structure

The structure of the facade pattern is pretty straightforward. First of all, we have Facade, that's our `DiscountFacade`. This Facade has knowledge about the subsystem classes that are responsible for a request, and it delegates requests to the appropriate subsystem classes, that subsystem that are all our other services or classes, the `OrderService`, `CustomerDiscountBaseService`, and `DayOfTheWeekFactorService` are all subsystems. They implement their own functionality as needed, and they handle the work assigned by the Facade object. In other words, methods called by Facade are implemented by the subsystem. These subsystems don't have knowledge of the Facade. Let's implement our real-life facade example.

Demo - Implementing the Facade Pattern

The first thing we want to do in this demo is creating the subsystem classes. Our first subsystem class is an `OrderService`. It should check whether the customer has ordered enough in the past to be eligible for a discount. It has one method, `HasEnoughOrders`, which accepts a `customerId`. We do a fake calculation here for demo purposes. Any customer with an ID larger than 5 is assumed to have made enough orders. The second subsystem class is the `CustomerDiscountBaseService`, which is

used to calculate the discount percentage base. These calculations can be quite extensive. You could take the country into account, you could take the amount of orders or total order price into account and so on. We'll fake this too for demo purposes. If the ID of the customer is larger than 8, he or she gets a 10% discount, otherwise a 20% discount. The last subsystem class is the `DayOfTheWeekFactorService`. We use a switch statement on the `DayOfWeek`, stating that during the weekend, the factor is 0.8, and otherwise, it's 1.2, which means orders done during the weekend get discounted less while orders done on other days get discounted more. So these are our helper classes. On to the facade itself. We'll name it `DiscountFacade`. It's a new class, which should contain references to our subsystems, so we add three references to the subsystems. We could inject these instances via constructor injection. But in our case, we don't have an IoC container set up, so this time, we'll just new them up. We could also add the constructor anyway and let the Client class provide us with the subsystem class instances. Now we need to calculate the discount. We add one public method, `CalculateDiscountPercentage`, which accepts a `customerId`. First, we call into the `_orderService`. If the customer doesn't have enough orders, we return 0 as the discount percentage. Otherwise, we multiply the discount base with the `DayOfTheWeekFactor` and return that value. And that's it for the facade. Our pattern is already done. Let's test this. We new up a Facade object. For that, we need to add a using statement to Facade, and then we test our facade by trying to get a discount for customer with id 1 and one for customer with id 10. Let's give that a try, and there we go. No discount for the customer with id 1. Because the ID of this customer isn't larger than 5, customer with id 10, on the other hand, does get a discount. That's it for this very simple pattern. Let's check out the use cases for it.

Use Cases for the Facade Pattern

We'll start our use cases with a few high-level use cases for this pattern. After which, we'll dive into some more real-life use cases you may run into in your day-to-day developer life. First use case, high level for this pattern, is obviously when you want to provide a simple interface into a complex subsystem. That complex subsystem would typically consist of a bunch of subsystem classes. If you provide a simple interface to that complex subsystem, clients using the facade won't notice it when the subsystem becomes more complex. It's also useful to consider this pattern when there are many dependencies between a client and the implementation classes of the abstraction. By putting a facade

in between, you decouple the subsystem from the clients. Now let's map these to some real-life scenarios. A good first real-life use case would be integration with legacy systems. When you need to integrate a new system with a legacy system, you can use a facade to create a bridge between the old and new system. It abstracts the complexities of dealing with the legacy code you're integrating with, which allows the new system to interact seamlessly with the legacy one. Other types of software where you commonly see the facade pattern being used are content management systems. These types of systems often need to handle various types of content, including articles, images, and videos. With a facade, you can simplify content management by providing a unified interface for creating, retrieving, updating, and deleting different content types. Another one would be multimedia playback. Multimedia players often need to support various formats and provide controls for play, pause, and volume. With a facade, you can abstract away the details of handling different media formats and offer a unified playback control interface. And lastly, there's payment processing. In ecommerce systems, there are often multiple payment gateways and methods available. With a facade, you can abstract away the differences between these payment options, allowing the application to interact with a single unified interface for processing payments. And with that, we've covered the use cases for this pattern. As you may have noticed, some of these use cases are also a good fit for other patterns or for combinations of patterns. It just goes to show that there is not necessarily just one best option for solving a specific use case. The more you learn about these patterns, the more you will be able to get a feeling of which pattern could fit for your use case and which one you would want to implement. Let's continue with the pattern consequences.

Pattern Consequences

As every pattern, this pattern has a few consequences. One, it shields clients from subsystems, which means that the number of objects clients have to deal with are reduced. It also promotes weak coupling between the subsystem and its clients. The client is coupled to the facade and not to the subsystems directly. Thanks to that weak coupling, the subsystem components can vary without this affecting the client. That's the open/closed principle from the SOLID coding principles at work. And lastly, even though it shields clients from subsystems, clients are not forbidden to use the subsystem classes if they want to. Let's have a look at the related patterns next.

Related Patterns

Let's start with a related pattern from the creational family, the abstract factory pattern. This can be used together with a facade to provide an interface for creating subsystem objects while remaining independent of the subsystem. An abstract factory can also be used as an alternative to the facade to hide away platform-specific classes. Up next from the behavioral family, the mediator pattern. Like the facade pattern, it abstracts functionality of existing classes, the subsystems in our example, but it's not the same. The purpose of the mediator is abstracting communication between objects, often centralizing functionality that doesn't belong in those objects. Facade, on the other hand, simply abstracts the interface to the subsystem objects to promote ease of use. Lastly, it's also related to the adapter pattern. With the adapter, you're making an existing interface usable by wrapping one object. With a facade, you're defining a new interface for an entire subsystem of objects. Let's have a look at the module summary.

Summary

The facade pattern is a very common pattern that makes it easier for the client to use subsystems by providing one or more interfaces into those subsystems. It's a higher-level layer to make the subsystem classes easier to use. Up next is the proxy pattern.

Checkpoint 11 - Implementing the Facade Pattern

Structural Pattern: Proxy

Coming Up

In this module, we'll cover the proxy pattern, also known as the surrogate pattern. It's intended to provide a surrogate or placeholder for another object to control access to it. We'll start by describing the pattern using a real-life example. As a real-life example, we will use a remote API call for starters. After that, we'll map it to a structural example. We will also look into variations of this pattern because there's quite a few. We'll choose an example of this to implement in a demo. We'll continue with

learning about use cases for this pattern and pattern consequences, which can be positive or negative. We will end the module by looking into related patterns. And with that, we can dive in.

Describing the Proxy Pattern

The proxy pattern is a structural pattern from the well-known Gang of Four design patterns. Its intent is to provide a surrogate or placeholder for another object to control access to it. And that's important to know because this means that the reason for this pattern, the problem it solves, is one of access control. If you need to control access to another object for security reasons, performance reasons or other reasons, then the proxy pattern is a good one to look into. The archetypical example of a proxy, which some of you have undoubtedly encountered, is accessing a remote API from an application. Those of you who are old enough to remember the Add service reference option in Visual Studio will know about this. The idea here is that your application uses a proxy to interact with the actual remote API. That proxy can be completely written from scratch. But as mentioned, there was something like the Add service reference dialog, which would generate a proxy for you. The proxy is responsible for controlling the actual access to the actual remote API. It can contain code that deals with the underlying protocol, for example, the actual HttpClient calls. It can also contain code that ensures the most performant, reliable way to access that remote API is used and so on. In principle, proxies have the same interface as the object they're controlling access to. So for the client, it feels transparent. The client shouldn't care whether it's directly interacting with a remote API or if it's going via the proxy. But truth be told, sometimes you see code generating what they call proxies that don't adhere to this principle, which makes them more of a combination between facade and adapter, but that's another discussion. Anyway, why would you want to do that? Why not directly call the API with an HttpClient? Well, imagine that you need to execute something right before or right after calling the API with HttpClient, which isn't uncommon. When you have a proxy in between, the proxy can take on that responsibility. You don't have to change the Client class anymore. Let's have a look at the proxy pattern structure.

Structure of the Proxy Pattern

The first player in the proxy pattern is called the Subject. This is what defines the common interface between two other classes, the RealSubject and the Proxy. It's the contract both adhere to. Map this to a remote API call example, and you could see this as a service contract, say with one operation request. The subject can be written as an abstract class, but it can also be implemented as an interface. The RealSubject implements this interface. This would be the remote API service. This is the real object that the proxy represents. And then, we have the Proxy class. That one also implements this interface and therefore also has a request method. It thus provides an interface identical to the subject so that the proxy can be substituted for the RealSubject. It also maintains a reference to and controls access to the RealSubject, and it's potentially responsible for creating and deleting it. A few variations of this pattern exist. The example we just looked into is the remote proxy variation. Let's have a look at these variations first before we implement an example.

Variations of the Proxy Pattern

First, variation is the remote proxy. This is the one we used as an example. The idea here is that a client can communicate with the proxy, thus treating it as a local resource. The pattern hides the fact that the actual object exists in a different network space. The client doesn't need to know that a remote service is called. That's because the proxy hides away these network and protocol details. More often than not, these proxies are generated from some sort of service definition. That can be a WSDL or, more common these days, an OpenAPI, commonly known as Swagger, specification. The second variation is the virtual proxy. This one allows creating expensive objects on demand. You can look at it as a stand-in for an object that is expensive to create. So, performance optimization is a big advantage of this type of proxy. Another variation is a smart proxy. The purpose here is to allow adding additional logic around the subject. Typical examples are managing the caching of objects or locking access to shared resources. The last variation is the protection proxy. This one is used to control access to an object. It can be useful when objects should have different access rights. Like that, checking for that no longer has to be done by the client code or in the object itself. It's the protection proxy that's responsible for this. Sometimes the separation between these different types of proxies isn't always that clear. It's not uncommon for code generators to, for example, generate a proxy that both hides away the fact that the remote service is accessed and takes care of permissions and access restrictions. That, of course, is in violation of the single responsibility principle. A better

approach would be to chain proxies. In other words, proxy the proxy, sticking with a single responsibility per proxy. We'll cover that in a demo. These variations all have their own responsibilities and consequences. But as far as implementation is concerned, they all follow the same structure. So, what we're going to do in an example is implement two of these. We're going to implement a virtual proxy, and we're going to implement a protection proxy, as that one is very closely related to the smart proxy. That one is very closely related to the smart proxy, both allowing additional housekeeping tasks when an object is accessed. As an example, we'll use a document object, which represents a PDF or a text file. Imagine that this is an object that's expensive to create, which is pretty common. You might, for example, want to read out the underlying file from a disk as part of the creation process, and reading out a file from disk, that's pretty expensive. The virtual proxy pattern can be used to ensure that creation only happens when needed. Then, we'll add a business rule, stating that the document can only be viewed by people in the viewer role. We'll use a protection proxy for this, and we'll chain that with the virtual proxy. Let's start coding.

Demo - Implementing the Proxy Pattern

In this demo, we will implement the proxy pattern. Let's start by adding a Document class, our RealSubject. This consists of a few common properties you might expect from a document like Title, Content, AuthorId, and LastAccessed. We also add a private file name field to it so we know where to load it from. We don't want the document to be constructed without a file name having been passed through. So, we add a constructor to convey that. The constructor expects a `_fileName` parameter, and we set the private `_fileName` field to the passed through parameter. Up next, we want to load the document from disk. For that, we want to add a LoadDocument method, which expects the fileName. This effectively loads the document from disk. This is an expensive method. We fake that by adding a `Thread.Sleep` statement, so it takes a bit of time to execute this. Once the document has been loaded from disk between quotation marks, we fill out its properties. This method currently gets called from the constructor, so the expensive operation happens at the moment the document object is being constructed. We add one more method, something a client can call into to display the document. In that, we write out some property values to the console output window. That makes this a bit easier to test. And with that, we've got our RealSubject, the Document class. This doesn't have anything to do with a proxy yet, but what's coming up has. From this Document class, we can now extract an

interface. This will act as the subject from the proxy pattern structure. Both the RealSubject and the proxy must adhere to this. Let's say we want to put one method in it, DisplayDocument. LoadDocument is private, so the only one we're going to put in it is DisplayDocument. Seeing RealSubject must implement it, we do that. There we go. Let's give this a try already. Again, nothing proxy-related in there at the moment, so we simply write out we're constructing the document, we load it, and we write out that the document is being constructed. After that, we display it. Let's add a ReadKey statement, and let's give this a try. We see that the expensive operation happens when the document is being constructed and not when it's first being used, in other words, not when it's being displayed. This may not be what you want, as this can make applications that require a lot of objects like this unnecessarily slow. Expensive operations like loading files from disk may be executed without them being needed because if you don't display what's in the file, you don't really need it to be loaded from this either. So it's better to offload this to the moment the document is first accessed, and that's where the virtual proxy comes into play. Let's add a new class. We'll name it DocumentProxy. As the name implies, this is the proxy. It too must implement the IDocument interface, so let's do that. That means we need to implement a DisplayDocument method on it, but let's first construct our DocumentProxy. We do that by accepting a file name, but we don't construct the underlying RealSubject, our document, yet. So we don't call into the base constructor or anything like that. At this moment, the expensive operation has not been executed yet. We do need to implement this DisplayDocument method. In this method, we need the underlying document, which may not have been constructed yet. So, we add a private variable, _document, which will hold the document. It's nullable because the document hasn't been constructed yet after we leave the DocumentProxy constructor. Constructing the document, that's what we do when a method on the DocumentProxy is called, DisplayDocument. So, it's here that we check whether the document is null. If it is, that's when we create it. Afterwards, we call DisplayDocument on the _document variable. This means that the first time we call into this method, the document is constructed and the expensive operation will run. After that, the document stored in the _document variable will be used. Let's give this a try. For that, we write a bit of extra code in our program file. We write out an empty line, and then we use the DocumentProxy to call DisplayDocument on. As you notice, the way to interact with the document and the DocumentProxy is transparent for the client. The client executes exactly the same statements. So for the client interacting with one of both is the same. Let's give that a try, and there we go. The

expensive operation is only executed when we actually need it to be. So the advantage here is that the cost of creation of the actual object can be deferred to when it actually needs to be used. The proxy thus lazily initializes the actual object that's interacted with. And this brings me to a small improvement we can make, something we also did when we implemented the singleton pattern. .NET includes support for lazy initialization out of the box via `Lazy<T>`. When correctly used, it ensures that the object of type `T` is only used when needed and not when initialized, so we can change our `DocumentProxy` to support this. Instead of working on a document directly, we now want to work on a document that is of type `Lazy<Document>`. We also no longer need it to be nullable, as the `Lazy` object is constructed in the constructor of our proxy. We construct it by passing through an action, which tells it how it should construct an instance of `Document`. When the `Lazy<Document>` object is newed up, construction of the actual document doesn't happen yet. Our `DisplayDocument` method then becomes easier. We call into `_document.Value.DisplayDocument`. If it's the first time the value is accessed, and `Value` is our document instance, the actual `Document` object is created. Each time after that, the previously created instance will be used. Let's give this a try. We shouldn't have to change anything about our client code, and there we go. It still works as before, but this time in a thread-safe manner because that is what `Lazy<T>` makes sure of. Let's look into the protection proxy and proxy chaining in the next demo.

Demo - Chaining Proxies

In this demo, we will learn how to chain proxies. Let's think about a new requirement. Our document should only be displayed when users have the `Viewer` role. We could build this in the `Document` itself or in the `DocumentProxy`, but that would lay too many responsibilities on one class. Instead, we can load off this access right decision to a new proxy. So let me scroll down a bit, and let's add such a `Proxy`. We'll name it `ProtectedDocumentProxy`. These two should implement the `IDocument` interface, which makes this another proxy over the same type. We construct the `ProtectedDocumentProxy` by storing `fileName` and `userRole` and by constructing a `DocumentProxy`, which we also store. As we remember, the `DocumentProxy` is our proxy from the previous demo, and it's this one that makes sure that the actual document will not be constructed at the moment `DocumentProxy` is constructed. That also means that when the `ProtectedDocumentProxy` constructor is executed, the eventually underlying document isn't constructed yet. We are now effectively creating a chain of proxies, first to protect the

DocumentProxy, and after that, the `_documentProxy`. So, in `DisplayDocument`, we check the role and throw an `UnauthorizedAccessException` if it's not `Viewer`. Otherwise, we display the document. We surround this with a few `Console.WriteLine` statements so we can see exactly what happens when. And with that, we've chained our two proxies. Now let's test this. First, we new up a `ProtectedDocumentProxy`, passing through the file name and the role. We surround it with a few `WriteLine` statements so we can see what happens when. After the `WriteLine` statements, we call `DisplayDocument`. Seeing the role is `Viewer`, we should have access. And let's write something likewise, which will not give us access. All right, let's build and run this. We hit an `UnauthorizedAccessException`. That sounds like what we expect because the last proxy chain we newed up shouldn't give us access. Anyway, let's continue so we can see what's been written out to the console output window. The first two paragraphs are from the previous demo, so we're interested in the third paragraph here. We see that it's the `ProtectedDocumentProxy` that's being constructed. And once we enter `DisplayDocument` in the `ProtectedDocumentProxy`, the expensive action is executed, so that's the original proxy at work, and the content is written out. From this statement, we can nicely see the chain of proxies. And the next paragraph, as expected, tries the same, but throws in an `UnauthorizedAccessException` because we didn't pass through a role that would grant access. And with that, we've implemented proxy chaining. Let's continue by having a look at the use cases for the proxy pattern.

Use Cases for the Proxy Pattern

The basic use case for this pattern is when you need to add some form of access control to an actual object, something that's more sophisticated than a simple object reference. Let's have a look at some high-level use cases for the pattern variations. For the remote proxy, the typical use case is providing a local representative for an object in a different network address space. Any time you need to call into a remote API or service, this can be helpful. For virtual proxies, the main use case is creating expensive objects on demand. Smart reference proxies are useful for caching and locking scenarios, and protection proxies are useful when objects should have different access rules. A good real-life use case was the one we just implemented. Let's have a look at a few other real-life proxy use cases. Imagine you need to gather metrics. You could use a proxy for that. Use it to gather usage statistics and performance metrics about how an object is being used. This information can be valuable for

optimization, profiling, and fine tuning your system. Another good example of the proxy pattern you may encounter in your day-to-day developer life is a proxy for feature toggling. To implement feature toggles or feature flags in your application, you can use a permission proxy that checks if a user or a system has permission to access a particular feature or has the feature enabled. That allows you to enable or disable features dynamically without redeploying the application. You could also use a proxy for performance optimization. Consider a web app that fetches data from a remote server, a pretty common use case, in other words. To reduce network latency and improve performance, a cache proxy can store previously retrieved data locally. It checks if the requested data is in the cache and returns it if available, otherwise it fetches the data from the remote server and stores it in the cache for future requests. And lastly, a proxy can also be used for rate limiting. When dealing with external services or APIs that have rate limits, a throttle proxy can control the rate at which requests are made to stay within the allowed limits. This helps prevent overloading the external service, and it ensures that your application operates within its constraints. Coming up, let's have a look at the pattern consequences.

Pattern Consequences for Proxy Pattern Variations

What this pattern does is introduce a small layer or wrapper between a client and the actual object. The consequences from that, again, vary depending on the proxy type. For a remote proxy, a consequence is that the proxy hides the fact that an object resides in a different network space. For the virtual proxy, the consequence is that the object can be created on demand. And for protection and smart proxies, a consequence is that additional housekeeping tasks can be executed whenever an object is accessed. As a general consequence, the pattern allows introducing new proxies without changing the client code. That's the open/closed principle from the SOLID principles at work. It also has a few potential disadvantages. One is added complexity because a set of new classes is introduced. Another one is a potential performance impact because you're passing through one or more additional layers. Up next are the related patterns.

Related Patterns

There's two related patterns to the proxy pattern, and both of them are structural, the adapter pattern and the decorator pattern. The adapter provides a different interface to an object it adapts. Proxy, which is related, provides the same interface to the underlying object. Decorators have similar implementations as proxies, but serve a different purpose. As we remember, the decorator adds responsibilities to an object. The proxy, on the other hand, controls access to an object. So that's how these patterns are related. Let's have a look at the module summary.

Summary

We learned about a proxy pattern in this module. It's a very common pattern that's intended to provide a surrogate or placeholder for another object to control access to it. So, access control for whatever reason and that doesn't need to be permission-based is the main goal. The pattern comes in different flavors. The first variation is the remote proxy. The idea is that a client can communicate with a proxy, thus treating it as a local resource. The pattern hides the fact that the actual object exists in a different network space. The second variation is the virtual proxy. This one allows creating expensive objects on demand. You can look at it as a stand-in for an object that is expensive to create. Another variation is the smart proxy. The purpose of this one is to allow adding additional logic around this object. And lastly, we've got the protection proxy. That one is used to control access to an object. We implemented an example using a Document class, and we learned how to chain proxies, which is a good way to stick to the single responsibility principle. Up next is the last structural pattern, the flyweight.

Checkpoint 12 - Implementing the Proxy Pattern

Structural Pattern: Flyweight

Coming Up

In this module, we'll cover the flyweight pattern. That is a pattern that uses sharing to support large numbers of fine-grained objects efficiently. We'll start by describing the pattern using a real-life example, the characters that make up a document. While doing so, we will cover intrinsic and extrinsic

state. After that, we will map that to a structural example, and we will implement it. Then we will look into a special variation of the flyweight, the unshared concrete flyweight. We'll continue with learning about use cases for this pattern and pattern consequences, and we'll end the module by looking into related patterns. Let's dive in.

Describing the Flyweight Pattern

The flyweight pattern is a structural Gang of Four design pattern. Its intent is to use sharing to support large numbers of fine-grained objects efficiently. It does that by sharing parts of the state between these objects instead of keeping all that state in all of the objects. That requires a bit of explanation. Let's map it to a real-life situation. It's not that hard to come up with a typical good real-life example. We're going to use this pattern to share the characters of a document. That is a variation of the example that was used in the 1994 book. Afterwards, we will see why a lot of potential real-life situations that could call for this pattern actually might not be the best possible fit. Anyway, in our case, it does look like a good fit. If each character in a document would have to be a new object instance, the amount of these that are required would explode. We'll start with an `ICharacter` interface. All characters must implement this. This can also be an abstract base class, of course. In our example, there's one method on it, `Draw`, which gets the `fontFamily` and `size` passed through that the character needs to draw itself in. Implementations of these are `CharacterA`, `CharacterB` and so on. We can consider these as shared objects that can be used in multiple contexts simultaneously. Each of these acts as an independent object in each context. In that regard, it cannot be distinguished from an instance that is not shared, and shared it is. Because even though our document might have lots of A, B, and C characters and so on, we will hold on to one instance of each related class and not more. The key concept here is intrinsic versus extrinsic state. The intrinsic state can be described as state data that is independent of the context. In other words, it's data that doesn't change if the object is used in a different context. The character itself, the glyph, if you want, A, B, C, etc. remains as is. That is intrinsic state. Extrinsic state then is data that does vary with context. So, different class instances might have different extrinsic state data, which cannot be shared. In our case, that would be, say, the font family and font size. We could expose that by adding a `Draw` method to the character that accepts this `fontFamily` and `fontSize`, set it, and draw the character. But with that, we're not there yet. We need something that will manage these flyweights for us, a component that will parse the document and for

each letter in the document check whether the underlying character has already been instantiated, and if it has, ensure that it can be reused. That is our CharacterFactory. And lastly, there's, of course, a client, something that consumes these characters and the factory. This is the example that we'll implement. Now, there's a lot more examples you can think about that you can put in a pattern like this. Think about products with a fixed category and non-fixed weights and names, an ordering system with a few fixed intrinsic values per order, a library system and so on. We will look into more examples once we get to the use cases clip. The thing is though, often the flyweight pattern might not be the best fit. There's a few things to consider before implementing this pattern. One, does the app use a large number of objects? In our case, documents definitely use a lot of character objects, potentially. Two, are storage costs, that's memory use, high because of the large amount of objects? In case of a document, that's, again, a yes. Three, can most of the object state be made extrinsic? Well, definitely in our case. The actual character glyphs are intrinsic, but things like color, font, size, etc. can be made extrinsic. Four, if you remove extrinsic state, can a large group of objects be replaced by relatively few shared objects? In our case, that's true again. In that case, we just end up with all the different characters in the alphabet. And five, does the application require object identity? If it doesn't, it's a good candidate. And that's true for our example as well. The application doesn't need to know an ID or unique way to identify each character object. That's five things to keep in mind. And the general rule is only use the flyweight pattern when all five are true. Truth be told, it's a general rule. It's not set in stone, so you can divert from that. But let's say that if only three of these match your use case, you may want to consider other patterns. I've got to say I don't have a lot of need in my day-to-day work life for this pattern, but it does have its usefulness, as you hopefully agree with me from this example. All right, let's map our example to the pattern structure.

Structure of the Flyweight Pattern

The first player in the flyweight pattern is the flyweight itself. That's an abstract base class or interface through which flyweights can receive and act on extrinsic state. In our case, that could be an ICharacter interface that exposes a Draw method, which accepts a fontFamily and a fontSize. The second player is then the concrete implementation, the ConcreteFlyweight. These are our character A, character B, etc. classes, implementing the Flyweight interface and thus implementing the Draw method. It also adds storage for intrinsic state, our character itself, the A glyph, B glyph and so on.

Then we have the FlyweightFactory. That's our CharacterFactory. It creates and manages flyweights, and it ensures that they are properly shared. In other words, when the flyweight doesn't exist yet, it creates an instance of it. And otherwise, it reuses the previously created one. On it, typically a method to get a flyweight exists. And lastly, we have the client itself. This one maintains a reference to flyweights and computes or stores the extrinsic state of flyweights. Let's implement this in a demo.

Demo - Implementing the Flyweight Pattern

In this demo, we will implement the flyweight pattern. We'll start by creating the flyweight itself. This is the interface concrete implementations must adhere to. We'll name it ICharacter, and we ensure that one method is required, Draw, which gets the extrinsic state passed in, fontFamily and fontSize. Then on to the concrete implementations. We won't do this for all letters of the alphabet. Two, A and B, will suffice. So we create a new class, CharacterA, and have it implement ICharacter. We add a field for the _actualCharacter. This is intrinsic state. The two other fields, fontFamily and fontSize, are extrinsic state. These can be changed, and changing that is done via the Draw method. We get the extrinsic state passed in, we store it, and we write out a message to the console output window to mimic drawing the actual character on screen. And then we do the exact same thing for CharacterB. Up next is implementing the factory. Let's name that class CharacterFactory. A dictionary, which provides fast lookup by key is, in my opinion, a good way to store our concrete flyweights. As key, we give a character and as value, an ICharacter. We're working on interface here and not on implementation. We add one method to it, GetCharacter, accepting an identifier and returning an ICharacter implementing instance or potentially null if we cannot match the identifier to a flyweight. In it, we first check whether our character is already in the dictionary, and we return that instance if it is. While doing so, we write that out to the console output window. Like that, we can easily follow along with what's happening. If the character requested isn't in the dictionary, we create the correct instance, store it, and return it. A switch statement is a convenient way of handling something like that in our case. When none of these options apply, we return null. That leaves us with writing the client code to test. The first thing we do is create a string, which we're going to use for testing. And after that, we new up the CharacterFactory. For that, we need to import the Flyweight namespace. Up next, we use the CharacterFactory to get a flyweight. We pass through the first character of our aBunchOfCharacters string. Seeing this is the first time the flyweight should be created. Afterwards,

we pass through the extrinsic state, which is the `fontFamily` and the `fontSize`, to the `Draw` method on the newly created `characterObject`. After that, we do the exact same thing for the other three characters in our `aBunchOfCharacters` string. This should result in `CharacterA` and `CharacterB` being newed up. The first two times we call into `GetCharacter` on our factory, and those two instances being reused with another extrinsic state, the third and fourth time we call into `GetCharacter` and `Draw`. Let's give that a try. And as we can see, that is indeed the case. We see two `Character` construction statements and two `Character` reuse statements, yet effectively drawing the character is different all four times. So far, so good. Let's look into a variation, the unshared concrete flyweight.

Working with an Unshared Concrete Flyweight

We're looking at the pattern structure we ended up with. I must make a confession. When we looked into the pattern structure, I skipped something to make the pattern easier to grasp, the `UnsharedConcreteFlyweight`, which also implements `flyweight`. This type of flyweight implementation does not need to be shared. The flyweight interface enables sharing, but it does not enforce it. The `UnsharedConcreteFlyweight` is such an implementation. It enables acting on extrinsic state while having unshareable intrinsic state. That may sound a bit strange, so let's clarify this with an example. Say we want to introduce the concept of a paragraph to our document. A paragraph contains a location in the document, and it contains characters, so our concrete flyweights, but it doesn't contain shareable state. Its location and the characters it contains are linked to the paragraph. We cannot reuse the paragraph instance. A new paragraph means we need a new instance. As it does contain concrete flyweights or characters, it can implement the same flyweight interface and use the method from that interface, `Draw`, to pass on the passed-in extrinsic state to the flyweights. Now obviously, because the state is not shared, we use the storage advantage the flyweight pattern is used for. But we do keep the advantage that working with the flyweight is transparent to the client, whether it's unshared or not. Let's implement this example.

Demo - Supporting an Unshared Concrete Flyweight

As unshared concrete flyweight, we'll create a `Paragraph` class. This holds a location, for example, 1, 2 or 3. It stands for where in the document the paragraph lives. Next to that, a paragraph also holds a

set of characters. We add a constructor that accepts these two. Then we implement `ICharacter`. It's a flyweight, so it should implement this. And this means we have to add a `Draw` method. Drawing a paragraph, in our case, means that we want to ask each character to draw itself. So what we do here is pass through the extrinsic state to the `Draw` method on each character, and that's it already for our paragraph. On to the factory. Our `CharacterFactory` doesn't know how to create this unshared concrete flyweight yet, so we need to teach it how. For that, we add a new method to the factory, `CreateParagraph`. A paragraph is always a new instance. When we're working with an unshared concrete flyweight, we're always working with new instances. We are not sharing state in this case. So, our `CreateParagraph` method accepts a location and a list of characters and simply news up the unshared concrete flyweight, the paragraph, and returns it. Then, let's use this. We first create an unshared concrete flyweight. We do that by calling into `CreateParagraph`, passing through a list of characters and location for the flyweight. The warning here means that `characterObject` might be null. If we don't want to allow that, we can add a null check, but for demo purposes, this isn't required. It doesn't have anything to do with the pattern implementation as such. And then we call `Draw` on the paragraph. As you notice for the client working with a flyweight or an unshared flyweight, it's completely transparent. What's returned from `CreateParagraph` is an `ICharacter`, which is the same interface as is returned from the call into `GetCharacter`. So as far as the client is concerned, these can be treated in a likewise manner. Let's build around this. And there we go. We see that we are drawing a paragraph, which results in drawing of the first character, as that's the character object we pass through. Let's have a look at the different use cases for this pattern.

Use Cases for the Flyweight Pattern

The use cases for this pattern at a higher level match the reasoning you'd use in the first place to decide on whether your case is a good match for this pattern. And we actually already looked into that when we described the pattern, so let's recapitulate that. One, your application uses a large number of objects. Two, storage costs are high because of the large amount of objects. Most of the object state can be made extrinsic. And if you remove extrinsic state, a large group of objects can be replaced by relatively few shared objects. Lastly, the application doesn't require object identity. If those five rules check out or let's say at least four of them, you've got a good use case. A library system might be a good use case, a product categorization system might be a good use case, but it's not a good use

case by default. That will always depend on your specific requirements against these five rules. Let's look into a few other potential use cases you might use these for, always keeping these five rules in the back of your head. Music playback is a potential use case. You can use the flyweight pattern to optimize memory use when playing large playlists. Reusable elements like album cover images, song metadata or audio codec information can be represented as flyweights. That allows the media player to conserve memory and thus deliver a smoother user experience. An inventory management system could be another potential use case. Imagine you're building one for a retail store. You could represent products with the same attributes as flyweight objects. Each product instance can then reference the shared flyweight, reducing the memory required to store product data and making it easier to update and maintain the inventory. Another potential one is document parsing and document analysis. When you're processing a large document or a dataset such as an XML or JSON file, you could apply the flyweight pattern to efficiently manage the repetitive parts of the data structure. Common elements like tags or attributes in XML can be represented as flyweights while unique data values are stored separately. This, again, reduces memory footprint and speeds up parsing and analysis. And lastly, imagine you're building a game. If you're doing that, there may be numerous instances of objects with similar attributes like trees, rocks or bullets. By using the flyweight pattern, you can create shared flyweight objects for these common elements and store unique attributes for each instance. This, again, reduces the memory footprint of the game and thus improves rendering performance. Let's check out the pattern consequences.

Pattern Consequences

A positive consequence of this pattern is that you may save lots of memory when you use this for an applicable use case. That being said, it might be that you'll end up with some more processing costs because of the need to inject or compute extrinsic state if that state was stored as intrinsic state previously. That's a disadvantage, yet it's almost always offsetted by the reduced storage costs, in other words, by the reduced memory use. Another disadvantage is that this is quite a complex pattern, which means that your codebase as a whole will become a lot more complicated. Let's look into a few related patterns.

Related Patterns

The first related pattern is the state pattern. When you implement it without instance variables in the state objects, you're essentially making these state objects shareable, which makes them flyweights. And the second related pattern is the strategy pattern. A strategy can also be implemented as a flyweight. Up next is the module summary.

Summary

The flyweight pattern is a pattern that uses sharing to support a large number of fine-grained objects efficiently. Two key concepts related to the pattern are intrinsic and extrinsic state. The intrinsic state can be described as state data that is independent on the context. In other words, it is data that does not change if the object is used in different contexts. Extrinsic state then is data that does vary with context. So, different class instances might have different extrinsic state data, which cannot be shared. The intrinsic state is reused in a way, while the extrinsic state varies. We implemented an example using reusable character objects in a document. Afterwards, we looked into a variation working with unshared concrete flyweights, in other words, one that doesn't have state to share. We added the notion of a paragraph to our document to illustrate this. And with that, we covered all the structural patterns. Time for the next category, behavioral patterns. We'll start with arguably the most commonly used design pattern of the course, the template method.

Checkpoint 13 - Implementing the Flyweight Pattern

Behavioral Pattern: Template Method

Coming Up

In this module, we'll dive into the template method pattern. Its intent is to define a skeleton of an algorithm in an operation, deferring some steps to subclasses. This pattern is actually one of the fundamental techniques for code reuse. I dare to guess that you've already used it hundreds of times.

Let's have a look. We'll start by describing the template method pattern. For that, we'll use a real-life example, parsing mail for a startup company's software that wants to revolutionize the way we look at email. We'll map that to the pattern structure, and we'll implement it in a demo. We'll continue with learning about use cases for this pattern and pattern consequences. We'll end the module by looking into related patterns.

Describing the Template Method Pattern

The template method pattern is a behavioral pattern from the Gang of Four library of design patterns. Its intent is to define the skeleton of an algorithm in an operation, deferring some steps to subclasses. It lets subclasses redefine certain steps of an algorithm without changing the algorithm structure. Let's clarify that with an example. Imagine that we're writing part of an application that wants to make working with email easier by accumulating mail across different servers and analyzing them. The idea is that the company's proprietary artificial intelligence will ensure that your inbox services the emails that are important to you. To be able to do that, the software must connect to different mail servers and parse emails. So, we've got a few classes, `ExchangeMailParser`, `ApacheMailParser`, `EudoraMailParser` and so on. For the sake of the demo, we'll assume that a parser parses one HTML mail at a time by accepting an identifier. The thing is, a lot of what has to happen in the algorithm to parse one mail is the same for these three classes. First, the server needs to be found, then we need to authenticate to the server. That's probably different for each type of mail server. And then the mail body must be parsed. And it's highly likely that this is exactly the same for each of these servers, but we don't want that kind of code duplication. What we can do is create an abstract base class with a template method on it. That template method, say `ParseMailBody`, accepting an identifier calls the other three methods in a certain order. The other three methods are thus also defined on this abstract class. Subclasses like `ExchangeMailParser` or `EudoraMailParser` call this template method on the base class to parse mail, yet we allow these subclasses to vary the algorithm by overriding abstract methods. We'd make the `AuthenticateToServer` method abstract because this differs for every class. That means that each subclass must implement this method with its own implementation. Moreover, what we can also do is allow subclasses to optionally overwrite the other methods as well in case they would require changes. We do that in code by making them virtual. In Gang of Four lingo, these types of methods or operations are called hooks. The only thing that absolutely must not be overridden is

the template method itself, so ParseMailBody. And this is what a template method pattern is, a straightforward, often used pattern in OO design. Let's map this to the pattern structure.

Structure of the Template Method Pattern

There's only two players in this pattern structure. The first one is the AbstractClass. That is our mail parser. It defines abstract primitive operations that concrete subclasses define to implement the actual steps of the algorithm. Our AuthenticateToServer method is an example of that. A variation is that not all methods are primitive operations that are abstract and thus must be overridden, but some methods are virtual so they can be overridden if needed. The other methods we had could optionally be examples of that. The AbstractClass also implements a template method, which defines the skeleton of the algorithm. That's the order in which the other methods are called. It thus calls primitive methods, as well as other methods defined on the abstract class or on other objects. The second player then is the ConcreteClass. This implements the primitive operations to carry out subclass-specific steps of the algorithm. They map to our ExchangeMailServer, ApacheMailServer, and EudoraMailServer classes. And with that, we know enough about the pattern to implement it in a demo.

Demo - Implementing the Template Method Pattern

In this demo, we'll implement the template method pattern. We'll start with an AbstractClass. That's the AbstractClass component from our diagram, and we name it MailParser. On it, we define three different methods to start with. FindServer is used to find the MailServer. This method tends to be the same across concrete implementations, but we mark it as virtual just in case there's a MailParser for which this differs. AuthenticateToServer is abstract. This is a primitive operation, which differs for each subclass. We want subclasses to implement it, and that's why we mark it as abstract. And then we have ParseHtmlMailBody. This method remains the same across different implementations, so we don't mark it as abstract nor as virtual. And then we add one more method. We name the method ParseMailBody, and this is our template method. In it, we define the algorithm by calling the other methods in order. That's it for our AbstractClass. Now let's create concrete implementations of it. We start with the exchange implementation. Here, we override one method, the abstract one, to authenticate to the MailServer. The ApacheMailParser follows likewise principles. We only need to

override the abstract method. And then we have the EudoraMailParser. This one is slightly different. We're assuming that finding such a server takes a bit more effort. Luckily, we can override the virtual FindServer method from the base class to achieve this. Note that the template method, ParseMailBody is available on all these classes through their base MailParser class. That is the one clients will use. Nothing what we did changed the order of what's in that method, and we cannot override it nor change it. That's it for our pattern implementation. Let's test this. The first thing we'll try is our exchangeMailParser. We need to import TemplateMethod namespace for that, so let's do that. And on it, we call the template method, ParseMailBody. We pass through an identifier as well. This one will be written out to the console window. So like that, we can nicely see what's going on. And now we do the same for the ApacheMailParser and the EudoraMailParser. When we run this, we should be able to nicely see that subclass in the base class, and overriding an extra method for the EudoraMailParser will have an effect, even though we are calling the same template method three times. Let's give that a try, and there we go. As we notice, the AuthenticateToServer method is different for each of these implementations, and we also notice that for the EudoraMailParser, finding the server is different as well. With that, we know how to implement the template method. Let's have a look at the use cases for this pattern.

Use Cases for the Template Method Pattern

Let's start with pattern use cases at a generic abstract level. You may want to consider using this pattern when you want to implement invariant parts of an algorithm only once, and at the same time, you want to leave it to subclasses to implement the rest of the behavior. Those are the parts that can vary like our AuthenticateToServer method. Another use case would be when you want to control which part of an algorithm subclasses can vary. Those are the methods you make virtual or abstract. Lastly, you could use it when you find yourself in a situation where you have a set of algorithms that don't vary much, maybe only in one or two methods. Those two are the methods you'd make abstract or virtual. Now let's look at a few real-life use cases where you might consider this pattern. Truth be told, this design pattern is so common that you've probably already encountered it yourself in your own applications, but let's look at a few use cases anyway. I'm going to stick to the same types of examples I've used in some of the other modules. A first good use case would be document generation systems like PDF or Word document generation. These often use the template method

pattern. The high-level algorithm specifies the structure of the document, header, body, footer and so on, and subclasses or interface implementations provide the content for each section. This way, you can generate different types of documents where each type could be represented by a specific subclass. Another common one, albeit something we don't necessarily write as much ourselves these days anymore, are sorting algorithms. I still remember sorting algorithms like the bubble sort. Sorting algorithms like a quick sort, merge sort or bubble sort can implement a common sorting interface using this template method pattern. The high-level algorithm defines the steps for sorting while the subclasses or implementations provide the specific sorting logic for different scenarios. And then another good one would be an online shopping cart. In an ecommerce application, the process of adding items to a shopping cart and checking out can be implemented with a template method pattern. The high-level algorithm defines the steps like adding an item, calculating the total, applying discounts, processing payment while subclasses or interface implementations provide specific implementations for different types of products or payment methods. The last use case I'd want to touch upon are automated testing frameworks. These often use the template method pattern for defining test case structures. The high-level test case template specifies the setup, execution, and teardown phases while individual test cases, which would be implemented as subclasses or interface implementations, provide the specific test steps and assertions. Let's continue with pattern consequences.

Pattern Consequences

Using this pattern comes with a few consequences. First of all, template methods are a fundamental technique for code reuse, especially when creating class libraries. And that's because template methods are how you factor out common behavior by pulling it out into a parent class, thus removing code duplication. A potential negative consequence is that template methods cannot be changed. The order of the methods they call, so the skeleton, is fixed, and that may be too strict for some clients.

Related Patterns

The template method pattern is related to a few other ones. First up, factory method and template methods are related. You can even look at the factory method as a specialization of the template method aimed at object creation. Also, often template methods use factory methods as part of their

skeleton structure. It's also related to the strategy pattern. Both are used to vary an algorithm. Template method allows varying part of an algorithm through inheritance, the subclass that implements the abstract base methods. It works at class level, so this is static. Strategy, on the other hand, is based on composition. By providing different strategies, the object's behavior can be altered. It works at object level, so it's dynamic. Behaviors can be switched at runtime with a strategy pattern, and you can't do that with a template method. On to the summary.

Summary

In this module, we covered arguably the most often used behavioral pattern, the template method pattern. Its intent is to define the skeleton of an algorithm in an operation, deferring some steps to subclasses, so it lets subclasses redefine certain steps of an algorithm without changing the algorithm's structure. As far as implementation goes, the template method must be defined on the abstract base class, and it cannot be overridden. Methods that differ per subclass must be marked with the abstract keyword and must be implemented in subclasses. Lastly, you may let subclasses change base behavior by marking some methods as virtual. Up next, the strategy pattern, one of the patterns related to this one and also very commonly used.

Checkpoint 15 - Implementing the Template Method Pattern

Behavioral Pattern: Strategy

Coming Up

In this module, we'll cover the strategy pattern, also known as the policy pattern. It's one of the most commonly used behavioral patterns. The idea behind it is that it will let an algorithm, which is named the strategy, used by clients to achieve something vary independently from the clients that use it. We'll start by describing the pattern using a real-life example, exporting an order to a certain format. After that, we will map that to a structural example, and we'll implement it in a demo. We'll also implement an alternative approach to the original pattern template. We will continue with learning about use

cases for this pattern and pattern consequences, which can be positive or negative. And we will end the module by looking into related patterns. Let's dive in.

Describing the Strategy Pattern

The strategy pattern is a behavioral pattern from the Gang of Four library of design patterns, and it's probably one of the most often used behavioral patterns in existence. Its intent is to define a family of algorithms, encapsulate each one, and make them interchangeable. The pattern lets the algorithm vary independently from clients that use it. But in other words, you define a family of algorithms, put each one in a separate class, and make them interchangeable. That definition might sound a bit weird at first, but just think about technically implementing an interface, and it already becomes a lot clearer. Imagine the following use case. You're working on part of a system that manages orders. At a certain moment, a business requirement comes in. The order needs to be exportable as a CSV file. You solve that by adding an export method on the order that creates this CSV file. So far, so good. A bit later, another requirement comes along. You need to support exporting to JSON as well. This is still manageable. You can accept the parameter in the Export method, exportFormat, and depending on the parameter, execute the code. Yet, two formats still aren't good enough. A few weeks later, XML must be supported, and now it's slowly but surely starting to become more difficult to manage. You could continue on the same route, adding an additional format type and extending the conditional statement you've got inside of the Export method, but you're already noticing some disadvantages to this approach. The Order class is becoming more complex with all this export logic contained in it. Moreover, when you want to change export logic or add a new way to export, you always have to change the Order class. You can partially solve this with the single responsibility principle. You can put the different types of export functionalities in classes of their own, CSVExportService, JsonExportService, and XMLExportService. You could then store instances of those on your Order class, and when exporting, use the correct instance. This means that the Order class is now responsible for the lifetime management of the different export services, and you've tightly coupled the Order class to these implementations. That still makes it hard to add export functionality for new formats. Enter the strategy pattern, which solves these issues. The first thing we'll do is declare an interface, IExportService. On it, there's one method that needs to be implemented, Export, which accepts the thing to export, the Order. Our three or potentially more export services now become

implementations of this interface, CSVExportService, JsonExportService and XMLExportService. In the Export method implementation, the concrete export is handled. The Order class then has a property of type IExportService, which is set to one of the concrete implementations. Whenever Export is called on Order, the currently set IExportService implementation is used to handle the export. This means that the Order object isn't responsible for selecting the appropriate algorithm or interface implementation for the job. It's the client that passes this implementation to the Order class. The Order class itself doesn't need to know about it. All it knows about is the interface, not the implementation. We've thus achieved loose coupling, making the order independent of the implementation, which makes it easier to add new implementations or modify existing ones. There's no need anymore to change the Order class to do so. Now if you think about this in modern architectures, a client passing the correct implementation to the Order class is often replaced by constructor injection. In other words, you let the client depend on the IExportService interface and make your IoC container responsible for providing the correct instance. Let's map this to the pattern structure.

Strategy Pattern Structure

Let's start with our IExportService. That, in pattern lingo, is named the strategy. It declares an interface common to all supported algorithms. The context, which is our Order class, we'll get there soon enough, uses this interface to execute the concrete algorithm. And that brings us to the ConcreteStrategy. These are implementations of the strategy interface. In other words, they implement the algorithm using the strategy interface. This maps to our different service implementations. And lastly, we've got the Context, which maps to our Order class. This is configured with a ConcreteStrategy object, and it maintains a reference to a strategy object. That's what the arrow from Context to Strategy means. It also often contains a way for the strategy to access its data. In our example, that was the Export method. It would require the data that needs to be written to the file, for example, by passing through the full object that needs to be exported, which is the order itself. And with that, we know about the pattern structure. Let's implement our real-life scenario, and let's also implement a variation of it in the upcoming two demos.

Demo - Implementing the Strategy Pattern

Let's start with creating the strategy. That's our `IExportService` interface. We're using an interface here, but the pattern can also be implemented with an abstract base class in case you want to provide some out-of-the-box implementation of parts of its functionality. I defined one method on it, `Export`, which accepts an `Order` object, as it needs its data to be able to export it. This is just one way of providing an interface to the `Order` data. Another way could be to accept just the property values that need to be exported. In our case, we're mimicking that we need access to all properties of the `Order` object. We haven't got that `Order` class yet, but we'll get to that. In our real-life example, we had three different formats to export to. So, let's create implementations of the strategy for those. These are concrete strategies. So we've got one concrete strategy, `JsonExportService`, which implements the `IExportService` interface. That means we need to implement the `Export` method, which we do. Our second concrete strategy is the `XMLExportService`, which follows likewise principles as the `JsonExportService`. And the last one is our `CSVExportService`, which also follows the same principles. We implement one method on it, `Export`. And maybe now it's time to get that order to resolve, so let's add an `Order` class, that `Order` class that maps to the context component of our pattern structure. It's a pretty straightforward class. We add a few common order-related fields and a constructor. The important part is what's coming now. We need to add one more property to our `Order` class. And this one is of type `IExportService`, our strategy. It will be set from outside of our `Order` class, so it is nullable. And then we add an `Export` method on the `Order` class. As we learned on the slides, it's in this export method that we use the strategy to export the order. Important to know is that the `Order` class doesn't know about the concrete implementation of the strategy that's going to be used. All it knows is that something of type `IExportService` will be used to export itself. And with that, we've created all the different moving parts of our pattern. Let's add a bit of code to test this. So let's new up an order, and let's try exporting it. It's up to the client to provide the concrete implementation of the strategy, and we do that by setting the `ExportService` property on our order. We set it to an instance of `CSVExportService`, and we try exporting. Afterwards, we set the `ExportService` to a new instance of `JsonExportService`, and we try exporting again. Let me add a `ReadKey` statement here, and let's give this a try. And there we go. This works as expected. Our different strategy implementations made sure that the correct export service was used. What would we now have to do to add a new export service? Well, the nice thing is that we no longer need to make changes to the `Order` class, as it's not tightly coupled to an implementation of `IExportService`. Note that this implementation does mean that `Export`

can be called without an export service being set. To avoid that, we could provide a default value. For example, in the Order constructor, we can set the ExportService to a new instance of CSVExportService, stating that that will be our default value. Another approach would be to not allow the order to be constructed without providing an IExportService implementing object. When we implement it like this, it's still the caller that provides the default and not the Order class itself. That being said, these days dependencies like this are often injected via an IoC container, which tend to have their own null checks in place. But even then, it still remains a good practice to check for null reference issues. Let's get rid of this to ensure that our demo code still builds, and let's continue with looking into a small variation of this in the next demo.

Demo - Implementing a Strategy Pattern Variation with a Method Parameter

In this demo, we will look into a variation of the pattern, which is still a valid strategy pattern implementation. What we can do is have the Export method accept an IExportService object instead of storing it on the Order class. We add a null check, and instead of using the ExportService property on the Order class, we used the passthrough instance. That means we can also get rid of this property. There we go. This is a variation of the template, and it thus looks a little bit different from the structural design pattern. But in essence, we are still leveraging the strategy pattern here. The intent is still the same. Let's change our test code to accommodate for this adjustment. So, we don't have an export service property to set anymore. We simply pass through an instance to the Export method, and let's give this a try. And there we go. This still works as expected. This was a very small change, but it is one that you'll often encounter in today's codebases. In essence, all these methods that accept an interface instead of an implementation to achieve a task are thus implementations of the strategy pattern, and that is why this pattern is so commonly used. Let's look into a few of those use cases next.

Use Cases for the Strategy Pattern

Let's start with pattern use cases at a generic abstract level. You can use this pattern when many related classes differ only in their behavior, so not in their interface. Strategies provide a way to configure a class with one of many behaviors, as we've seen with our three different export service

implementations. You can also use it when you need different variants of an algorithm, which you want to be able to switch at runtime, an export algorithm to three different formats, for example, again, as we've seen. Another good use case is when your algorithm uses data, code or dependencies that the clients should not know about. Imagine that exporting to, say, a proprietary format needs some additional configuration or properties. The clients don't need to know about this nor should they be allowed to change it. When they're working on the interface, they don't know about these. They only know about the Export method existing. Lastly, use it when a class defines many different behaviors, which appear as a bunch of conditional statements in its methods. In other words, when you're dealing with a large if or switch state, and that's the example we started with. If export format is CSV, then do this. If it's JSON, then do that and so on. You can use the strategy pattern to avoid this. All right, now let's see if we can find a few more common real-life use cases where you might consider this pattern. I'll start with compression algorithms. In a file compression utility, you can have multiple compression algorithms. Think about zip, gzip, etc. Each of these algorithms can be encapsulated as a strategy, allowing users to choose the compression method when compressing or decompressing files. The next example is a payment processing system. Consider an ecommerce application that accepts various payment methods, credit cards, PayPal, Apple Pay, and more. Each payment method can be implemented as a separate strategy. Depending on the user's choice or availability, the system can switch between different payment processing strategies seamlessly. And then we've got a travel booking system. A travel booking system might offer different strategies for calculating the cheapest or fastest route between two locations. Strategies could be based on different factors like cost, time or mode of transportation. And lastly, imagine if you're in video game development. In video games, AI behavior can vary based on the game's complexity and goals. Strategies can be used to implement different AI algorithms for path finding, decision-making or enemy behavior. The game engine can switch between AI strategies based on the game state or the difficulty level. And with that, we can continue with the pattern consequences.

Pattern Consequences

The strategy pattern offers an alternative to subclassing your context. Continuing with our example, we could have created an XML order, a CSV order and so on. But that would have hardwired the behavior into the context, which is the order, and that makes it harder to understand and maintain that

context. The strategy pattern avoids that. In other words, what you're doing here is replacing inheritance with composition, and that is one of the things the Gang of Four favors in OO design. Important as well is that new strategies can be introduced without having to change the context. That is the open/closed principle from the SOLID design principles at work. Strategies also eliminate conditional statements. You don't need a conditional statement anymore to select the required behavior. It's the client that injects it or whichever dependency injection system you want to use. Strategies can also provide a choice of implementations with the same behavior if needed. The client can then choose which one fits best at runtime. And that brings us to a drawback. If it's the client or the IoC container that injects the strategy to use, it must be aware of how different strategies behave differently, which means that they can be exposed to implementation issues. Therefore, only use this pattern when the variation in behavior is relevant to clients, otherwise you're giving them knowledge they shouldn't have. There's also a bit of overhead in communication between the strategy and the context. For example, when creating export services, some of them might not export the full order, even though we do pass it through. And lastly, this pattern does introduce an additional number of objects. If you only have a few algorithms, it might not be worth the additional complexity. Let's have a look at the related patterns.

Related Patterns

The Gang of Four only defines one related pattern, which is the flyweight, and that's simply because strategy objects often make good flyweights. Yet, in my opinion, there's more related patterns. For one, strategy, bridge, and state are all related and look similar because they are based on composition, even though they solve different problems. Plus, the strategy pattern is also related to the template method pattern. Both are used to vary an algorithm. Template method allows varying part of an algorithm through inheritance, the subclass that implements the abstract base methods. It works at class level. Strategy, on the other hand, is based on composition. By providing different strategies, the object behavior can be altered. It works at object level, not at class level, so it is dynamic. Behaviors can be switched at runtime. Time for the module summary.

Summary

We learned about the strategy pattern in this module. Its intent is to define a family of algorithms, encapsulate each one, and make them interchangeable. Strategy lets the algorithm vary independently from kinds that use it. It's a commonly used pattern these days. The implementation is pretty straightforward. Of importance here is that the variation in which we injected the concrete strategy via a method parameter is a very common approach these days and still a valid strategy pattern implementation. Let's continue with another often used pattern, the command pattern.

Checkpoint 16 - Implementing the Strategy Pattern

Behavioral Pattern: Command

Coming Up

In this module, we'll cover the command pattern, also called the action or transaction pattern. It allows us to encapsulate requests as objects. I'm Kevin, and I'll guide you through it. We'll start by describing the pattern using a real-life example. As a real-life example, we'll use UI interaction, clicking a button to add an employee to a list. After that, we'll map that to the pattern structure. We'll also implement our real-life example in a demo, and we will look into a variation, which will allow us to support undo by using a command manager. We'll continue with learning about use cases for this pattern, pattern consequences, and lastly, related patterns. Let's dive in.

Describing the Command Pattern

The command pattern is a behavioral pattern from the Gang of Four library of design patterns. Behavioral patterns define ways to communicate between classes or objects, letting you concentrate on the way objects are interconnected. The intent of the pattern is to encapsulate a request as an object, thereby letting you parameterize clients with different requests, queue or log requests and support undoable operations. It thus turns the request into a standalone object. And once we've got an

object, we've got something to work with and/or pass around. Let me clarify this a bit. A very typical example of a command is clicking a button in a user interface. For example, to open a file, to add a product to a shopping cart, to add a new employee to a list and so on. In a naive implementation, we could just write the code for all of that in the button itself. Yet, this comes with a few potential issues. You might want to separate concerns by not having the code for handling the request, which is a button click, at the same location as the button itself. You might want to be able to easily reuse the actual code that needs to be executed. Your application architecture might not even technically allow you to implement the code for a button click in the same place as where the button resides, for example, when you've separated out the buttons or the menu in a separate project that doesn't know about the receiver of the actual request. All these are cases where the command pattern comes in. It allows you to decouple the request of an action from the receiver. It is a very common pattern in mobile or rich UI development where you would typically use the model-view-view-model pattern combined with the command pattern for transferring UI interactions to your view model where the actual command is implemented. A command would be bound to some UI XAML code, or if that's not possible, it may be manually executed on a button click. You can see examples of those two on screen now. Don't worry if that code doesn't look familiar. It's just here for illustration purposes for those of you who've got some experience with the MVVM pattern and/or XAML development. Let's learn how we can create these commands. Say we're building an employee manager application in which we want to add an employee to a list of employees a certain manager manages when a user clicks a button. The object that invokes the command could be a button on the user interface to which a command is bound or on which a command property is set, adding an employee to that list that is executing the command. So, we would have an interface, `ICommand`, containing one method, `Execute`, to execute the command. The interface could also be implemented as an abstract class, as we know by now. A good name for the command, in our case, could be `AddEmployeeToManagerList`. This class implements the `ICommand` interface and executes the action. That means effectively adding the employee to a manager's employee list. For that, the command needs to know something. It needs to know who the employee is and who the manager is. That's additional state. That's also stored at that level. Once our command is being executed, something needs to happen. The employee needs to effectively be added to the manager's employee list. We could use a repository for that on which we can call a method to add the employee. And that brings us to the final part, which is the client

application. It creates the commands. In our case, it would pass the repository through to the command when it creates the command, typically via the command constructor. Let's map this to the pattern structure.

Structure of the Command Pattern

We'll start with the object that invokes the command. This is the Invoker, and it's responsible for asking the command to carry out a request. It maps to the button from our example. ICommand, which is our command interface, maps to Command. That declares an interface for executing an operation. The implementation of that interface then is the ConcreteCommand. That maps to our AddEmployeeToManagerList command. It defines a binding between a receiver and an action. It implements execute, and it does that by invoking the corresponding operations on the receiver. And that brings us to that receiver, that is the object that knows how to perform the operations associated with carrying out a request. In our case, that's our repository, but any class may serve as a receiver. And lastly, we've got the client, which creates the ConcreteCommand, and sets its receiver. And that's the pattern structure. Let's implement this in a demo.

Demo - Implementing the Command Pattern

In this demo, we will implement the use case we covered when describing the command pattern. First thing we'll need for that is a bit of plumbing code. We need classes for our employee, for our manager, and we also need a repository. Let's start with the Employee. This is a pretty straightforward class. It mimics an employee. Every employee has an ID and a name, and we use a constructor to convey, meaning that we want this employee to be initialized with an ID and a name. And now we have a manager. A manager is also an employee, so we let it derive from the Employee class. But next to being an employee, a manager also manages a bunch of employees. We mimic that by adding an Employee list. So these two classes will work. Then we need something to manage these classes, or rather, for our example, we need a way to add employees to a manager's list, remove employees from it, and check whether or not an employee is already in the manager's list. We can do that via a repository. First, we add an interface that defines the contract of our repository. This acts as the interface for the receiver, which is one of the components from the structure of the pattern. As you can

see, it has the three methods we already talked about and one additional method, `WriteDataStore`. That's simply a helper method we're going to use to write out some info to the console output window. Like that, we can see what's going on. Now we need to implement this repository. I'm going to paste that in so we can run through it. So, as far as the data store is concerned, we're going to use an in-memory list. In a real-life scenario, you would probably want to persist this to a database somewhere. But for the pattern implementation, that is not important. This is not where the focus lies. But we do need something to mimic this, which is why we hold a list of managers as data store. And then we have methods to add employees to these managers from the data store. So we have one method that accepts a `managerId` and the employee to add to add it, another one to remove it, and then one more to check whether a manager already is managing an employee. We can use this to check whether or not adding the employee can go through. Lastly, as mentioned, the `WriteDataStore` method is just a helper method, which will write out the data store to the console output window. So with that, we've got our plumbing code, and we also already have the receiver. This receiver is the action that will eventually get executed when the command gets executed. The first thing we need for that is an `ICommand` interface. That is the command from the pattern structure. In the pattern structure, it contains just one method, `Execute`. But as you know by now, these patterns should be seen as templates. In real-life implementations, additional methods or variations often exist, and this is such a case. We will immediately extend the pattern with an additional action, `CanExecute`. You can use this to check business rules that might block the command from executing. Like that, executing the command and knowing whether it can be executed are both contained within the command itself. The next class is the concrete implementation of a command. We name that `AddEmployeeToManagerList`. We wanted to implement the `ICommand` interface. So if we do that, we see that we have to implement `CanExecute` and `Execute`. Our command needs a few things to be able to correctly function. It needs access to our repository, but it also needs to know the `managerId` and the employee. That's the state of the command. Seeing we need all of these for the command to correctly function, we can convey that by adding a constructor that accepts these. Like this, we state that the command shouldn't be constructed without them, and we store these in private readonly fields. With that, our command now has access to everything it needs to correctly function. Let's implement the `CanExecute` method first. As an example, we will ensure that a command can only be executed if the employee isn't already an employee on the manager's employee list. We also add a

null check to ensure that the employee we want to add isn't null. Additional rules can exist as well like ensuring that an employee can only be managed by one manager at a time. But for demo purposes, this is sufficient. On to the actual execution of the command. That's the Execute method. In this method, we add the employee to the manager's employee list. But if we try to do that, we see that we get a warning. This warning states that `_employee` may be null, and that is true. We do add a null check in the `CanExecute` method, but that doesn't mean that the `Execute` method can't be called without first calling the `CanExecute` method. So it does make sense to add an additional null check here, and there we go. With that, we've got our command. Then, we need something to invoke this request. That's the Invoker. In our example, the command is invoked by a user clicking a button in a user interface. So, the Invoker could be that button, either via binding or by setting the command in some other way. And that works, but we haven't got a user interface. We're building a console app, and that's okay because what I prefer to do is add a small `Manager` class in between, `CommandManager`, which is responsible for effectively invoking commands, not just one command, but multiple commands throughout your application. In our demo, we can use that `CommandManager` as an Invoker class, which makes our life a little bit easier. At this moment, the `CommandManager` class is pretty straightforward. In real life, the user interface would now call into the `CommandManager` to execute the commands. You'll notice that working like this makes it easy to add in additional functionality like undoing commands. More on that in an upcoming demo. In this iteration, the `CommandManager` simply acts as a passthrough class that doesn't really do anything, but we will build upon this. For now, this is sufficient. Then on to the last part, the client, that's our `Program` class. Let's new up our `CommandManager` and our repository. For that, we need to import the `Command` namespace, so let's do that, and now let's create a command and invoke it. For that, we use the `CommandManager`. We call `Invoke` on it, and we pass through a new command. To new up the new command, we need to pass through the repository an ID for the manager and an employee. Once the command has been invoked, we write out the data store so we can see what just happened. And let's try the same thing two more times. Let's give this a try, and let's put the output next to our code. So we have two managers, Katie and Geoff. At the beginning, they don't have any employees. We add a new employee, Kevin, and we write out the data store. That results in Katie having Kevin as an employee. Once we invoke the second command, Clara is added as an employee for Katie. And as we see, this works as expected. Lastly, we add Tom as an employee for manager, Geoff, and that works out as

expected as well. So far, so good. Now, what would happen if we try adding the same employee again? Employees are identified by their ID, so newing up another command and invoking it is what we're going to try. In principle, seeing in the Invoke command, we check whether the command can be executed. And in the CanExecute method, we check whether an employee is already on the employees list for a manager. This should result in the employee not being added. The command, in other words, will not be executed. And let's see if that checks out. And that is indeed the case. This is our data store right after we add Tom for the first time, and this is our data store right after we tried to add Tom for the second time, which obviously resulted in the command not being executed. And with that, we've got our pattern implementation. This is already quite useful, but we can extend this example so it becomes more useful. We're going to do that by adding undo functionality in the next demo.

Demo - Supporting Undo with a Command Manager

Time to extend our example with undo functionality, and this is where our command manager will shine. To enable undo functionality, we need to add two things. Let's scroll up a bit. Here's our command, and let's think about what undoing a command would mean. Seeing executing a command, in our case, means that we add an employee for a manager, undoing this command will mean that we want to remove the employee from the manager. So our Undo action is quite straightforward. We call in to RemoveEmployee on the repository after a null check. On the CommandManager, we can then keep a list of commands. Seeing undoing means executing the undo actions on each command in reverse order, a stack is a good structure to store these commands in due to the fact that it's a first-in-last-out structure. So, when a command has been executed, we push the command onto the stack, and then we can add one or more undo actions on our CommandManager. Say that we have an Undo action that will reverse the last command. The first thing we do is check whether there's still something on the stack. And if there is, we call Pop. That results in the last item on the stack being returned. If it's not null, we call Undo on it. And I see that Undo doesn't resolve. That's because we're working on ICommand, so the interface. And I think I just added the Undo action on Command, but not on the ICommand interface. So obviously, I need to add it here as well. There we go. That should fix it, and indeed, now it resolves. What I also like to do is add an UndoAll action. The idea is that this one will reverse all commands. It's a bit easier for the client to interact with the command manager like

that. And that's already it. Let's adjust our client a little bit so we can test this. So let's say that after we add Kevin as an employee, we undo that and we write out the data store again. And in the end, we undo all the actions we previously executed. So we should end up with a data store with two managers without employees. Let's have a look. So, the first time our list is written out, Kevin has been added. Right after that, we undo the last command, which means that Kevin is removed again. After that, our commands all get executed, as we know from the previous demo. And the last statement we do is an UndoAll statement. This means that all commands have been reversed, and that results in the same situation we started from. All managers have 0 employees. In a likewise manner, you could now implement redo functionality. Let's have a look at this and additional pattern use cases.

Use Cases for the Command Pattern

Let's start with pattern use cases at a generic, rather abstract level. A very obvious one is when you want to parameterize objects with an action to perform. That is exactly what we did in our example. We turned an action, a button click, into a standalone object, which we can pass around. But there's more potential use cases. Supporting undo is one of them, as we've covered in our second demo. Also, consider this pattern when you want to specify, queue, and execute requests at different times. The lifetime of the command object can be independent of that of the original request. You can transfer the command object to another process and fulfill it there. And lastly, you can also easily augment this pattern to store a list of executed changes. So they can be reapplied in case of, for example, a system crash. To implement something like that, store or log the command object and reexecute it in case an issue happens. Now, let's look at a few more real-life scenarios, which you may consider in your day-to-day development life. The first one I'd like to touch upon are financial transactions. Imagine you're building financial applications. In those types of applications, you can use commands that can be used to represent financial transactions. If a transaction fails at some point, you can roll it back if you've implemented support for that via the command. Workflow management is not a typical use case. In applications dealing with workflows or business processes, you can represent each step in the workflow as a command. This enables dynamic configuration and execution of complex processes without hard coding the specific steps. Macro recording and playback is another use case. Applications that support this such as spreadsheet software often use the command pattern.

Each user action inside of the macro is recorded as a command, allowing users to replay a sequence of actions. And lastly, why not game engines? Game development often involves implementing complex interactions and behaviors. The command pattern can be used to create a system where game actions like move character, shoot, jump, etc. are encapsulated as command objects. This allows for easy addition of new actions or even recording and replaying game sequences. Let's have a look at the pattern consequences.

Pattern Consequences

The main consequence for this pattern is that it decouples the class that invokes the operation from the one that knows how to perform it. This is the single responsibility from the SOLID coding principles at work. Interesting to know is that commands can be manipulated and extended like any other object. Commands can also be assembled into a composite command. For example, a button which restarts creating a list of employees from the context of an employee might first execute a `ClearEmployeeList` command, followed by an `AddEmployeeToList` command. In general, these types of commands are an instance of the composite pattern, which we've covered in another module. Lastly, as far as the positive consequences go, it's easy to add new commands because you don't have to change existing classes. You simply add a new class that implements the required `ICommand` interface. This is the open/closed principle at work. A potential negative consequence is complexity. By transferring requests into objects, we're adding an additional layer. Let's check out the related patterns.

Related Patterns

The first related pattern, as you've probably guessed, is a structural one, the composite pattern. This can be used to implement commands composed of other commands. Memento, a behavioral pattern, can be used to store the state a command requires to undo its effect. That's coming up in the module after this one. And this pattern is also related to a creational pattern, the prototype pattern. That's because a command that must be copied before being placed on the history list, in case you want to support undo, acts as a prototype. There's also a relation between the chain of responsibility and the command pattern. Handlers in the chain of responsibility can be implemented as commands. Talking about that chain of responsibility pattern, that is a pattern which connects senders and receivers of

requests. We will look at some other patterns like that in this course. In that regard, chain of responsibility, command, mediator, and observer are all related. The chain of responsibility passes a request along a chain of receivers. That can be defined at runtime until at least one handles it. The command connects senders with receivers unidirectionally, for example, from a button that's pushed in a UI to the action that finally gets executed. The mediator pattern eliminates direct connections altogether. Communication runs through the mediator. And lastly, the observer allows receivers of requests to subscribe and unsubscribe to them at runtime. With that, let's check out the module summary.

Summary

We learned about a command pattern in this module. It's a very common pattern, especially in rich UI design projects that's intended to encapsulate a request as an object, thereby letting you parameterize clients with different requests, queue or log requests, and support undoable operations. As far as implementation goes, methods on the command are defined on an abstract base class or interface, the `Command`, which is implemented by `ConcreteCommands`. Remember that the receiver can be any object, and the `Invoker` is often a UI element, but it pays to add a small `CommandManager` class in between, which can then act as `Invoker`. Like that, you can build in undo, redo or likewise functionality in that command manager. For once, let's not continue with the next most often used pattern, but instead cover a closely related pattern, the memento pattern.

Checkpoint 17 - Implementing the Command Pattern

Behavioral Pattern: Memento

Coming Up

In this module, we'll cover the memento pattern, sometimes also called the token pattern. It's all about capturing and externalizing an object's internal state. I'm Kevin, and I'll guide you through it. We'll start by describing the pattern using a real-life example. As a real-life example, for once we'll build upon

something from a previous module. We will use the memento pattern to store a command's state to support undo actions. We'll map that to the structural design pattern, and we'll also implement a real-life example in a demo. After that, we will continue with learning about the use cases for this pattern, pattern consequences, and related patterns. Let's dive in.

Describing the Memento Pattern

The memento pattern is a behavioral pattern from the Gang of Four library of design patterns. The intent of the memento is to capture and externalize an object's internal state so that the object can be restored to this state later without violating encapsulation. In other words, this pattern will allow us to store and restore private field and property values. Let's clarify that with a real-life example. We're going to use undo functionality of commands. If you've watched the previous module, you know what this is about. And normally, I try to keep the modules as separate as possible. But as it's such a nice use case to explain the advantages and especially disadvantages of using memento, I'm going to build upon what we ended up with in the previous module. With that in mind, if you want to learn about commands, simply look at a command as a class that has some internal state, so private state, in this case. That needs to be stored to be able to undo the command. It's easy enough to store public state, but private state is a bit different. An example we previously used was a command that added an employee to a manager's list of employees. That command needed to know about the employee to add, about the manager and about the repository through which interaction with the underlying data store could happen. We use the `CommandManager` that was responsible for executing the command and storing the necessary information about the command to undo this. We did that by storing the command itself in memory. And while that works, this could mean we'd be storing too much information. We don't necessarily need to store the full command object. We simply need to store its state or part of its state. Let's switch back to that command implementation. If that's what we want, we could make the state public and access it as such to store it somewhere, yet this would break the command's encapsulation. And this is exactly what a memento solves. It allows saving and restoring internal private state without breaking encapsulation. If we look at our command again, we see that the internal state that we want to store consists of the `Employee` object and the manager ID. We don't need to store a reference to the repository. That remains the same for all `AddEmployeeToManagerList` commands. The first thing we'll need is a `Memento` class that can contain this state. A good name

would be `AddEmployeeToManagerListMemento`, containing the state we want to store via properties. On the command itself, `AddEmployeeToManagerList`, we then add two methods, one to create a memento and return it and one to restore a memento. That would be a method that accepts the memento and that's responsible for restoring the command's state. And now we need something that manages this. That is our `CommandManager`. In the implementation we ended up with in the previous module, we stored commands on a stack. We no longer have to do that. This time we store mementos on a stack. Undoing a command then means that we take one existing `AddEmployeeToManagerList` command, restore the memento on it, and call the Undo action. And that takes care of our use case. Important here is that we want two different interfaces to this memento, one wide interface that's used by the command to create the memento. This passes through state and has access to set the state. The other interface to the memento is used by the command manager. That one stores these mementos and passes them through to the command, but it cannot change the memento itself anymore after it has been created. It has access to the getters, but not to the setters. That is a narrow interface. We can achieve that with private setters, as we will see in the implementation. Let's map this to the pattern structure.

Structure of the Memento Pattern

We've got three players in this pattern. The first one is the memento itself, which maps to our `AddEmployeeToManagerListMemento` class. It's responsible for storing the internal state of the originator object. It may store as much or as little of the original state as is necessary. Important here is that we try to protect this state against access by other objects as much as possible. That's that wide/narrow interface explanation we just covered in the previous clip. Then we have the originator. That maps to our `AddEmployeeToManagerList` command. It's responsible for creating a memento with a snapshot of its current internal state, for example, via a `create memento` method. It uses the memento to restore its internal state, for example, via a `restore memento` method. And lastly, we have the Caretaker. This object is responsible for the memento's safekeeping. It should not operate on or examine the memento's content. It simply passes it through to the originator. That's what our command manager does. Such a caretaker can work on one or more mementos. In our use case, the command manager is a caretaker that works on multiple mementos. Let's check that out with a demo.

Demo - Implementing the Memento Pattern

We'll start this demo with copying over the implementation and program classes from the previous module. For once, we will work on what we ended with there. (Working) There we go. Now let's add a Memento class to hold the internal state of our AddEmployeeToManagerList command. We name it AddEmployeeToManagerListMemento. Seeing we want to hold the managerId and employee, we give it two properties, a ManagerId and Employee. We also give these properties private setters. Like that, the caretaker will not be able to modify the memento. Seeing our properties have private setters, that means we can only set them via the constructor. So we do have to add a constructor, which accepts the managerId and the employee. But that makes sense because a memento without these fields doesn't make sense. So it's a good idea to convey that meaning via the constructor. And now we need to extend our command a little bit. The first thing we need is a method that will create a memento. So we name that CreateMemento in which we create an instance of our AddEmployeeToManagerListMemento. And now we need a method to restore the state of the command. We name that RestoreMemento. This is used to restore our command's state, and it accepts the memento. This does mean that we can no longer keep the ManagerId and Employee fields at the level of our command readonly, as we now have to be able to change them outside of the constructor. There we go. That takes care of that. And that's it for our Command class, in other words, the originator. What is left is the caretaker. That is our command manager. As you notice, once you start combining different patterns, which is often the case, the same class will often be used by the different patterns, thus mapping to different component names from the pattern structure. The first thing we need to do here is change the stack. We no longer want to store commands on it. We want to store mementos on it. So, it's not a stack of ICommand anymore. It's a stack of AddEmployeeToManagerListMemento. Let's also change the name and new it up. We also want to add one command field. We need that to hold our AddEmployeeToManagerList command. This command will be reused. Also, we don't instantiate it yet. We didn't store a reference to the repository in our memento, as that remains the same over all command instances, in case you need more. Therefore, we need to set this field to the first instance of the command that's created by the client, so we can get a hold of the correct repository reference. That's what we do in the Invoke method. Also, note that we do have to cast the incoming ICommand to an AddEmployeeToManagerList command to be able to store it. We are now, again, tying this to concrete implementation. In a real-life scenario, this

might become a bit more difficult to manage, but we'll get to this and other disadvantages right after we finish our implementation, so let's first continue with that. So, next to storing our command, we also push a memento on the stack by calling `_mementos.Push`. Now, as you notice, calling `CreateMemento` on command doesn't resolve. That's because `CreateMemento` is implemented on the concrete command class, while command is of type `ICommand`, the underlying contract or interface. That means that we need to cast here as well. And there we go. Now the code does resolve. But as you've noticed, we are now tying the `Invoke` method to one specific type. That's a bit of a disadvantage, and there's more, but we'll look into that right after we finish writing the demo. That's it for invoking the command. When we want to undo the last one, we first check whether mementos exist in the list. And if one does, we restore that memento on the command by calling into `RestoreMemento`, passing through the memento we pop from the stack. Now, the state of the command is that of the memento. And when we call `Undo`, we effectively undo the correct state change. For undoing all commands, we do the same, but in a loop that loops through all mementos. Let's try this out. We do not need to change anything to the client code. As far as that is concerned, all of this is of no importance. So this code, which we copied from the command demo, remains the same. Let's give that a try, and there we go. This still works as expected. This is the exact same outcome we had at the end of the command demo. So we're now using mementos, but you can already see a negative consequence here. The command that's reused is always the same command. If you want this to work with different commands, which you typically want the command manager to do, you will have to implement support for that. Moreover, you'll also need `Memento` classes for each command, and it would also be nice to find a solution for all this casting. So, this has a tendency to become a hassle pretty fast. We will look into the consequences of this pattern right after the use cases.

Use Cases for the Memento Pattern

Let's start with a few pattern use cases at a generic abstract level or rather use case because there's just one main use case for this pattern consisting of two parts. Use it when part of an object state must be saved so it can be restored to that state later and when a direct interface to obtaining the state would expose implementation details and break the object's encapsulation. So if you find a requirement for this in your application, which matches these two things, then the memento pattern is

something to look into. And there's actually quite a few real-life use cases or scenarios where this effectively happens. The first good example would be a text editor with undo and redo functionality. If you're writing something like this, you will often want to use the memento pattern to implement undo and redo functionality. Each time a user makes a change to the document, a snapshot of the document's state is taken and stored in a history stack. The user can then undo or redo actions by restoring the document to a previous state using these snapshots. Another use case you're probably familiar with, version control systems. Version control systems like Git can use the memento pattern to store the history of changes made to a codebase. Commits in Git are essentially snapshots of the codebase's state at a specific point in time, allowing developers to track changes, revert to previous versions, and collaborate effectively. A third example, e-commerce shopping carts. If you're writing an e-commerce application, you could use the memento pattern to implement the save for later or wish-list functionality. Those types of functionalities allow users to save items from their shopping carts to a list and later restore them to the cart, preserving the state of the items and their quantities. And lastly, content management systems, of course. If you're writing such a system, you can use the memento pattern to save and restore the state of web pages or documents. Like that, you allow users to review and revert to previous versions of content. It's a pretty common piece of functionality in a CMS, or content management system. Let's have a look at the pattern consequences.

Pattern Consequences

A major consequence of this pattern is that it preserves encapsulation boundaries. Information that only an originator should manage but must be stored outside the originator is no longer exposed. It also simplifies the originator. It no longer needs to concern itself with storage management. On the other hand, using mementos might be expensive if the originator must copy over large amounts of information to store in the memento or if clients create and return mementos to the originator often. So, make sure that encapsulating and restoring originator state is cheap, otherwise the pattern might not be appropriate. And last, but not least, it also tends to complicate things. We notice that with the command manager. We're now just using one command on which we change state for undo functionality. But code tends to become cluttered and/or complex when you need to work with many different types of state that you need to store and restore. Let's check out the related patterns next.

Related Patterns

The first related pattern is the command pattern, as it can use a memento to store and restore its state. That's exactly what we did in the demo. Next to that, mementos can also be used for iteration. In that case, a memento can be used to capture the current iteration state and potentially roll it back.

Summary

We learned about the memento pattern in this module. It's not that common anymore, but it's still used when you need to support undo or redo functionality. Its intent is to capture and externalize an object's internal state so that the object can be restored to this state later without violating encapsulation. As far as implementation goes, the important thing to remember is that we are working on a narrow and wide interface. We can do that by only allowing the memento properties to be set privately. Like that, the originator can do that, as it uses the constructor for creating the memento, but the caretaker cannot. Onto the next pattern, the mediator pattern.

Checkpoint 19 - Implementing the Memento Pattern

Behavioral Pattern: Mediator

Coming Up

In this module, we will cover the mediator pattern. Its intent is to define an object, the mediator, that encapsulates how a set of objects interact. I'm Kevin, and I'll guide you through it. We'll start by describing the pattern using a real-life example. The archetypical example for a mediator is a chatroom, a common use case, which everyone can easily understand. And chances are you'll have to implement something along these lines during your career. We will map that to a structural design pattern, and we will implement it in a demo. We'll continue with learning about use cases for the pattern, pattern consequences, and related patterns. Let's dive in.

Describing the Mediator Pattern

The mediator pattern is a behavioral pattern from the Gang of Four library of design patterns. The intent of the mediator pattern is to define an object, the mediator, that encapsulates how a set of objects interact. It does that by forcing objects to communicate via that mediator. By doing so, it promotes loose coupling by keeping objects from referring to each other explicitly. It also lets you vary their interaction independently. Let's elaborate on that a bit. On the left, you see a set of objects that need to interact with each other. To be able to do that, these objects need to hold references to each other. If we're talking about just a few objects, that is still okay. But imagine that there are not just a few objects, but tens, hundreds or thousands of objects holding on to all these references and managing them, and keeping communication in sync becomes a pretty big issue pretty fast. Enter the mediator pattern. What this pattern does is provide a central object, which is the mediator object. That maintains references of objects that want to communicate with each other, and it handles communication between them. As you can see, all of a sudden this has become more manageable. Let's map this to a real-life example. Imagine that we're writing software for a team of people working at a bank. Amongst them, account managers that sell banking products and lawyers that need to check whether these products are compliant. A part of the software that we need to write is a chat application, which needs to be custom built due to high security and confidentiality requirements. We are the developers building that app. We could now create team member objects that hold to references to all other team member objects that need to communicate with each other, but that is exactly what we don't want to do. Instead, we will use the mediator pattern. We'll start with one central object, the chatroom. This is abstract. It defines the interface for communicating with the members. Remember, as usual with these patterns and as detailed in the introduction modules, these patterns want to encourage working on interfaces and not on implementations. Technically, that interface is either an abstract class or an interface, as we defined it in C#. Let's go for the last option this time, an interface. We can name it `IChatRoom`. A concrete implementation of this would then be a team chatroom named `TeamChatRoom`. And now we have a team member, again, an abstract class, which holds a reference to the abstract chatroom interface. It needs this to be able to communicate with it. So, instead of communicating directly with other team members, the team member communicates with the chatroom. Our lawyers and account managers are then concrete implementations of the team member abstract base class. These are the objects that effectively receive and send messages

through the team chatroom. This team chatroom thus holds the actual references to the team members. Let's map this to the mediator pattern structure.

Structure of the Mediator Pattern

The abstract chatroom class, that is our mediator. This defines an interface for communicating with Colleague objects. Such a colleague maps to our TeamMember abstract class, whereas AccountManager and Lawyer are ConcreteColleague classes. They know they're mediator and communicate with it when they would have otherwise communicated with another colleague. Lastly, we have the ConcreteMediator, our team chatroom. This derives from or is an implementation of mediator. It knows and maintains its colleagues, and it implements cooperative behavior by coordinating Colleague objects. In other words, it routes messages between them. And with that, it's demo time. We'll start with implementing our bank team chat example. After that, we will extend it by supporting communication between specific objects only.

Demo - Implementing the Mediator Pattern

We'll start this demo with the mediator object itself, the ChatRoom, an abstract base class of which will require a concrete implementation later on. The functionality required is sending a message, so we add an abstract method for that. The parameters we want to accept here really depend on your use case. What's accepted here is what the mediator has access to and can thus communicate to the team members or use for additional logic. A from parameter and a message parameter seem like a good idea for our demo. We currently don't support sending something to a specific member or set of members. The other type of functionality we need is registering users for our chatroom, so we need a Register method. As a parameter, it accepts a TeamMember that needs to be registered. We don't have that class yet, so let's add it. We won't add implementation yet. This is just to make sure that this resolves. As we learned on the slides, the TeamMember class too is an abstract class. We've used an abstract base class here, but as we didn't provide default implementations, you could technically implement this as an interface as well. So, why not do that? So we'll switch this around a bit. We create an interface for our mediator, and we get rid of the abstract class. When you don't have default implementations, there really is no need to work with an abstract class. Let's continue with our

abstract TeamMember class. That's the colleague from the pattern structure. It needs to hold a reference to our mediator, as it's true this mediator that it communicates with other team members. In principle, this mediator doesn't need to be accessible from outside of this class. We can have this class send the message through the mediator and have subclasses simply use the base method we define. We also add a Name property to hold the member's name. To convey that we don't want this to be null, we add a constructor, which accepts both the name and the IChatRoom implementing instance, but there's a potential issue with this. It's the IChatRoom implementation that sets the mediator for each team member when team members are registered on it. So, at that time, the team members are already constructed. So we need to change this a bit. We make IChatRoom nullable, and we add an internal method to set the ChatRoom instance to avoid this being possible from outside of our assembly. We now no longer accept a chatroom via our constructor. There we go. And then we needed to implement the method to send messages. So we add a Send method, and in it, we accept a message and use the mediator to send it. The from parameter is the name of the team member. Team members don't only send messages, they also need to be able to receive them. So, that's the last method we want to add here. A concrete mediator will call this method on each team member it manages to communicate messages to them. For our demo, we will write out the message we receive to the console output window. What we also want to do is mark this method as virtual. By doing so, we allow concrete implementations to override it and provide their own specific implementation, if needed. So we're working on an abstract class here, and this time it does contain implementation details. We cannot switch this out for an interface. Then, let's add a concrete implementation of this TeamMember abstract base class, the Lawyer. We call into the base constructor, and we override the Receive method. We mark it as virtual, so we are allowed to do that. Like that, we can write out which concrete class received the message. Next to that, we also call into base.Receive to write out the message itself. And then we do something likewise for the AccountManager class. And with that, we've got our two team members. Now we can continue with the concrete mediator implementation. That's our TeamChatRoom, which implements the IChatRoom interface. So we choose to implement it. And that means we need to provide implementations for two methods, Register and Send. Team members are registered via the Register method. Seeing our TeamChatRoom needs to know which members to send messages to, it needs to store them. There's multiple ways of doing that, collections, lists, whichever construction you want to use. As I happen to know that I'm going to want to search for

people by name in the next demo, I'm going to use a dictionary. In the Register method, we then link the mediator to the team member by calling into SetChatroom. When we call into it, we pass through the mediator itself, this. SetChatroom expects an IChatRoom implementing object, which in this case is our TeamChatRoom instance. After that, we add it to the dictionary. This, by the way, does assume that name must be unique as we're using it as key. If that's not something you desire, use another key or another collection construct. And then there's the Send method. Sending a message through the mediator assumes that all members must receive it in this scenario. So, we run through all of them and call Receive on them. And that's it as far as the pattern implementation is concerned. Let's write some code so we can try this out. First things first, we new up our TeamChatRoom. For that, we need to import the Mediator namespace. Then let's new up a few team members so they can chat with each other. We'll add two lawyers, Sven and Kenneth, and three account managers, Ann, John, and Kylie. These are the five team members that will chat with each other, so we need to register them on our chatroom. For that, we call in to Register on our TeamChatRoom instance for each of them. And now let's see what happens when we send the message. Let's say that Ann is going to send a message because she needs a compliance check on a file. Sven sees the message and says he's on it. Let's add a ReadKey statement, and let's give this a try. And there we go. We see that our five team members all got Anne's message. And after that, we see that the message that Sven sent was also delivered to these five team members. Our mediator works as expected, but maybe not everyone should receive every message all the time. Let's see what we can do to improve our example in the next demo.

Demo - Supporting Communication Between Specific Objects

A common requirement when creating a system that requires communication between lots of different objects is limiting communication to specific objects only. Mapped to our system, we could say we want to support functionality in which a team member can choose to only communicate with a specific person by name or another member from the same type, for example, lawyers that only want to chat with lawyers and don't want their messages to be delivered to account managers and vice versa. That's what we'll implement in this demo. We'll start with the first scenario, sending messages to a specific person. In the previous demo, we decided that we would use the name as key for the dictionary, making it unique. We can use that same name to select the person to send the message to.

So the first thing we need is a method that accepts this parameter on the mediator. And then we need to implement this on the concrete mediator. That is our TeamChatRoom. Let's implement the interface, and here's the method we need to implement. We want to send a message to a specific team member and seeing that teamMembers is a dictionary which stores team members with name as key, we use the dictionary indexer to get the correct team member. If it's not null, we call Receive on it. With that, we have ensured that only the correct team member will receive a message. What's left now is ensuring that a team member can send such a message. So let's scroll up a bit to the TeamMember abstract base class and add an additional method on it, a Send method that accepts a to string. The only thing that this method does is pass it through to the sentiment that we just implemented on our chatroom. And that's it already. Let's test this. Let's say we want to have Sven send a message to Ann to jump on a team's call to talk about the file he's checking. Let's give that a try, and there we go. That last message we sent was only delivered to Ann. So far, so good. Another useful scenario would be sending a message to all team members of a specific type. Let's do that next. The implementation is not that different from what we just did save for the fact that we can use some generics for that, which is why I'd like to show it. The first thing we need is another method in our chatroom. We want to pass through a type, Lawyer or AccountManager. We can use generics for that. The type must derive from the TeamMember class, so we pass that as a restriction. That's it for the adjustment to our interface. Onto the implementation. Let's implement the interface. We see that we need to implement one additional method. In this method, we want to run through all team members of a specific type, and the type they are of, that's T, the type that's passed through. So that's exactly what we do. We get all values of type T from our dictionary, and we let them receive the message. All that's left is one more method on our abstract TeamMember base class. Just like we added a method to send a message to a specific person, we now add a method to send a message to a specific type, which calls into the Send method on the underlying chatroom. And that's it. Let's give that a try. Once Sven lets Ann know that the file is good to go, Ann wants to communicate this to all other account managers. She can use the SendTo method, passing through AccountManager as type for that. Let's give that a try, and there we go. After Sven sent the message to Ann, Ann sends the message to all other account managers. As we can see, that means that the message was received three times. That's it for our demo. Let's continue by checking out a popular implementation of this pattern, MediatR.

Alternative: MediatR

What we're looking at right now is MediatR, a very popular .NET implementation of the mediator pattern made by Jimmy Bogard, the guy who also made AutoMapper. This project is on NuGet and supports request response, commands, queries, notifications, and events, synchronous and asynchronous with intelligent dispatching via C# generic variance. The way Jimmy implemented it does differ a little bit from our pattern implementation. Ours is closer to the original design pattern, but that's okay. As I mentioned a few times, pattern implementations can and will differ. As long as the principles and the intent remain the same, that's okay, and that is the case with MediatR. With that, let's have a look at the main use cases for this pattern.

Use Cases for the Mediator Pattern

Let's start with pattern use cases at a generic abstract level. There's three good use cases when applying this pattern is something to consider. First, when a set of objects communicate in well-defined but complex ways, which results in interdependencies that are unstructured and difficult to understand. A second case is when because an object refers to and communicates with many other objects, it is difficult to reuse, in other words, when you've got tight coupling between a set of likewise classes. Both of these apply to our real-life chatroom example. And lastly, when behavior that's distributed between several classes should be customizable without a lot of subclassing. In that case, the mediator pattern can avoid a lot of that subclassing. What this means is that the mediator localizes behavior that would otherwise have to be distributed amongst different classes. It does that because all relations between components are contained within the mediator. From that follows that when using the mediator, the colleague classes can be reused, as is when the behavior has to be customized. It's contained within the mediator without requiring changes to or subclassing of the colleague classes. Now, let's look at a few real-life use cases or scenarios you might encounter in your day-to-day life where you might consider this pattern. A first good use case would be when you're working on a hospital information system. In such a system, different departments and systems like the billing department, patient records, the inventory system, etc, those different things need to share data and coordinate activities. What you can do is use the mediator pattern to centralize communication between these components, ensuring that updates in one department are properly reflected in others while at the same time reducing interdependencies. A second good use case or scenario is smart

home automation software. In a smart home system with various IoT devices like lights, thermostats, security cameras and so on, you could use the mediator pattern to coordinate interactions between these devices. For example, a smart home hub or rather the software on it can act as the mediator, receiving commands from the user directly or from sensors and orchestrating the actions of different devices such as turning off the lights when a security alarm is triggered. A third good use case is a stock trading system. If you're working on such an application, you'll have to deal with multiple traders and market data sources that need to interact. You could use the mediator pattern to centralize the trading platform's communication. Traders can place orders and receive updates through the mediator, which then forwards the orders to the appropriate market and release market data back to the traders. And lastly, traffic management systems. In a smart traffic management system for a city, you can use the mediator pattern to help control traffic signals, cameras, and sensors. A central traffic management system can act as the mediator, collecting data from various sensors, making decisions on traffic signal changes, and communicating those changes to traffic signals at intersections to optimize traffic flow.

Pattern Consequences

There's five important potential consequences to this pattern. First of all, as we just detailed in the previous clip, it limits subclassing. A second consequence is that it decouples colleagues. Thanks to the mediator, they are now loosely coupled instead of requiring the colleagues to know about each other. It also simplifies object protocols. What I mean by that is that interactions between objects that would normally be many-to-many interactions are replaced with one-to-many interactions between the mediator and the colleagues. Those types of interactions are simpler to maintain, extend, and understand. The open/closed principle also applies. New mediators can be introduced without having to change the components. And lastly, a disadvantage. Mediator centralizes control, which, as a consequence, can make the mediator complex. That is a disadvantage because it could turn your mediator in a monolith that's hard to maintain. Before we continue with the summary, let's check out the related patterns.

Related Patterns

The first related pattern is a structural one, the facade pattern. Both mediator and facade organize collaboration between a lot of normally tightly coupled classes, but there are differences. Mediator, like facade, abstracts functionality of existing classes, but it's not the same. The purpose of the mediator is abstracting communication between objects, often centralizing functionality that doesn't belong in them. Facade, on the other hand, simply abstracts the interface to the subsystem objects to promote ease of use. When we look at some of the other patterns in this course, we find others that connect senders and receivers of requests. In that regard, chain of responsibility, command, mediator, and observer are all related. Chain of responsibility passes a request along a chain of receivers that can be defined at runtime until at least one handles it. Command connects senders with receivers unidirectionally, for example, from a button that's pushed in a UI to an action that finally gets executed. The mediator pattern eliminates direct connections altogether. Communication runs through the mediator. And lastly, the observer allows receivers of requests to subscribe and unsubscribe to them at runtime.

Summary

We looked into the mediator pattern in this module. The intent of the pattern is to define an object that encapsulates how a set of objects interact. It promotes loose coupling by keeping objects from referring to each other explicitly, and it lets you vary their interaction independently. When implementing, think about encapsulation. Use an internal method to ensure that the mediator can't be set outside of an assembly. Also, don't allow overriding methods when it's not needed like the Send method. as that would add unnecessary complexity. Up next, the chain of responsibility pattern.

Checkpoint 20 - Implementing the Mediator Pattern

Behavioral Pattern: Chain of Responsibility

Coming Up

In this module, we'll cover the chain of responsibility pattern. Its intent is to avoid coupling the center of a request to its receiver by giving more than one object a chance to handle the request. I'm Kevin, and I'll guide you through it. We'll start by describing the pattern. As a real-life example, we'll use a document validation and approval chain. After that, we will map that to a structural example, and we will implement the real-life example in a demo. We'll continue with learning about use cases for this pattern and pattern consequences, which can be positive or negative. We will end the module by looking into related patterns. Let's dive in.

Describing the Chain of Responsibility Pattern

The chain of responsibility pattern is a behavioral pattern from the Gang of Four library of design patterns. Its intent is to avoid coupling the sender of a request to its receiver by giving more than one object a chance to handle the request. It does that by chaining the receiving objects and passing the request along the chain until an object handles it. Let's clarify that with an example. We're building part of an application, which validates a document. The first thing that needs to happen is ensure that the document's title is filled out. Up next, we've got a requirement that the document's last modified date must be less than 30 days old. If those check out, the document must be read and approved by the litigation department after which it can be sent to the manager and approved there. Only after the manager has approved the document, it's considered valid and can be sent to the customer. As you gather from our code example, that's a lot of conditional statements that need to evaluate to true before the document is considered approved and ready to send to a customer. The method that checks the document becomes bloated and hard to manage, and there is no easy way to reuse these checks in other parts of our code. Enter the chain of responsibility pattern. The idea is that we're going to put all these validations and approvals in objects of their own, handlers. So, we'd end up with an interface, say, `IHandler`, which states that there should be a method to handle the incoming request. We could scope this to a `Document` class, as that's what we're using in this example, but using generics makes this more reusable, so that's what I'd suggest here. As each handler can pass the request on to the next, we also need to provide a way to set the next handler, the potential successor. For each of our validations, we have an implementation of this interface, a handler. So we've got `DocumentTitleHandler`, `DocumentLastModifiedHandler`, `DocumentApprovedByLitigationHandler`, and `DocumentApprovedByManagementHandler`. I'm only putting the first one on the diagram here. The

other ones follow the same principle. These handlers get the document passed into their handle method and can execute their part of the document validation. If the validation doesn't check out, we don't continue, but if it does, we can continue to the successor. Like that, the document flows through the chain. And of course, we have a client that initiates the request. It changes the handlers and provides it with a document to validate. Before we continue with mapping this to the pattern structure, there's something I'd like to point out. The example I've used is a real-life example that's in line with today's interpretation of the chain of responsibility pattern. However, if you'd look up the original implementation from the Gang of Four book, it's a bit more strict. In that interpretation, a request moves up a chain of handlers, and each handler simply checks whether it can handle the request or not. If it can't, nothing happens to the request except passing it on. And if it can handle it, it's handled and not passed on anymore. Both are, in my opinion, valid implementations. By the way, as a side note, for those of you who've worked with ASP.NET Core, this should feel quite familiar. You're looking at a slide from my Web API Fundamentals course in which I describe how the ASP.NET Core request pipeline works. Essentially, you're chaining pieces of middleware that can handle and/or alter a request and potentially pass it on to the next piece of middleware. That's a prime example of the chain of responsibility pattern at work. Let's map our example to the pattern structure.

Structure of the Chain of Responsibility Pattern

The first player in the pattern structure is the handler, which maps to our `IHandler` interface. This defines an interface for handling requests and optionally implements the successor link. Then we've got our concrete implementations, which are named `ConcreteHandler`. These handle requests. They can access the successor and potentially pass on the request to it. Lastly, we've got the Client. The Client is responsible for initiating the request to a `ConcreteHandler` object on the chain. And with that, it's time for a demo.

Demo - Implementing the Chain of Responsibility Pattern

In this demo, we will implement the chain of responsibility pattern using our document validation example. So, let's start with the `Document` class so we have something to validate. This is a pretty straightforward class. A document consists of a title, a last modified date, and two booleans stating

whether or not it's been approved by litigation and/or management. In a constructor, we accept these four values and set the properties. Then, we need to create an interface for our handler. This is the handler component from the pattern structure. You could potentially use an abstract class for this as well if you want to provide some basic implementation logic. We're okay with an interface, and we name that `IHandler`. We also make it generic so we can use it on other classes as well and not just on the `Document` class. It has a method, `Handle`, which will handle the incoming request. The request that comes in is of type `T`. That's our document. We also need a way to set the successor, the next handler in the chain. For that, we define a `SetSuccessor` method, which accepts that next handler. We conveniently have this method return an `IHandler<T>`. That's the successor. This creates a fluent interface, allowing us to easily chain these handlers. Then let's implement a `ConcreteHandler` for checking the title. We'll name it `DocumentTitleHandler`, and we'll have it implement our `IHandler` interface. The type we pass through is `Document` because that's what this handler works on. Let's implement the interface. In the `handle` methods, we do our validation. Let's rename the parameter that comes in. We check whether the title is empty or not. If it is, validation fails, and we throw a new `ValidationException`. That's defined in `System.ComponentModel.DataAnnotations`. It's built in in the framework, so let's import that namespace, and there we go. If the title isn't empty, we can go to the next handler, but we haven't got a field for that yet. So, let's add a new field, `_successor`, which we will use to store a reference to the next handler in line. Now we can call `Handle` on the successor, passing through the document. Note that `successor` is nullable. The handler doesn't have to have one. We only go to the next handler when `successor` is not null. Then we need to implement the `SetSuccessor` method. This is pretty straightforward. We simply set the `_successor` field, and we return the successor itself, which, as mentioned, will allow us to fluently chain another one. That's it for our handler. We have three more to go. The only thing they're different in is the actual validation rule. The first one is the `DocumentLastModifiedHandler`. We check whether the document was modified in the last 30 days. If it wasn't, we throw a `ValidationException`, otherwise we potentially continue to the next handler in line. The `SetSuccessor` method simply sets the successor value, as was the case with the previous handler. Our next concrete handler is the `DocumentApprovedByLitigationHandler`. It follows likewise principles as the previous two handlers. The only difference is in the `Handle` method where we now check whether `ApprovedByLitigation` is true or false. And then we add our last handler, the `DocumentApprovedByManagementHandler`, again following the same principles as our other

handlers. The only difference is that here we check the `ApprovedByManagement` boolean. That's it for our implementation. Let's write a bit of code to test this. Let's create a valid document first. We need to import the chain of responsibility namespace for that, so let's do that. We pass through a title, a last modified date, and values stating that this document was approved by litigation and management. And now we create a second one, `invalidDocument`. As the name implies, this is invalid, and we can see that because the boolean we passed through as value for the `ApprovedByLitigation` field is false. Now let's create our chain of handlers. We start with a `DocumentTitleHandler`. That was our first validation rule. And then we chain the other three handlers. For that, we can call into `SetSuccessor`, passing through instances of the handlers we want to add as successor. Seeing `SetSuccessor` returns an `IHandler`, we can chain these three statements. And with that, we've got our chain. Now let's effectively call `Handle` on the chain. I'm surrounding this with a try catch statement in which I catch a validation exception, which is defined in `System.ComponentModel.DataAnnotations`. And I do that because, well, if all works out, we should hit a `ValidationException` when handling the invalid document. So the first thing we do is try and handle the `validDocument`, which should work out, and then we try and handle the `invalidDocument`, which shouldn't work out and throw an exception, which we catch and then write out a message to the console output window. Let's add a `ReadKey` statement, and let's give this a try. And there we go. Our valid document is valid. We never see invalid document is valid written out to the console output window, but we do see a validation exception message. The document must be approved by litigation, which means it isn't valid. Let's have a look at the use cases for this pattern.

Use Cases for the Chain of Responsibility Pattern

Let's start with pattern use cases at a generic abstract level. There's three common ones for this pattern. Use the pattern when more than one object may handle a request and the handler isn't known beforehand. In other words, it should be ascertained automatically. This is made possible because with the chain of responsibility pattern, you create a chain of linked handlers, and each handler gets a chance to process the request, but it doesn't have to. It can also simply pass it on. Another use case is when you want to issue a request to one of several objects, which are the handlers, without specifying the receiver explicitly. You don't know beforehand whether a handler is effectively going to receive the request. And lastly, when the set of objects that can handle the request should be specified dynamically. Translated to our handler example, all handlers and their order in the chain are specified

at runtime. Now, let's look at a few real-life scenarios and use cases where you might consider this pattern. A first good example would be approval workflows. Imagine that you're building an expense approval system within a company. Different levels of management, for example, a team lead, the department, the CFO and so on can each approve or reject an expense request. Instead of hard coding the approval logic into the sender, which would be the expense request form, you can implement a chain of responsibility where each approver is a link in the chain. The request is passed along the chain until it's approved, rejected or reaches the end. Another example would be logging and error handling. In a logging system, you can make different loggers part of the chain of responsibility. For example, you might have loggers for console output, file logging or remote logging. When a log message is generated, it is passed through a chain of loggers, and each logger decides whether to handle the message based on its configuration. A third example is security authorization. Authorization systems can utilize the chain of responsibility pattern. Each security module or authority, for example, role-based access control, a new out token that needs to be validated and so on. So each of those can be a link in the chain to determine if a user has the necessary permissions to access something. And lastly, text processing filters. Consider a text processing pipeline where you want to apply a set of filters to a piece of text. Each filter performs a specific transformation like replacing a specific word or filtering out profanity. These filters can be organized in a chain of responsibility, and the text passes through each filter in the chain.

Pattern Consequences

There's three main consequences to this pattern. The first one is reduced coupling. Our client does not know which handler will handle the request. The object that invokes the operation is decoupled from the one that handles the operation. Like that, we can work towards a single responsibility per class. A second consequence is added flexibility in assigning responsibilities to objects. We can control the order in which handlers are executed at runtime, and we can control which handlers are in the chain, also at runtime, if needed. And the third consequence is a disadvantage. Receipt isn't guaranteed. A request doesn't have an explicit receiver, so there is no guarantee that each handler will receive it nor that it will be handled at all.

Related Patterns

Let's have a look at some related patterns. The chain of responsibility pattern is often applied in conjunction with the composite pattern. There, the parent of a leaf component can act as the successor. In other words, when a leaf component gets a request, it can pass it on to the parent, which can pass it on to its parent and so on. That is a chain of responsibility. There's also a relation between the chain of responsibility and the command pattern. The handlers we implemented can be implemented as commands. When we look at some of the other patterns in this course, we find others that connect senders and receivers of requests. In that regard, chain of responsibility, command, mediator, and observer are all related. Chain of responsibility passes a request along a chain of receivers that can be defined at runtime until at least one handles it or none at all potentially. Command connects senders with receivers unidirectionally, for example, from a button that's pushed in the UI to the action that finally gets executed. The mediator pattern then eliminates direct connections altogether. Communication runs through the mediator. And lastly, the observer allows receivers of requests to subscribe and unsubscribe to them at runtime.

Summary

We looked into the chain of responsibility pattern in this module. Its intent is to avoid coupling the sender of a request to its receiver by giving more than one object a chance to handle the request. It does that by chaining the receiving objects and passing the request along the chain until an object handles it. When implementing it, make sure you provide an easy way to set the next handler. A good idea is to return the next handler, the successor, from that method to enable a fluent interface. Also, use generics to make the handler more, well, generic instead of tying it to a specific type. Up next is one of the related patterns, the observer.

Checkpoint 21 - Implementing the Chain of Responsibility Pattern

Behavioral Pattern: Observer

Coming Up

In this module, we'll cover the observer pattern. Its intent is to define a one-to-many dependency between objects so that when one object changes state, all its dependents are notified and updated automatically. It's also known as publish subscribe, or Pub/Sub. We'll start by describing the pattern using a real-life example. As a real-life example, we will use part of a ticket management system in which services have to communicate with each other. We'll map that to the structural design pattern, and we'll implement it in a demo. We'll continue with learning about use cases for this pattern, pattern consequences, and related patterns. Let's dive in.

Describing the Observer Pattern

The observer pattern is a behavioral pattern from the Gang of Four library of design patterns. Its intent is to define a one-to-many dependency between objects so that when one object changes state, all of its dependents are notified and updated automatically. This is a pretty common pattern. In Angular, you've got an observable class that implements exactly this pattern. When you're working with microservices, you'll often have services that subscribe to changed events on other services to allow them to keep their data stored in sync and so on. What we will use as an example is part of a ticketing management system. It exposes a bunch of services, all with their own data store that needs to be kept in sync, not unlike what you'd expect in a microservices architecture. For example, imagine that there's an OrderService that has a notion of a set of tickets for a certain artist. When the order is fulfilled, the tickets are no longer available, and it updates its data store. But our system knows of other services as well. There's a TicketStockService, which needs to be notified of the fact that the tickets are no longer there so it can update that in its data store. And then there's a TicketResellerService, which also needs to be notified of the fact that the tickets are no longer available. Other services that need to know about this can exist as well, but we'll leave it at this. In any case, we're dealing with a system in which a set of related objects, our services, need to maintain consistency. We can quite easily achieve that. We could simply store a reference to the TicketStockService and TicketResellerService on our OrderService and let the OrderService call methods on them to notify them that the tickets have been sold. But that would introduce tight coupling. It would also become quite complex to manage once the amount of services that need to be notified start growing. Moreover, if we don't know in advance how many or which services need to be notified, this won't work at all. Enter the observer or Pub/Sub pattern. We'll start with a base class, a

TicketChangeNotifier. Our OrderService will be a subclass of this one. On the TicketChangeNotifier, we implement methods to add and remove observers and notify observers. We won't add, remove or notify concrete observers, though. We'll also work on an interface, say ITicketChangeListener. So, it's this interface that's implemented by our observers, the TicketStockService and the TicketResellerService. The interface defines one method, ReceiveTicketChangeNotification, with the change in tickets as the parameter. What happens now is that our two observers are added as observers on our OrderService. The OrderService itself is a concrete implementation of the TicketChangeNotifier. It calls into the method defined on that. But next to that, it holds its state, the data store, containing the information it has on tickets. Once a sale happens, the OrderService adjusts its state by updating its data store. It then notifies all observers of this change through the Notify method on its base class. That method runs through all observers, calls to ReceiveTicketChangeNotification on them, after which each concrete observer can update its own state, in other words, its own data store accordingly. And with that, we know how the observer pattern works.

Structure of the Observer Pattern

Let's have a look at the pattern structure. The first player is the observer. That's our ITicketChangeListener. It defines an updating interface for objects that should be notified of changes in a subject. And that subject, that's our TicketChangeNotification base class. It knows its observers. Any number of observers can observe a subject. Next to that, it provides an interface for attaching and detaching observer objects. The concrete observer maps to our TicketStockService and TicketResellerService. These store state that should remain consistent with the subject state, and they implement the Observer updating interface to keep its state consistent with the subjects. Lastly, the ConcreteSubject stores state of interest to ConcreteObserver objects, and it sends a notification to its observers when its state changes. That would be the OrderService from our example. And there we go. That is the pattern structure. I do want to mention something here. I do want to mention something here. The diagram you see here slightly differs from the original pattern template. In our implementation, we're passing through the state from the concrete subject to the concrete observers as a parameter via the Notify method. In the original template, the concrete observer would hold a reference to the concrete subject so it could query it for state changes. That's a valid implementation

as well, of course. But these days, the one we detailed is more common. In any case, the intent remains the same. And as already mentioned a few times in this course, that is what is important, not the exact pattern template.

Demo - Implementing the Observer Pattern

In this demo, we'll implement the observer pattern, and we're going to start with a helper class, `TicketChange`. This will hold the state change information that needs to be sent to observers. For example, a `TicketChange` for artists with ID 1 and amount 2 means that 2 tickets have been sold for this artist. It's an object of this type that will be received by observers. Then let's add the subject. That is our `TicketChangeNotifier`. It's an abstract class. We don't want it to be used on its own because it doesn't contain the state information observers want to be notified about. That is contained in the concrete implementation. What this does contain is a list of observers, and those are objects that implement `ITicketChangeListener`. We haven't defined that interface yet, so let's do that next. Like that, our code will resolve again, and there we go. It's named `ITicketChangeListener`, and this is our observer. It contains just one method, `ReceiveTicketChangeNotification`, which is used by the subject to notify its observers about a change in the amount of tickets. Then we can start adding methods on our `TicketChangeNotifier`. We need a few methods to manage observers. After all, it is the `TicketChangeNotifier` that needs to keep a list of observers it needs to notify. So we call into `AddObserver`, which accepts an `ITicketChangeListener` as parameter. When the method is executed, the observer gets added to the list of observers. `RemoveObserver` removes an observer from the list. And when `Notify` is called, all observers receive a `TicketChange` notification with the `TicketChange` object passed through as parameter. That's it for the abstract class. Onto the concrete implementations. Our subject is the `TicketChangeNotifier`. And we're going to start with the concrete subject, which is our `OrderService`. Its subclass is the subject, the `TicketChangeNotifier`. What this class is concerned with is its state. For all registration and communication-related tasks, it relies on its base class, the `TicketChangeNotifier`. We're going to add one method to it, `CompleteTicketSale`. It's this method that is called by clients when a ticket sale is completed. That means that at this moment, the `OrderService` must update its state. It must update its data store so the correct amount of tickets for the correct artist is deducted. For demo purposes, we'll assume that when we see an `OrderService` is changing its state message that the local data store is adjusted. After that, it calls the `Notify` method

on its base class, the `TicketChangeNotifier`, to notify the observers. And with that, we've got our concrete subject. The last two parts of the pattern are the two concrete observers. The `TicketResellerService` is our first one. It needs to implement the `ITicketChangeListener` interface. That forces us to implement the `ReceiveTicketChangeNotification` method. This gets a state change from the concrete subject passed in, and that is the `TicketChange` object. Thanks to the info it gets from that object, it can adjust its own data store accordingly. Our last concrete observer was the `TicketStockService`. This too receives a `TicketChangeNotification`, allowing it to update its data store, and that's it for the pattern implementation. Now let's write a bit of code to test this. First of all, we new up our three services. For that, we need to import the `Observer` namespace. So we've got our two concrete observers and we've got our concrete subject. To that concrete subject, we add these two observers. For that, we can call `AddObserver` on the `orderService`. As a parameter to `AddObserver`, we need to pass through something that implements `ITicketChangeListener`, and both our `TicketStockService` and `TicketResellerService` do. Now let's say that a ticket sale happens for the artist with ID 1. Registering such a ticket sale on the `OrderService` means calling the `CompleteTicketSale` method, passing through 1 as ID for the artist and 2 as the amount of tickets. This call should result in both observers being notified of this. Let's give that a try. Let me just add a `ReadKey` statement here, and there we go. And that seems to work just fine. `OrderService` is changing its state, after which it is notifying its observers. And then we see messages from our observers who are being notified of the ticket change. So far, so good. The nice thing here is that we can manage all of this at runtime. We added the observers at runtime. That means we can also remove them. So, we remove one observer, the `TicketResellerService`, and then we complete another ticket sale. Let me add a `Console.WriteLine` statement here, and that makes it a bit more readable. And let's give this another try. And there we go. The first four lines being written out to the console output window remain the same. After that, the `TicketResellerService` observer is removed, and that results in only the `TicketStockService` being notified of the new ticket sale. And with that, we have successfully implemented the observer pattern. Let's have a look at some use cases for it.

Use Cases for the Observer Pattern

Let's start with pattern use cases at a generic abstract level. There's three main situations in which this pattern should be considered. One, use it when a change to one object requires changing others and

you don't know in advance how many objects need to be changed. That's a pretty common scenario in Pub/Sub use cases. It's common in microservice architectures where services that need to be notified can change dynamically. Also, use it when objects that observe others are not necessarily doing that for the total amount of time the application runs. A pattern allows you to remove and subscribe observers dynamically. Lastly, use it when an object should be able to notify other objects without making assumptions about who those objects are. In other words, use it when you want to avoid tight coupling between these objects. Now let's have a look at a few real-life scenarios and cases where you might consider this pattern. The first good example would be stock market data updates. Imagine you're building a financial application. In that case, you could use the observer pattern to keep track of stock prices. Multiple clients or components can observe the stock's price. And when the price changes, all the observers are notified. This allows portfolio trackers, alerts, and analytics tools to stay up to date with real-time stock data. Another very typical use case is event handling in web development. The observer pattern is often used in JavaScript to handle events. DOM elements can act as subjects, and various JavaScript functions can subscribe to these elements to listen for events like a button click, mouse movements or keyboard input. When an event occurs, all the subscribe functions are notified and can react accordingly. Another example might be a weather monitoring system. Imagine you're writing something like that. In that case, the observer pattern might also be useful. Various displays such as digital signs, websites, and mobile apps can subscribe to weather stations. When weather conditions change like the temperature, humidity or wind speed, all subscribers can be updated with the latest data. And the final example I'd like to touch upon is a traffic management system. In such a system, traffic lights can be observed by multiple sensors and cameras. That's a good use case for the observer pattern. When a sensor detects a vehicle approaching an intersection, it notifies the traffic light controller. The controller, acting as a subject, updates the state of the traffic lights accordingly.

Pattern Consequences

Time to look at the pattern consequences. The most important one is that subjects and observers can vary independently. In other words, you can change one or add a new subclass of one without having to change the other. This is in line with the open/closed principle of the SOLID coding principles. Very much related to that, the subject and observer are loosely coupled. All a subject knows is that it has a

list of observers that conform to an interface. It doesn't know about concrete implementations. This too is in line with the open/closed principle. Something to look out for are unexpected updates. In some implementations of this pattern, observers hold a reference to the subject and can change its state. Observers do not know about the potential consequences of this. It can lead to a cascade of change notifications. In our implementation, observers cannot do that.

Related Patterns

Let's look at the related patterns. When we look at some of the other patterns in this course, we find a few others that connect senders and receivers of requests. In that regard, observer, chain of responsibility, command, and mediator are all related. Chain of responsibility passes a request along a chain of receivers that can be defined at runtime until at least one handles it. Command connects senders with receivers unidirectionally, for example, from a button that's pushed in the UI to the action that finally gets executed. The mediator pattern eliminates direct connections altogether.

Communication runs through the mediator. And the observer, as you've learned in this module, allows receivers of requests to subscribe and unsubscribe to them at runtime.

Summary

Time for the module summary. In this module, we covered the observer pattern. Its intent is to define a one-to-many dependency between objects so that when one object changes state, all its dependents are notified and updated automatically. It's also known as publish subscribe, or Pub/Sub. Important to remember, as far as implementation goes, use an abstract base class to implement the Notify, AddObserver, and RemoveObserver functionality. Concrete subjects are not responsible for that. They are responsible for managing their state. Up next is the state pattern.

Checkpoint 22 - Implementing the Observer Pattern

Behavioral Pattern: State

Coming Up

In this module, we'll cover the state pattern, also known as objects for state. It allows an object to alter its behavior when its internal state changes. We'll start by describing the pattern using a real-life example, withdrawing or depositing money from or on a bank account, which can result in the bank account changing state from regular to overdrawn. After that, we'll map it to a structural example, and we'll implement it in a demo. We'll continue with learning about use cases for this pattern and pattern consequences, which can be positive or negative. We will end the module by looking into related patterns. Let's dive in.

Describing the State Pattern

The state pattern is a behavioral pattern from the Gang of Four library of design patterns. Its intent is to allow an object to alter its behavior when its internal state changes. The object will appear to change its class. That may sound a bit weird, so let's clarify this with an example, withdrawing or depositing money from or on a bank account. This can result in the bank account changing state from regular to overdrawn or vice versa. If your bank account is defined by a class, `BankAccount`, you could add an enumeration property, `BankAccountState`, to it. When you withdraw money, the state can potentially go from regular to overdrawn. And when you deposit money, it can potentially go from overdrawn back to regular. Depending on the state of the account, other rules might apply. You may not be able to withdraw money when your account is in overdrawn state. You can solve that with a conditional statement in the withdraw method. Easy enough, right? Now, imagine an additional state, gold, which you get when your account has a lot of money in it. It allows you to withdraw more money at once. That means the logic for transitioning becomes a bit more complex, and the conditional statement for withdrawing becomes a bit more complex as well, but it's still manageable. Yet, it's not hard to see how this can become very complex very fast. Add in a few additional states and/or rules like only allowing you to take out a credit card when your account is in a certain state, changing state of the linked account when you overdraw a credit card, an interest rate that changes, depending on the bank account state and so on. The way the account behaves depends on the state, and the state can change at runtime. Taking care of all of these transitions and states in the way just shown will lead to large, difficult to manage conditional statements and state transitions. The state pattern can help with that. To keep this a bit easier to implement, we'll start with just the two states from the initial

example, regular and overdrawn. We still have the BankAccount class with on it methods to withdraw and deposit money. What we could do is introduce an abstract class that represents the state of our bank account. It declares an interface common to all classes that represent different operational states of the bank account. It includes methods to handle requests that are coming from the BankAccount class like Deposit and Withdraw. We can use an abstract base class for that, the BankAccountState class, as you see on screen. The BankAccountState property on the BankAccount class from our example becomes of this base class type. For each state, we have concrete implementations, so RegularState and OverdrawnState. In these implementations, the logic resides to transition from one state to another. So, in the RegularState's object's Withdraw method, we have code that will change the state from RegularState to OverdrawnState when the balance goes below 0 and vice versa. The more states you have, the more you will notice that this logic will be less conditional and complex than when working with an enumeration because typically you can't just go from each state to all other states. The logic in the withdraw and deposit methods becomes easier as well. For example, there is no conditional statement anymore in the withdraw method on our overdrawn state to check whether we can withdraw money or not. We don't have to check for the state we're in, as the state is represented by the object itself. When depositing or withdrawing money via the BankAccount object, it calls into the underlying Deposit and Withdraw methods on the state object respectively. And as we just detailed, doing that takes care of state-specific logic and state transitions. By doing so, we vastly diminish complexity. Each state object handles just its own possible transitions and logic. The more different states there are, the more obvious the advantages become.

Structure of the State Pattern

The first player in this pattern is the context. That's our BankAccount. This defines the interface that's of interest to clients. It maintains its state via a property of a ConcreteState subclass, which defines the current state. And that's the next player, the ConcreteState subclasses. These map to our regular BankAccountState and OverdrawnBankAccountState classes. Each of these subclasses implement behavior associated with a state of the Context. Lastly, we have the State class. This defines an interface for encapsulating the behavior associated with a particular state of the Context. So that's our abstract base class, BankAccountState. And with that, it's time to implement our real-world example.

We'll do this in two parts. We will first implement the example as we described it. And afterwards, we'll extend the example with an additional state.

Demo - Implementing the State Pattern

Let's start with the State class, `BankAccountState`. We'll implement the pattern with an abstract base class, but you could use an interface for this as well. As we learned, state objects will be responsible for transitioning between states. As state is defined on the Context object, that's the `BankAccount` class from our slides, we need a reference to it. So let's add that Context class just so it resolves, and let's add a reference to it on our `BankAccountState` class. On our State class, we also need methods that handle the request on the Context. We can deposit and withdraw money, so we add two abstract methods for that. Depositing and withdrawing money has an effect on the balance of the account, which in turn is used to decide on potential state transitions. So, we also need a `Balance` property. I made the setters of `BankAccount` and `Balance` protected. Like that, subclasses of our `BankAccountState` can set it, but outside of that only getting the info is allowed. I also added a null forgiving operator to `BankAccount`. Otherwise, we would get a warning stating that `BankAccount` might be null. However, since we know that subclasses will ensure that when they are constructed the `BankAccount` property is set, we can safely add this null forgiving operator, as to give a hint to the compiler that it shouldn't worry. Then, let's add two implementations of this class, `RegularState` and `OverdrawnState`. In the constructor of `RegularState`, we ensure that the balance and `BankAccount` are set. That takes care of ensuring non null values, It's this constructor that will be used when transitioning state. We also see a warning because we didn't completely implement the abstract base class yet. So let's do that, and there we go. Apparently, we need to implement the `Deposit` and `Withdraw` methods. Let's add the `OverdrawnState` ConcreteState as well. Likewise logic applies. Let's implement the abstract class, so the error is gone. And there we go, two methods to implement. Let's implement our `RegularState` first. When depositing money, we're going to write out some info to the console window. That will make it easier for us to see what's going on. We also change the balance by the inputted amount. When you're in `RegularState` and depositing money, there is no state transition that can happen at this moment, so no further logic is required. When withdrawing money, on the other hand, we can change state. When we enter the method, we write this out to the console output window, and we change the balance. Now let's say that when the balance is below 0 after the amount

has been deducted, we're going to change the state to OverdrawnState. To be able to do that, we need to set the state on the BankAccount property to a new instance of OverdrawnState. Yet, our BankAccount class doesn't contain that instance yet. So, let's add an instance of BankAccountState to our BankAccount class first. We will implement this BankAccount class later on. For now, let's continue with implementing the state classes. So we're back in the Withdraw method of our RegularState. And here, we change the state of the BankAccount to a new instance of OverdrawnState. We construct that OverdrawnState object by passing through the current balance and the BankAccount object itself because the OverdrawnState object potentially has to change BankAccount values like the state of the bank account. Let's continue with that OverdrawnState object. When depositing money, we can potentially change back to regular state. If the balance after depositing is larger than or equal to 0, we change the state back to regular. And when we're in this OverdrawnState object, we won't allow withdrawing money anymore. Then let's continue with implementing the Context class. We already added the BankAccountState property. Important here is that we're working on the abstract base class. So we are not working on the implementation. We are coding against an interface to avoid tight coupling and enabling our state scenario. We also add a Balance property, as you would expect a client to want to access the balance from the BankAccount class. Important here is that this is a request to get the balance, which is handled by the state object. So, the getter returns the Balance field on the current state. In the constructor, we set the default state and initialize the account with a balance of 200. The remaining methods are easy. When a request for a deposit comes in, we pass it through to the state object to handle, and we do the exact same thing, which requests for withdrawals. All right, with that, our pattern has been implemented. Let's write a bit of code so we can test this. So we new up our BankAccount. For that, we need to import the State namespace, and then we see what happens when we deposit something, withdraw something, and try to withdraw even more money. This should result in a state switch. Let me add a ReadKey statement, and let's have a look. And there we go. We start in a RegularState. And as you remember, we constructed a bank account with \$200 in the account. We deposit 100, which means we're at 300. So the next time we do something, we're in RegularState, we withdraw 500 from 300, which means our balance is now beneath 0. That means that the state should transition to OverdrawnState, which was indeed the case because the next time we try to do something with our account, withdrawing money, we see that we are in the OverdrawnState. As a result, we are not allowed to withdraw money. Our balance remains at -200.

And there we go. With that, we implemented the state pattern. Let's see how we can extend this example with an additional state.

Demo - Extending the Bank Account Example With an Additional State

In this demo, we will implement support for an additional state, `GoldState`. An account gets gold status when the balance is higher than 1000. We also need an extra rule for demo purposes. So, let's say that when you have gold status and you deposit money, you get an immediate interest of 10%. I think this is the type of scenario we all wish would exist in real life. Nevertheless, it's a nice scenario that will allow us to see how less conditional statements are required. And let's make it a bit more complex. You can transition from gold to regular or overdrawn and from regular to gold, but you can never directly transition from overdrawn to gold. You will need to be in regular status for at least one deposit. That means that the rules are a bit more complex than just looking at the balance. You can be in regular status even when you have a balance of more than 1000. All right, let's implement this scenario. We start with adding the `GoldState` class. It derives from `BankAccountState`, which means we should implement the `Deposit` and `Withdraw` methods. And just as with the other `ConcreteState` implementations, we're going to add a constructor that accepts the balance in `BankAccount` and sets them. The deposit method is easy. We're in `GoldState`, so that means that when money is deposited, we get 10% interest applied immediately. So no conditional statement is required here. We simply add 10%, as we are already in the `GoldState` class. We don't need to check the status. In the `Withdraw` method, we do need conditional statements to potentially transition to the two other states. So what we're saying here is that you can go from `GoldState` to either regular or `OverdrawnState`, depending on your account balance after you've withdrawn money. We also need to make a small change to our `RegularState`, as you can potentially transition to `GoldState` from `RegularState` when depositing money. So, if the balance is over 1000 or equal to it, we switch to `GoldState`. Another rule we wanted to implement was that you cannot transition from overdrawn to gold directly. You'll at least need to be in `RegularState` for one deposit. And this rule is remarkably easy to implement. We don't have to change anything to our codebase, as our `OverdrawnState` object only implements a transition to `RegularState`. All that's left is adding a few additional statements for testing to our `Program` class. First, we're going to deposit enough money to get to gold status. Then we check whether that's indeed the case, and then we try and go back to overdrawn status. After we deposit more money, we should

transition to regular instead of gold, and we can check for that by doing a deposit of 100. Let's give this a try. The first three statements are the same as in the previous demo. Then, we're in `OverdrawnState`, and we deposit 2000. That means we now have a balance of 1800, so we switch to `RegularState` because we can't directly switch to `GoldState`. When we deposit 100 while in `RegularState`, we move to `GoldState` because our balance is now 1900. And when we withdraw 3000 from 1900, we automatically go to `OverdrawnState`. And when we're in `OverdrawnState` and deposit 3000, we have a balance over 1000. But because of the rule that we cannot immediately transition to `GoldState`, we remain in `RegularState`. So these are already quite complex rules, but all we had to do to implement all of this is create one new subclass and change the transitions. We didn't have to change the `BankAccount` class itself, and we didn't end up with as many conditional statements if we would have worked with an enumeration property for state on the `BankAccount` class. Even our rather complex transitioning rule was easy to implement. Let's have a look at the use cases for this pattern.

Use Cases for the State Pattern

Let's start with the abstract use cases for this pattern of which there are two. The first one is when an object's behavior depends on its state, and it must change its behavior at runtime, depending on that state. That is the use case we just implemented with our bank account example. Another good use case is when your objects are dealing with large conditional switch or if statements that depend on the object's state, for example, by storing the state in an enumeration field on the object. This often results in several of your object operations to contain the same conditional structure. That is what we would have ended up with in our naive implementation, as you've learned when we described the pattern. By using the state pattern, each branch of the conditional is put in a separate class, which lets you treat the object's state as an object in its own right. It can thus vary independently from other objects. Then let's have a look at a few real-life use cases, scenarios you may encounter in your day-to-day life as a developer. A first good use case is document editing software. Consider a text editor where the user can switch between different editing modes such as insert, override, and select. Each mode has a different behavior when the user types or interacts with the document. If you're building a system like this, you can use the state pattern to manage the behavior associated with each editing mode. The next example is vending machine software. Vending machines have various states such as idle, payment, product selection, and dispensing. The machine's behavior changes depending on which

state it's in. For instance, it should only dispose a product when it's in the dispensing state. If you're building such a system, you can use the state pattern to encapsulate the behavior associated with each state of your vending machine. And now we have workflow management. It's related to the previous example because in a workflow management system, tasks or processes can also have various states like pending, in progress, completed or canceled. The behavior of a task like notifying stakeholders, updating progress or allowing further actions can change based on the current state. Here too, you can use the state pattern to model and manage these states and their transitions. All right, as a final example, let's look into a pretty cool one, robotics. Imagine you're building the software for one of these. Autonomous robots often have different operational modes such as exploration, obstacle avoidance or charging. You can use the state pattern to represent these modes and govern the robot's actions and decision-making processes accordingly.

Pattern Consequences

Let's have a look at the pattern consequences. The first consequence is that this pattern localizes state-specific behavior and partitions behavior for different states. We saw that, as all the behavior associated with a particular state was put in an object of its own. This works towards the single responsibility principle from the SOLID coding principles. The good thing is that because state-specific code lives in a state subclass, new states and transitions can easily be added by defining new subclasses. We saw that when we extended our example with a new state in the second demo. That is the open/closed principle from the SOLID coding principles at work. But there's also a potential negative consequence. The pattern distributes behavior across different state subclasses. That means that the number of classes is increased, which adds additional complexity. But as long conditional statements are rarely desirable, this becomes more of a positive consequence than a negative one once the amount of state subclasses is large.

Related Patterns

There's a few related patterns to the state pattern. The first one being the flyweight pattern. When you implement the state pattern, without instance, variables in the state objects, you're essentially making the state objects shareable, which makes them flyweights. There's also a relation to the singleton

pattern. As state objects often are singletons strategy bridge and state are all related and look similar because they are based on composition even though they solve different problems.

Summary

Let's have a look at the module summary. We learned about the state pattern in this module. The intent of the state pattern is to allow an object to alter its behavior when its internal state changes. If you find yourself in a situation where you have an object, which can have different states you're currently storing in an enumeration, which you then use inside of that object to modify its behavior, depending on that state, well, in that case, this pattern is a really helpful one to look into, in my opinion. As far as implementation goes, remember that it's in the state objects themselves that the transitions to a new state are handled. That means that each state object needs a reference to the Context class. What's also important to remember is that requests on the context that either need to take the state into account or can result in state transitions are handled by the state objects, so the context needs to pass these requests through to the underlying state objects to handle them. That means that you need methods to handle these requests on the abstract base class. Up next, the iterator pattern, something you use every day.

Checkpoint 23 - Implementing the State Pattern

Behavioral Pattern: Iterator

Coming Up

In this module, we'll dive into the iterator pattern. Its intent is to provide a way to access the elements of an aggregate object sequentially without exposing its underlying representation. I'm Kevin, and I'll guide you through this module. We'll start by describing the pattern, and you'll notice very soon that this is a very common pattern, at least to use, not to implement. Yet, we are going to implement a custom iterator by automatically iterating over the list in alphabetical order. We will use that as an example. After that, we'll map that to a structural example, and we'll implement it in a demo. We'll

continue with learning about use cases for this pattern and pattern consequences, and we will end the module as all modules by looking into related patterns. Let's dive in.

Describing the Iterator Pattern

Iterators are everywhere. They're one of the most common patterns used by frameworks to traverse their collection. Yet, even though it's a very common pattern, I don't see it used that much in custom code anymore by which I mean that the frameworks we use tend to already have all the iterators in place we commonly need. The intent of an iterator is to provide a way to access the elements of an aggregate object sequentially without exposing its underlying representation. That sounds a bit complicated maybe, but simply change aggregate object to enumerable or array or collection, and the intent all of a sudden becomes a lot more tangible. Whenever we use a `for each` statement over an enumerable, say a list of string, it's an iterator that is used to provide the correct previous or next item. Let's continue with that list example. That is an aggregate object, which aggregates a set of variables, potentially other objects, lists. But also other aggregate objects like collections, dictionaries, stacks, and queues keep their items in an internal structure. You don't want to expose this internal structure to avoid it being tampered with in unintended ways. Also, you might not always want to traverse the aggregate object in the same way. You may want to allow traversing it alphabetically or in reverse order, and you don't want to bloat the aggregate object interface with all these different ways of traversing it. That is what the iterator solves. It takes the responsibility for traversing out of the aggregate list object and puts it into an iterator object. Now, as mentioned, most common iterators on the collections we use already exist. So as an example, we're going to create a custom collection, a collection of people. On it, we're going to create an iterator that iterates over these people, but it's going to do that in alphabetical order by the person's name. We'll name our custom collection `PeopleCollection`. On it, common methods like you'd expect in a collection exist like `Add`, `Remove`, and `Count`. It should also have a `CreateIterator` method. That will return our iterator and will define the need for such a method on an interface, `IPeopleCollection`. So, the `PeopleCollection` interface defines an interface for creating an iterator object. We'll name that `IPeopleIterator`. And that too is an interface. Like this, we avoid having to use concrete implementations on our `PeopleCollection`, which means we can easily provide multiple different iterators without having to change the class. The `PeopleIterator` interface contains four methods, `First`, `Next`, `IsDone`, and `CurrentItem`. These are the methods

required for iterating over a collection. The last part of the story is the concrete implementation of the iterator, the `PeopleIterator`. It implements the iterator interface, and it will contain the actual logic for iterating over our `PeopleCollection`, sorted alphabetically by name.

Structure of the Iterator Pattern

Let's map this to the pattern structure. We'll start with the `Aggregate`. This defines an interface for creating an `Iterator` object. That maps to our `IPeopleCollection`. That `Iterator` object, that's what defines an interface for accessing and traversing elements. It maps to our `IPeopleIterator` interface. And now we have the two concrete implementations. The `ConcreteIterator` implements the `Iterator` interface and keeps track of the current position in the traversal of the `Aggregate`. It maps to `PeopleIterator`. And lastly, we have the `ConcreteAggregate`. This implements the `Iterator` creation interface to return an instance of the proper `ConcreteIterator`. And this one maps to our `PeopleCollection`. So far for the pattern structure.

Demo - Implementing the Iterator Pattern

In this demo, we will implement our custom iterator over the `PeopleCollection`, which will iterate over it in alphabetical order. Let's start with a helper class, a `Person`. `Person` has a name and a country where the person is coming from. Up next, we define our `Iterator` interface. It's named `IPeopleIterator`, and it's a copy from the pattern structure mapped to our example. So, what it returns from its methods, where applicable, are `Person` objects, as that is what we're going to iterate over. The next interface we want to define is our `IPeopleCollection`. That's the `Aggregate`. As we learned, it should define one method, `CreateIterator`, returning an `IPeopleIterator`. The `ConcreteAggregate` is then our `PeopleCollection`. Now, we could make this hard on ourselves. We could start from scratch with implementing a complete new collection with an internal data structure to hold our set of people with add and remove methods, indexes and so on. However, we already have all that functionality in the built-in framework classes. So, instead of starting from scratch, we're going to subclass the generic `List` class. We'll make this derive from `List<Person>`. The one thing we do need to add is an implementation for the `CreateIterator` method from our `IPeopleCollection` interface, so that's something we do have to do. The signature of the method signifies that it returns an `IPeopleIterator`. The method

itself should return a concrete implementation of that iterator. We haven't got that yet, but we know from the slides that we named it `PeopleIterator`. We also pass through the object itself, as the `PeopleIterator` will require access to it to iterate over the items. And that brings us to the last part to implement, the concrete iterator, our `PeopleIterator`. This is where most of the logic resides. Iterators can become quite complex, storing numerous in between positions and variables to return the correct item. Let's start by implementing the `IPeopleIterator` interface. In our case, we want to alphabetically order the list. We can again use built-in framework functionality for that. The first thing we will do is create a constructor that accepts our `PeopleCollection`. Like that, the code from our `PeopleCollection` that creates the iterator will resolve again. So, in our constructor, we accept that collection, and we store it in a private field. The other field we add is the `_current` field, which will store the current position in our `PeopleCollection`. So the idea is that now we're going to implement `IsDone`, `CurrentItem`, `First`, and `Next` in a way that returns our people in alphabetical order. Let's start with `First`. When we want to get the first element in our collection, we can say that we want the current position to be set at 0. And then we return the person at the current position after having ordered the `PeopleCollection` by the person's name. When we want to get the next person in line, we add one to the current position. As long as we are not done, that means as long as there are still items in the list, we will implement `IsDone` immediately. As long as that's not the case, we can select the next person in line. We again order the `PeopleCollection` by name, and we get the one at the current position. If we are done, in other words, if there are no more people in the list, we can potentially return null. What you want to do when someone tries to get the next item in a list and there is no next item kind of depends on your use case. Returning null is pretty common here. Two things left, implementing `IsDone` and `CurrentItem`. These were defined as properties. Getting `IsDone` means checking whether the current position is larger than or equal to the amount of items in the `PeopleCollection`, so that statement will either return true or false. Getting the `CurrentItem` uses the indexer, passing through current test position after having ordered our `PeopleCollection` by name. That should be it. Let's write a little bit of code to test this. First, we new up a `PeopleCollection`. For that, we need to import the `Iterator` namespace. Now let's add a few people to it. These are all fellow authors living in Belgium or the surrounding countries. The next thing to do is create the iterator we'll use to iterate over our collection. For that, we call in to `CreateIterator` on our `PeopleCollection`. And now we use that iterator to run through all items in the collection. For each person, we write out a name. Like that, we will

easily be able to see whether they're truly sorted alphabetically. As you notice, in our for loop, we also use the First, IsDone, and Next operations on our peopleIterator. Let's give this a try. Let me just add a Console.ReadKey statement here, and there we go. And as you notice, the authors are now sorted alphabetically. Let's look into the use cases for this pattern.

Use Cases for the Iterator Pattern

Let's start our look into use cases with a look at some generic rather abstract use cases. You can use this pattern to access an aggregate object's content without exposing its internal representation. As mentioned before, this avoids unintended access to and manipulation of that internal representation. But it can also be used to hide away complexity when you're working with a complicated data structure. Think about complicated tree structures, for example. Another good use case is when you want to support multiple ways of traversal for the same aggregate object. And yet, another one is avoiding code duplication. If the way you want to traverse the structure is complicated, chances are that it's not trivial to write. If in such a case you want to traverse your structure in multiple places in your app, it's a good idea to put that code in an iterator and reuse that to avoid code duplication. Let's switch to a few real-life use cases or scenarios you may encounter in your day-to-day life. A first good use case are database query results. When dealing with a large set of database records, you can use an iterator to fetch and iterate through query results. This abstracts the database-specific implementation details and provides a uniform way to access records, regardless of the database system used. And then we have document parsing and analysis. Imagine you're writing code to process a structured document like an XML or JSON document. If you're doing that, you can use an iterator to traverse the elements within the document. This would allow you to encapsulate the logic for parsing and navigating the document while keeping the client code clean and independent of the document's structure. And now we have inventory management systems. If you're writing such a system for a business, you can use an iterator to iterate through product listings, order histories or warehouse inventory. This simplifies the code and makes it easier to implement different traversal algorithms without changing the underlying data structure. And now we have a remote control interface for a smart home. In smart home systems, you can use an iterator in the remote control interface to cycle through connected devices such as lights, thermostats or smart appliances.

Pattern Consequences

Let's have a look at a few consequences of this pattern. An important one is that iterators simplify the interface of your aggregate structure. That's because the traversal code is separated out in classes of their own. This follows the single responsibility principle. Seeing we are working on interfaces and not implementations, the open/closed principle applies as well. You can implement new types of aggregate objects and iterators without them interfering with each other. Iterators can also exist next to each other at the same time on the same collection. In other words, you can be traversing a collection with an alphabetical iterator at the same time as you're traversing it in the order that items were added to the collection. As these iterators store their own position, they won't interfere with each other. A disadvantage is that using this pattern can be a bit overkill when you only use simple traversals and collections. Iterators you have to build yourself are mostly useful for more complex situations.

Related Patterns

There's a few related patterns for the iterator pattern. The first one is the composite pattern. That is a recursive structure, and iterators are often used to traverse those. An iterator can also be used in conjunction with the memento. In that case, the memento can be used to store the state of the iterator and potentially roll it back. And lastly, the visitor pattern can be used in combination with the iterator. You could use an iterator to traverse a potentially complex data structure and use the visitor to apply some logic to the items in that structure.

Summary

Time for the module summary. We learned about the iterator pattern in this module. It's very common, but mostly as far as using it is concerned, not as far as implementing goes. Its intent is to provide a way to access the elements of an aggregate object sequentially without exposing its underlying representation. In regards to implementation, it's quite easy to build all parts of the iterator pattern except the actual concrete iterator itself. Remember that for that, you can leverage existing framework code. There is no need to start from scratch here. Up next, we will look into the visitor pattern. That's one of the related patterns to this one.

Behavioral Pattern: Visitor

Coming Up

In this module, we'll dive into the visitor pattern. A visitor represents an operation that has to be performed on the elements of an object structure. I'm Kevin, and throughout this module, I will teach you what that means. We'll start by describing the visitor pattern. For that, we will use a real-life example, calculating discounts for employees and customers. After that, we will map that to a structural pattern, and we will implement it in a demo. We'll also implement a variation of the pattern. We'll continue with learning about use cases for the pattern and pattern consequences. We will end the module by looking into related patterns.

Describing the Visitor Pattern

The visitor pattern is a behavioral pattern from the Gang of Four library of design patterns. Its intent is to represent an operation to be performed on the elements of an object structure. Visitor lets you define a new operation without changing the classes of the elements on which it operates. That definition requires some further explanation. Imagine the following use case, you're building part of an order management system. You've got a large set of objects, but they are not all the same. You've got an `InternalCustomer` class, a `GovernmentCustomer` class, a `PrivateCustomer` class and so on, all deriving from `Customer`, but all with their own custom behavior and properties. At a certain moment, a requirement comes in to add new behavior to all these objects. You need to calculate a discount, and that discount can be different for each type of customer, as it depends on the amount of money the customer previously spent on orders. To add this behavior, you can implement it in all these classes or potentially, depending on what needs to be known about a customer, you could even implement it in a base class, but there's other types as well. Let's add an additional class to the mix, an `Employee`. Employees of companies also get discounts, but these are calculated differently. They are based upon the amount of years they are employed. Employee and the Customer classes don't have the same

base classes, so we cannot use that approach anymore. We can, however, still add the methods to calculate the discounts on each concrete class. Yet, what if after some time a new requirement comes in? You now need to calculate some shipping costs for each customer and employee. And after that, you need to add JSON exports to all the objects and so on and so on. This becomes cumbersome. You always need to change these classes, which is risky. Also, we need to start wondering whether it makes sense to have all that added behavior in all of these classes. Maybe it doesn't belong there if we keep the single responsibility principle in mind. Enter the visitor pattern. This allows adding behavior to classes by adding the behavior to another class, the visitor, which will visit the original classes and execute the behavior. To avoid overloading the screen with a huge diagram, let's just stick to a Customer class and an Employee class unrelated to each other and with our own properties and operations. But there can, of course, be many more. They have a Discount field, which we will calculate using our visitor. We have both of them implement an interface, the Element interface, which defines one method, Accept. This method accepts a visitor or rather an IVisitor, as we want to work on an interface here. On that interface, we define a method to visit a customer, which accepts a customer, and one to visit the employee, which accepts our employee. In other words, one method per class, so the discount can be calculated correctly, as the algorithm may be different for each class. Concrete class, DiscountVisitor, implements the IVisitor interface and contains the actual logic to calculate the discounts. The last thing we need is an object structure that accepts our DiscountVisitor and is responsible for running through the customers and employees and passing the visitor through to the Accept methods on them so the discount can be calculated. This can be a list or composite, depending on your use case. Let's say we want to name that one container. And with that, we've got our visitor pattern.

Structure of the Visitor Pattern

Let's have a look at the structure of the visitor pattern. We will start with the element, which maps to our IElement interface. It defines an accept operation that takes a Visitor as an argument. Then we've got ConcreteElement implementations. These are our Customer and Employee classes. A ConcreteElement implements the accept operation that takes a Visitor as an argument. The ObjectStructure then can enumerate its elements. It typically provides an interface to allow a visitor to visit its elements, and it can be a composite or a collection. As you've guessed, that maps to our

container. Up next is the Visitor, which maps to IVisitor. This declares a visit operation for each class of ConcreteElement in the ObjectStructure. And then we've got ConcreteVisitors. We've only got one, our DiscountVisitor. But as we learned in the previous clip, others are possible as well. These ConcreteVisitors implement each operation declared by the Visitor, so they contain the actual algorithm. Often, they also contain state that accumulates while traversing all the objects, which can be useful for callers like storing the total amount of discount given. And with that, we've got our pattern structure. Time to implement our example in a demo.

Demo - Implementing the Visitor Pattern

In this demo, we will implement the visitor pattern. Let's start with our concrete elements, the Customer and Employee classes. We will give our customer and employee a name. The way discounts are calculated for these are different. For the customer, it's 10% of the total amount ordered, so we add a property for that. We also add a Discount property, which will be set by the visitor, therefore the setter cannot be private. We want to ensure that a customer is constructed with the amount ordered and name passed through, so we add a constructor to that avail. Likewise, we create an Employee class. For an employee, the discount will be calculated, depending on the amount of years the employee has been employed. Up next are two interfaces, the IVisitor interface and the IElement interface. Note that these can be implemented as abstract base classes as well if you wish to provide some out-of-the-box functionality. For our use case, that's not applicable. Let's start with the IVisitor interface. As we learned on the slides, it needs methods to visit our concrete elements, VisitCustomer and VisitEmployee. The IElement interface then contains one method, Accept. That method must accept an IVisitor so it can visit the concrete elements and execute its algorithm on them. Our concrete elements must thus implement this IElement interface, so let's do that next. Let's scroll back up to our Customer class. We have it implement the IElement interface, leading to an Accept method on this Customer class. The IVisitor gets passed through via the Accept method. So what we can do now on this IVisitor interface is call VisitCustomer because that was one of the methods that was created on it. We pass through the Customer object itself, so the visitor has access to the customer properties it needs to do its calculation. Also, for demo purposes, we write out some info to the console output window. And we do something likewise for our Employee class. We implement the IElement interface. That leads us to implementing an Accept method. IVisitor gets

passed through. So on `IVisitor`, we can call `VisitEmployee`, passing through this, the current employee object. Here too, we write out some info to the console output window. We haven't got an actual visitor yet though, so let's create that next. We'll name it `DiscountVisitor`. It must implement the `IVisitor` interface. That means we need to implement two methods, `VisitCustomer` and `VisitEmployee`. In the `VisitCustomer` method, we can calculate the discount for a customer, 10% of what the customer ordered. The visitor accesses the customer properties it needs, and it also changes the `Customer` object. It sets the discount. So, visitors can change values on the object they visit, but they also often store information about their operation results that can be useful for callers. So let's add a property to our visitor, `TotalDiscountGiven`. It will contain a summation of the discounts provided across all elements. So that means that when we visit a customer and we calculate a discount, we add that to our `TotalDiscountGiven` property. For employees, the calculation is a bit different. When someone has been employed for less than 10 years, a €100 discount is given, otherwise a €200 discount is given. Here too, we add the discount that's given to the total amount. Lastly, we need a structure to hold employees and customers. That's the object structure from the pattern structure. We name it `Container`, and it's a pretty straightforward class. It stores lists of employees and customers, and it allows passing through a visitor to each of them. This is thus a composite. With that, our pattern implementation is done. Let's write a bit of code to test this. We new up a container. For that, we need to import the `Visitor` namespace, and then we add a few customers and employees to it. For these, the discounts will be calculated. We new up a `DiscountVisitor`, pass it through to the container, and write out the gathered amount of discount given in total. Once a `DiscountVisitor` is passed through, it will visit all our customers and employees, which means that once we reach this last `WriteLine` statement, `TotalDiscountGiven` should effectively contain the total discount given. Let's add a `ReadKey` statement, and let's give this a try. And there we go. As we notice, all employees and customers were visited. Discounts were calculated by the `DiscountVisitor`, and all these discounts added together results in a total discount given of €530. So there we go. But are we there yet? Let's study a potential issue with this implementation and the solution in the next demo.

Demo - Simplifying the Visitor Interface

We're looking at our `IVisitor` interface. One of the issues with this approach is that this interface needs to be changed whenever a new concrete element is added because it contains methods that work on

these concrete elements like Customer and Employee. To avoid that, this interface is often simplified. Instead of having it contain methods for each concrete type, we have it contain one method, Visit, that accepts an IElement instead of a concrete implementation. All our concrete implementations have to implement IElement anyway, so this should work, but it also has consequences. First of all, we need to make a change to our Customer and Employee classes. In their Accept methods, it's now the Visit method that needs to be called. We will also have to change the DiscountVisitor itself. If we implement IVisitor now, we see we only have one method to implement, Visit. So that means it's up to the DiscountVisitor to check the actual concrete type of the element so it can execute the correct logic. So, we adjust our old methods by making them private. And in our Visit method, we check for the actual type, and depending on that, execute the correct algorithm. No need to change anything to the client. That should still work as before. So let's give this a try, and there we go. It still works as expected. We've made our interface simpler, so we won't have to change that whenever the new concrete element is added to our object structure. Yet, we have introduced additional logic in the visitor implementation. It's this implementation I see more often in the wild, but both implementations we made are valid. The intent remains the same. It's just the template that's changed. Let's have a look at the use cases for this pattern.

Use Cases for the Visitor Pattern

Let's start with a few abstract use cases. A good use case for this pattern is when an object structure contains many classes of objects with differing interfaces and you want to perform operations on them that depend on their concrete classes. In our example, these concrete classes were Customer and Employee. Another good use case is when you're in a situation where the classes defining your object's structure don't have to change often. But you do often want to define new operations over the structure, a DiscountVisitor, a JsonExportVisitor and so on. Using this pattern avoids having to change the classes, Employee, Customer that define your object structure. Lastly, use it when you've got potentially many operations that need to be performed on objects in your object structure, but not necessarily on all of them. The visitor pattern avoids that you have to pollute all your objects with these operations, even when they don't apply to them. Of course, a bit of reflection can help here as well, but that's typically a costly operation you want to avoid. All right, let's continue with some additional real-life use cases and scenarios you may encounter in your day-to-day life. A first good use

case would be document element processing. In those types of applications, you may have various document elements like paragraphs, tables, images, and more. Instead of adding formatting or analysis logic directly to these elements, you can use the visitor pattern. For example, you can create a visitor for spellchecking or formatting, and each document element can accept the visitor to perform the necessary operations without altering their core structure. Another example would be a medical record system. Imagine you're building that. You'll probably end up with various patient data structures. In that case, visitors can be used for generating medical reports, for statistical analysis or for billing calculations. New types of analysis or reports can be added by creating new visitor classes, keeping the patient data structures clean and focused on their primary purpose. Another one is serialization of a data structure. When serializing complex data structures like trees, graphs or object hierarchies, you can use the visitor pattern to separate the serialization logic from the data structure classes. This way, you can create different visitors for JSON, XML or binary serialization, ensuring that the structure of the data remains intact while the serialization logic can be customized. And lastly, financial transactions. In a financial system, you might have various types of financial instruments like stocks, bonds, and options. These require different calculations or different ways to report on them. By using the visitor pattern, you can create visitors for doing risk assessments or for generating financial reports, and you can apply these visitors to the relevant financial instruments without modifying their core implementations.

Pattern Consequences

Let's have a look at a few pattern consequences. The visitor pattern makes adding new operations easy. You can define a new operation simply by creating a new visitor. There is no need to change the class as the visitor visits. This is the open/closed principle at work. A definite advantage is that you can use the visitor to accumulate info on the objects it visits. We did that in our example by accumulating the total discount. What's also an advantage is that the visitor gathers related operations together and potentially separates unrelated ones. For example, all operations related to calculating discounts are grouped together in the same Visitor class, all operations for calculating shipping costs as well and so on. Otherwise, they might be separated out into the classes that are visited where they would represent a group of unrelated operations. This advantage works towards the single responsibility principle. There's also disadvantages. Adding a new ConcreteElement class can be hard, as you will

potentially have to change all visitor interfaces and concrete implementations. And lastly, the visitor pattern might require you to break encapsulation. The visitor needs access to all properties or operations on each concrete element to do its job, which might require you to make more operations accessible on these classes than would otherwise be required.

Related Patterns

There's two related patterns to the visitor pattern. The first one is the composite pattern. We know this already, as our object structure is a composite. So, the visitor pattern can be used to apply an operation over an object structure defined by the composite pattern. It can also be used together with the iterator. You could use an iterator to traverse a potentially complex data structure and use the visitor to apply some logic to the items in that structure.

Summary

Time for the module summary. We learned about the visitor pattern in this module. Its intent is to represent an operation to be performed on the elements of an object structure. Visitor lets you define a new operation without changing the classes of the elements on which it operates. In regards to implementation, remember that the `IVisitor` interface and thus implementations work on concrete elements. As an alternative, you could have it work on the element interface, but your visitor will still require knowledge of the concrete element classes to execute its algorithm. Up next, the interpreter pattern.

Checkpoint 25 - Implementing the Visitor Pattern

Behavioral Pattern: Interpreter

Coming Up

In this module, we will dive into the interpreter pattern. It's not a pattern you'd use often, but it is pretty cool. It's used to define grammar and interpret languages. Let's learn what that's all about. I'm Kevin, and I'll guide you through this module. We'll start by describing the interpreter pattern. As an example, we'll write a piece of code to translate Arabic numerals to Roman numerals. We will map that to a structural example, and we'll implement it in a demo. We'll continue with learning about use cases for this pattern and pattern consequences. We will end the module with looking into related patterns.

Describing the Interpreter Pattern

The interpreter pattern is a behavioral pattern from the Gang of Four library of design patterns. Its intent is to, given a language, define a representation for its grammar along with an interpreter that uses the representation to interpret sentences in the language. In other words, it defines a grammatical representation for a language and an interpreter to interpret the grammar. And that is why it's not something you'll often create yourself. You don't define a language every day, yet it is something we use a lot in our day-to-day programming life. When you use LINQ queries, behind the scenes, an interpreter-like pattern can handle translation between LINQ and, for example, SQL or the interpreter for regular expressions or even the way the C# compiler interprets the code we write. All of these could use a variation of the interpreter pattern. There are other patterns and possibilities. We'll cover that once we get to the use cases, but know that it would be possible to use the interpreter pattern for these examples. But what does that mean? Imagine the use case in which we want to translate Arabic numerals to Roman numerals. Say that we've got the number 181. How would we go about that? If we stick to the hundreds as maximum, there's 999 or 1,000, if we include 0, possible outcomes. Imagine the switch statement we'd have to write. It's completely unmanageable. Enter the interpreter pattern. We can look at 181 as a sentence in a grammar we're going to define. That grammar consists of a set of expressions. Each expression is then responsible for translating the word, one number in our case, that is passed in. So, we define a grammar through a set of expressions. You can call that a syntax tree, and each expression interprets a word. So, we don't need a huge switch statement. What we do need is a way to define these expressions, a `RomanExpression` abstract class or interface that declares an `interpret` operation. This operation gets some information passed in, an object that contains info on what we want to translate and typically a way to store the result. Say we name that `RomanContext`. And then we have different implementations of that

expression, the RomanHundredExpression, which interprets the hundreds, and then two I put on the diagram, the RomanTenExpression, which interprets the tens, and the RomanOneExpression, which interprets the ones. I'm sticking to two on our diagram to keep it more readable. The client then defines the grammar, the syntax tree. and provides the RomanContext. And with that. we have described the interpreter pattern.

Structure of the Interpreter Pattern

Let's have a look at the pattern structure. We'll start with the first player, the AbstractExpression. This maps to our Roman expression abstract base class. What it does is declare an abstract interpret operation that is common to all nodes in the abstract syntax tree. Then, we have the TerminalExpression, which is an implementation of the abstract expression. This implements an interpret operation associated with terminal symbols in the grammar. An instance is required for every terminal symbol in a sentence. This matches to our RomanTen and RomanOne expressions. But what's a terminal symbol? And if there's something called a terminal expression, there's probably also something called a non terminal expression, right? That is indeed the case. But we're going to park that for now, as to not overcomplicate things. We'll get back to terminal and non terminal symbols and expressions right after the demo. The next player is the Context, which maps to our RomanContext and contains information that's global to the interpreter. Input and output properties are common on this one. Lastly, the Client. This builds or is given the abstract syntax tree, which represents a sentence in the language that the grammar defines. That tree is assembled from expressions. The client also invokes the interpret operation. With that, let's implement our example in a demo.

Implementing the Interpreter Pattern

In this demo, we will create a translator, which will translate Arabic numerals to Roman numerals. Let's start with the basics. We add a Context class, RomanContext, containing an input and output property. This is what will get passed through to our expressions. So for that, we create an abstract expression base class. In our case, we name that RomanExpression. I am using an abstract class in this example. But as I've mentioned previously in the course when using abstract classes, you could also use an interface for this as long as you don't have the need to provide default implementation logic.

Our RomanExpression contains one abstract method, Interpret, which accepts the context. Then it's on to the actual logic, the implementations of our abstract RomanExpression class. In our example, these are the terminal expression classes from the pattern and structure. Let's start simple with a one expression, so numbers 1 through 9, but how do we translate that to Roman numerals? Well, numbers 1 through 9 are represented as such in Roman numerals. So, we could write a switch statement for all nine possibilities. But if you think about it, we can simplify this. Take 3, for example. Translating that to Roman literals is three times capital I. That's the same as translating one to Roman literals three times, and two, that's two times one. So, instead of having a switch statement for 3, 2 and 1, we could say that we write a statement for dealing with 1 and subtract 1 each time. That means that if we have the number 3, we will hit our logic to write 1 as capital I three times, resulting in III, which is what we need. With that in mind, we can simplify our logic to just four cases, 9, 5, 4, and 1. All the rest of these are combinations we can reach with simple subtraction, putting code that translates to what you see on screen now. That takes care of our RomanOneExpression. The next one in line is the RomanTenExpression. And if you think about it, this follows likewise principles as the RomanOneExpression. These nine different numbers don't necessarily result in a switch statement with nine potential cases. They can be simplified to four. So, that results in a RomanTenExpression deriving from RomanExpression in which the Interpret method takes care of these four potential cases. That leaves us with the RomanHundredExpression. And you guessed it, we can implement likewise logic for this. And there we go. That takes care of our RomanHundredExpression. That's it. On to our client to write a bit of code to test this. First, we create a syntax tree. We do need to import the Interpreter namespace for that. So, we create a new list of Roman expressions, stating we are going to start with the HundredExpression, then the TenExpression, and then the OneExpression. That is the order in which our sentence will be translated word by word. Then we create a context. When creating that context, we pass through the input, 5, in our case. We run through our tree of expressions, and we interpret the input. After that, we write out the result. In this case, RomanHundredExpression won't result in anything and neither will RomanTenExpression, but RomanOneExpression will result in 5 being translated into 5 in Roman numerals. And let's do the same for a few other Arabic numbers. So we tried to translate 81 and 733. Let me add a ReadKey statement here, and let's give that a try, and there we go. My Latin is a little bit rusty, but I'm pretty sure

these numbers are correctly translated. Up next is the part we previously skipped when looking into the pattern structure, terminal versus nonterminal expressions.

Terminal vs. NonTerminal Expressions

When we looked into the pattern structure, we learned that one possible implementation of an abstract expression is a terminal expression. There's another possible implementation, a nonterminal expression. It's not applicable to our example, but it's nevertheless interesting to know about. So, all languages consist of symbols. A symbol is the smallest meaningful part or unit of language, a word, a number, an operator, things like that. These symbols are called terminals implemented through a terminal expression. When you think about that, you notice that all these terminals make up statements, for example, $5 + 10 = 15$. That is a statement made up of a set of terminals. In a programming language, you could have a while or a conditional statement made up of terminals. Well, the statements that are made up of terminals and that are allowed by a language, those are called nonterminals. Nonterminals can consist of terminals and nonterminals. So, that's what these two mean and how they differ. A nonterminal expression is symbolized by a nonterminal expression in the pattern structure. It maintains a list of abstract expressions, all the expressions that make up the nonterminal expression. Interpreting a nonterminal expression is typically done via recursion on all the variables that represent it.

Use Cases for the Interpreter Pattern

Let's have a look at use cases for this pattern, starting with some rather abstract use cases. The use case for this pattern is when there's a language you can interpret and you can represent statements in the language as abstract syntax trees. But that use case would apply to many types of languages, even though this pattern isn't necessarily a good fit for all of them. So, keep in mind that this is a good fit when the grammar is simple. If the grammar becomes complex, the class hierarchy for the grammar becomes large and unmanageable. Also, it is not the most efficient interpreter. The most efficient ones are usually implemented by translating parsed trees to another form first. Regular expressions, for example, are often transformed into state machines first. Yet, even then the translator itself can be implemented by the interpreter pattern, so the pattern is still applicable. Let's map this to a few

real-world examples, scenarios you may encounter in your day-to-day life. A first and very typical use case for this is a command-line parser. Interpreters can be used to parse and execute command-line instructions. For instance, in a command-line interface, you might define a grammar for commands, and an interpreter could execute those commands based on user input. And now we have HTML or XML parsing. Interpreters can be employed for parsing and manipulating HTML or XML documents. You define a grammar for the document structure, and the interpreter can navigate and manipulate the document accordingly. A very typical one is natural language processing. In certain natural language processing tasks, the interpreter pattern can be used to process and analyze sentences in a specific language. This is particularly useful in chatbots and virtual assistants. And as a final example, let's look into mathematical expression evaluation. When building a calculator or mathematical software, you can use the interpreter pattern to parse and evaluate mathematical expressions. Users can enter expressions like $2+3*[4-1]$, which can then be interpreted and evaluated correctly.

Pattern Consequences

Let's have a look at a few consequences for this pattern. An advantage is that it's easy to change and extend the grammar. Grammar rules are represented by classes, and these can be extended via inheritance. Also, implementing the grammar is easy too, as all classes that implement the abstract expression have similar implementations. A disadvantage is that complex grammars are hard to maintain. There's at least one class for every rule in the grammar. So, if you've got a large set of rules, you've got a large set of classes to maintain and manage.

Related Patterns

There are two related patterns for the interpreter pattern. First of all, the composite pattern, as the abstract syntax tree is, in fact, an instance of the composite pattern. And the second one is the iterator because in regards to traversing the structure, an iterator could be used. So therefore, the pattern is related to the iterator pattern as well.

Summary

Time for the module summary. In this module, we covered the Interpreter pattern. Its intent is to, given a language, define a representation for its grammar along with an interpreter that uses the representation to interpret sentences in the language. Such a language can consist of terminal and non-terminal expressions. A symbol, which is the smallest meaningful part or unit of language, is called a Terminal. Statements that are made up of terminals and are allowed by a language are called NonTerminals.

Checkpoint 26 - Implementing the Interpreter Pattern

Enterprise Pattern: Repository

Coming Up

Next to the Gang of Four design patterns, others exist that are commonly used these days. One of them is a Repository pattern. We'll dive into that in this module. I'm Kevin, and I'll guide you through it. We'll start with detailing the Repository pattern. What is it, and which problem does it solve? It exists in different flavors. Ones that adhere to a generic contract and ones that are catered to a specific set of functionalities. We'll detail both and implement both in a demo. Then, we will look into Entity Framework Core and the Repository pattern. You may or may not know this already, but Entity Framework Core provides an implementation of the pattern out of the box. Should you or should you not use that directly? That's the question we're going to try and answer in that section. And after that, we'll also look into a few real-world use cases. Let's dive in.

Detailing the Repository Pattern

So, what is the Repository pattern? I've used it in a few of my other courses, so you may have heard me mention it already. Imagine that you want to fetch a bunch of orders or order lines from your database, you can directly access it with ADO.NET or through Entity Framework Core by writing LINQ

queries in a data access layer. But that will probably lead to some code duplication, as it's not unlikely that you'll need the same set of data from different parts of your application. That then leads to another possible issue, your code will become more error prone if only for the duplication. And then there's testing. If you want a class that also contains behavior related to persistence, it's harder to pinpoint why something might go wrong. Is it the actual behavior of the method you're testing or is the persistence-related code the code that fails. If there was an easy way to mock the persistence-related code and test again, you could know that the mistake is or isn't related to that persistence logic. And that is where the Repository pattern comes in, it provides a way to abstract and encapsulate your data access logic from the rest of your application. It aims to make the code, say, for the repository implementation, persistence ignorant. The term persistence ignorant, also named persistence agnostic, might require some additional clarification. It means that the repository contract itself doesn't rely on or assume any particular type of data storage, that only comes into play when implementing the contract. Sometimes it's said that through a repository, you can switch out the persistence technology when needed. And, while that is strictly speaking true, that's not really the purpose of the Repository pattern. What it is very useful for is allowing us to choose which persistence logic or technology to use for a specific method on the repository. Creating an entity might be easier through Entity Framework. Getting an entity with some complex logic might be more advisable to ADO.NET, or you might even call into an external service. For the consumers of the repository, it's of no interest what goes on in the implementation. Rather than switching out one persistence technology for the other for a complete repository, it allows us to choose the technology that is the best fit for a specific method on the repository. As far as advantages go, maintainability is one of them. Changes to data access, for example, switching a part to a different database system, won't impact the business logic and vice versa. Then there's testability. Business logic can be tested independently by mocking or stubbing the data access layer through the repository. It also increases adaptability. As your application evolves, you can seamlessly adapt to changes in data storage requirements without rewriting large portions of code. And lastly, very much related to that is flexibility. Adapting to changes, for example, changing part of the data access logic, becomes easier because of the isolation of it from the rest of your code. So, in essence, you achieve separation of concerns between the business logic of your app and the data access logic, with the repository as the bridge between them. But how does this technically work then? How is this achieved? Well, the first part is the repository interface. For

example, a generic IRepository <T>, that declares the standard CRUD operations to be performed on data entities. Think of add, update, delete, get methods, and stuff like that. Then, there's the Repository implementation, concrete implementations of this interface that provide the actual data access logic, and they interact with the datastore and handle low-level operations. The business logic layer or whatever it is that consumes the repository, relies on the Repository interface. It doesn't directly interact with the datastore. Instead, it delegates data access tasks to the repository. Typically, dependency injection is then used to provide an instance of the Repository object to the consuming code. This decouples the components, allowing us to swap different Repository implementations without modifying the business logic. That can be very helpful for testing, for example, you could have a Repository that actually calls into a database and another one for testing that returns a few hard-coded Entity objects. So that is the general idea. Up next, let's learn about the two different flavors, generic and non-generic repositories.

Generic vs Non-generic Repositories

On screen, you see an example of a generic Repository interface with typical methods, like Get, GetAll, Add, and so on. An implementation of it on an Order entity with Entity Framework Core, may look as what you're looking at right now. So, the idea here is that you define common CRUD, Create, Read, Update, Delete, operations once and reuse them across different entities. Like that, you don't need to create separate interfaces for each entity. Taken one step further, you could also provide a generic base implementation to start from so you diminish code application even more. We will look into these approaches once we get to the demos. This has a few advantages. Reusability, as mentioned above, is one. Repetitive code is reduced. You don't need to write similar repository interfaces for every entity, which can be cumbersome and error prone. Instead, the same interface is reused or even mostly the same implementation. Next to that, it also means you have a consistent API for all entities, as the repository definition for all of them is the same. As a developer, you only have to learn to work with a single set of methods, like Get, Update, Delete, Add, instead of having to learn different methods for each entity. But there's also disadvantages. Lack of specificity is one. Generic repositories lack specific methods tailored to individual entities. For complex queries or specialized operations unique to an entity, you might need to supplement the generic repository with custom methods. Inheritance can help here. Related to that is over generalization. Sometimes generic

interfaces become too abstract, making it challenging to express domain-specific requirements that don't fit neatly into the generic CRUD operations. Get amount of orders for user on an IRepository of type Order would be an example of that. Then, there are performance concerns. Generic repositories might not optimize queries for specific entities. Non-generic repositories can fine-tune queries based on the entities characteristics, which potentially improves performance. And lastly, there's also potential maintenance issues. As your application grows, maintaining a single generic repository can become complex because it can lead to a large class with many methods, which is harder to manage and more likely to result in a lot of non-implemented methods on the repository implementations. So that leads me to the other approach, non-generic, or specific repositories. You're looking at an example of that right now. Instead of having one generic contract, each repository has its own contract and methods. So, we no longer have an IRepository of Order or IRepository of OrderLine, but we have an IOrder Repository and IOrderLineRepository with different methods in them, like GetAllByOrderId on the OrderLineRepository. That, too, has some potential advantages, which mostly come down to less chance of the disadvantages of the generic approach, of course. The most important two are, one, the fact that these allow different tailored methods, non-generic repositories allow you to define entity-specific methods. And two, the intent is more clear, non-generic interfaces convey intent clearly. When you see an IOrderRepository, you immediately know that it deals with order-related operations, more so than when you see an IRepository <T>. But, when there's advantages, there's also disadvantages. One would be potential code duplication, creating separate interfaces for each entity can lead to that because similar methods, like Get, GetAll, Add, etc., are often repeated across multiple interfaces. And then there's a learning curve. Developers need to learn and remember different repository interfaces for potentially each entity. This can become overwhelming in large projects. And here, too, there's potential maintenance issues. When you add more entities, managing multiple specific repositories becomes cumbersome, and changes to common methods may require updates in multiple places. So what do you do then? Well, what I would advise is to not be afraid to mix and match. Use a generic repository for common CRUD operations shared across entities and create specific non-generic repositories for entity-specific methods or complex queries. You can actually let both work together nicely, as we'll see in the demos. Talking about demos, let's continue with that.

Demo: Adding a Generic Repository Contract & Non-Generic Implementation

In this demo, we'll add a generic repository contract and implement it. But, let's first quickly go through the starter solution. For the EnterprisePatterns in this course, I've made a separate starter solution. You can find it in the exercise files, download for this course or on my GitHub, it's a simple console application with IOC enabled named EnterprisePatterns. In the Program file, which you're looking at right now, I set up the container. Then, I run a demo service, RepositoryDemoService, in this case. The code in the RepositoryDemoService doesn't do anything yet, as nothing has been implemented. We'll do that in this and the upcoming demos. I also already added an UnitOfWorkDemoService's for the next module. It's also empty. What is already there is a SQLite database, Orders.db. It also contains a little bit of dummy data. We're going to be working with Orders and OrderLines. And if you look at the OrderDbContext, you can see that I created a Shopping Spree and a Holiday Shopping Order and related OrderLines. If we want to work with our database, we could now directly inject this context in the RepositoryDemoService. Like that, we can access the Orders and OrderLine's Db sets, but that's exactly what we don't want to do. Instead, we're going to add a generic repository contract and implement it so that we can let the demo service rely on the repository implementation for data access. Let's start with the contract. I'm going to add a new folder here, Repositories, and let's add a new interface to it. I'll name the file, IRepositoryOfT. And let's name the interface IRepository, and we add T as a generic type parameter, so the interface is a generic interface working on something of type T. So, which methods should we define here? Well, very common to start with are methods to get something of type T by ID and a method to GetAll instances of type T, so that would be a Get and a GetAll method. Getting data means calling into the underlying datastore, and that is an I/O operation, so we ensure that we can use async code for that by returning tasks. This, of course, does mean that this generic repository interface is one that's intended to be used for datastore's that require an I/O operation to get or for test data. Up next, a generic repository interface tends to also contain methods for manipulating something of type T, so I can add methods to Add, Delete, and Update the entity. I don't return a task for Add, Delete, or Update, and this is on purpose. We're going to use this repository contract for repositories that integrate with Entity Framework Core. And in that case, adding, updating, and deleting entities are not I/O operations. When you add something on the DbContext, it's not immediately written to the database, instead that's just a simple in-memory operation. Persisting

data happens when calling `SaveChanges` on the `DbContext`, not when adding, deleting, or updating something. And that brings me to the last method, `SaveChanges`. This again, returns a `Task`, and we will use this to persist the changes. Let's save this. Now, I'm saving the changes on the repository itself. This is just one approach. Another approach is to not have this method here, but, for example, in a unit of work, and that is what the next module is. For for this module, which only deals with repositories, we'll keep it like this. All right, that's it for the contract, let's implement this, so I'm going to add a new class, `GenericOrderRepository`. Seeing we're going to call into our database via Entity Framework Core via this repository, it depends on the `OrderDbContext`, so let's inject that. Let's store in a readonly field. We also make the repository public, which comes in handy when you want to use it for testing from outside of this assembly. And we make sure it implements `IRepository`, which is our generic repository interface. This is a repository that should work on Orders, so we pass through order S type for this generic repository. There we go. Now, let's implement this contract, so we choose to implement the interface, and then let's see what we need to do here. The first method we need to implement is adding an Order entity, that's quite easy to do with Entity Framework Core, we simply call in to `Add` on the Orders Db set. For the deleting an Order entity, we call `Remove` on Orders. The next method to implement here is the `Get` method. We want to get a specific order by Id and it can or cannot be null. There's a method for that, `Find`. So, we call into `dbContext.Orders.Find`. We don't just want to use the `Find` method, we want to use to `FindAsync` method. Because this is an I/O call, it calls into the database. What we pass through to it is the id. Now, of course, as it is an async method call, we want to await it, and we want to make sure we can call await it by making the method async. Like that, we can get one order by id. If no order is found, this will return null, and that is why the Order parameter here is nullable. To get all orders, we await a call into `ToListAsync` on the Orders Db set, and we make sure we make the method at async as well. To save the changes, we call to `SaveChangesAsync`. This, too, is an Async call so we await it, and we make the method async as well. And then lastly, we've got the `Update` method we need to implement here. But for that, there is no code required in this implementation. This is because once we get an order, it's automatically tracked by the `DbContext`, that also means that if you make changes to properties on that order from somewhere else in your code, those changes will automatically be tracked as well. That means there is no need to write code in the `Update` method here. Now, of course, this isn't a rule that's set in stone. You can also explicitly state that you do not want to track something, something like this tends to

happen when you separate out reading from writing, so in a CQRS approach. We don't have that here, so for us this is not applicable. All right, that's it already. Now let's test this. First, let's open the Program class again, and let's make sure we registered this repository as an implementation of IRepository<Order>. For that, we call into AddScoped, passing through IRepository<Order> as the contract and GenericOrderRepository as the implementation. Now, let's inject this in our RepositoryDemoService. So, what I want to inject here is an IRepository<Order>, and I'll name that _genericOrderRepository. Let's add the namespace, let's add a using statement, and let's assign it to readonly field. All right, with that, we've got our _genericOrderRepository. Let's first try getting an order. For that, we call into Get, passing through the id, and let's also check if we can effectively update the order by changing the description. All right, that should be it. Let's give this a try. I'm going to put a breakpoint here. We hit our breakpoint, so far, so good. Let's continue. The order was successfully fetched. Let's continue again. Now let's have a look at the output, and we indeed see that an UPDATE command was sent to the database, so this seems to have worked. We created our first Repository pattern implementation, and by doing so, made our RepositoryDemoService not depend on Entity Framework Core, and more easily testable. But, Repository pattern implementations come in different flavors. A common approach to improve testability without having to write an overload of additional classes is by creating a generic implementation next to the generic contract we already have. Let's give that a try in the next demo.

Demo: Adding a Generic Repository Implementation

In this demo, we'll add a generic repository implementation. This allows us to avoid having to create implementations for each specific entity type. Now, of course, this does mean you're tying all of these implementations to EF Core. But for a lot of applications, EF Core is effectively the only way you're interacting with an underlying datastore, and such an approach can then be beneficial to avoid unnecessary classes. Let's add a new file to the Repositories folder. I'll name it GenericEFCoreRepository. It's generic, so we pass through a parameter of type T, and it implements IRepository of type T. We're going to need our Order dbContext, so let's use a bit of constructor injection to get that. Now, we want to work on an entity, so we'll need a generic way to access those. As you know, access to entities via Entity Framework Core is made possible via the Db set. In fact, on our context, we have these Db sets, so we have an Orders Db set and an OrderLines Db set.

There's also something else. There's a set method on the Context here. And with this set method, we can generically create such a Db set, and that is what will allow us to interact with things of type T. So, let's add a variable here of type dbSet of type T. And in the constructor, let's initialize it as a dbSet of type T. There's little problem though, it says here below that the type T must be a reference type in order to use it as parameter TEntity in the Set method. In other words, we need to ensure that T is a class, and we can easily ensure that by adding a generic constraint. So, we add where T class. There we go. The warning is gone. And with this, we are now able to implement our required methods. So, let's implement this interface. Let's start with the Add method. Simple enough, we used to call into the dbContext.Orders.Add. We now call into dbSet.Add. For the Delete method, we call into dbSet.Remove. And then, we've already added our Get method. This, too, matches what we did in our other repository implementation. We call into dbSet of FindAsync. It's an Async call so we await it, and we make the method async so we can effectively await something inside of it. We call into await _dbSet.ToListAsync, and we make the method async as well, and then saving the changes is a matter of calling into SaveChangesAsync, and here, too, awaiting the call for which we need to make the SaveChanges method async. Here, too, no code is required in the Update method, just as I explained in the previous demo. All right, let's save this because this is it already for this implementation. To change this, I'm going to change the registration of IRepository<Order>. Instead of registering it as a GenericOrderRepository, I'm going to say that the implementing type is a GenericEFCoreRepository of type Order. All right, let's give this a try. First, let me put my breakpoints here again. All right, we hit our first breakpoint. And if you look at the order, we see that it is indeed our Shopping Spree order. So far, so good. Let's continue. And we again see that an UPDATE statement was sent to the database. In case we would now want to work on OrderLines, we can simply register an IRepository of OrderLine as a GenericEFCoreRepository that works on type OrderLine without having to create another class. That's it for this demo.

Demo: Adding a Non-Generic Repository Contract & Implementation

In this demo, we're going to add a non-generic repository contract and implementation. As you remember, this is often needed when non-typical methods are required. For an OrderLineRepository, it's easy to imagine that we'd want a way to fetch all order lines related to an order. Let's create a repository for that, and let's start with the contract. So, I'm going to add an Interface here, let's name it

IOrderLineRepository. Let's think about the methods we need here. First methods we need are again the methods we already know. So, Get and GetAll to get one specific OrderLine or to GetAllOrderLines. And then, Add, Delete, Update, and SaveChanges, those are all so familiar. Well, let me clean this up a bit. The difference here between this and the generic repository is, of course, that these now work on OrderLines directly and not on something of type T. This time, we also have one additional method, GetAllByOrderId. This method accepts an orderId and returns an IEnumerable of OrderLine. This is, of course, something that wouldn't nicely fit in a regular generic repository contract. All right, let's save this contract, and let's add a new class implementation so we can implement it. I'll name that OrderLineRepository. There's nothing new here, so let me paste that in. I will have to import. I will have to add a using statement, though. There we go. So, the one method that's different from what we've previously done is the GetAllByOrderId method. This contains a where clause for filtering to ensure that we only return order lines related to an orderId, so related to a certain parent. All right, that's it. Let's register this on the container, so I add a new scoped service to it. The interface is IOrderLineRepository and the implementing type is our OrderLineRepository. And let's test this, so I can open my RepositoryDemoService. And this time, I need to inject the OrderLineRepository as well, so I inject something that adheres to the IOrderLineRepository contract. Let's name that orderLineRepository, and let's store that in a field. And then, let's create a new OrderLine for the previously-fetched order so when the order isn't null, it will be updated. We create a new OrderLine, we Add it, and we save the changes. All right, let's give this a try. We don't need those breakpoints here anymore, we can simply look at the console output to see whether our queries are effectively sent to the database. And there we go. Here is our INSERT statement for one OrderLine. Our code was executed correctly. Now, you may wonder, these two SaveChanges calls here on line 22 and online 29, are they really necessary, in this case, and why or why not? Well, we could have gotten away with one SaveChanges call in the same scope, but that is something for the next module in which we will look into the Unit of Work pattern. For now, it is time for the last demo of this module, in which we will see how we can combine generic and non.

Demo: Combining Generic and Non-Generic Repositories

So, we seem to have ended up with a potential code application issue. The code in our non-generic repository and related interface, which we're looking at right now, is mostly of the same kind, methods

to GetAll, GetById, and so on, as the ones in our generic repository, so the IRepository<T> implementing classes. We don't want to have to rewrite all of that for a non-generic repository. Luckily, in an OO language as C#, this is quite easy to avoid. In fact, there's multiple ways to avoid unnecessary code duplication. We can let a class implement two interfaces. We could derive from a base class, we can use composition to get an object in our repository that contains the common behavior, and so on. Yet, another approach, one I often see, is using interface inheritance. Let's do that. So, I'm going to change our IOrderLineRepository interface. Let me just comment out what we have here so you still have it for reference. Now, let's paste this in here. And now, I'm going to derive from our generic repository interface, so the IOrderLineRepository derives from or implements, IRepository<OrderLine>. This means that anything that implements the IOrderLineRepository, also has to implement the methods on IRepository of type OrderLine. Next to that, the one additional method GetAllByOrderId will have to be implemented. And what we see here is that we see these little green lines underneath the Get, GetAll, etc. methods, these are the methods that are also defined on the IRepository interface, and it states here that these methods had inherited members on the base class or the base interface. We actually don't want to do that. We have no need for these. We effectively want to avoid code duplication, so we can simply remove this. All right, let's save that. Now let's have a look at our current implementation, so that is the OrderLineRepository. First, let me comment out what we have so you still have it for reference. Let's work on a copy of that. There we go. We can improve this, too, because here, too, most of these required methods have already been implemented somewhere else, on our GenericEFCoreRepository class. So, what we can do here too is derive from that on type OrderLine, for which I need to pass through the OrderDbContext, and we still implement the IOrderLineRepository. And we see the same happening here. The methods that already exist on our GenericEFCoreRepository are underlined with a green line because we are hiding them from the class we derive from. We don't want to do that, we want to use the implementation that is already in the class we derive from, so here, too, we can simply delete all of this duplicate code. And there we go, that should be it. We don't have to change anything else. The registrations are still okay. So if we run this now, we should again see our eventual INSERT statement for the OrderLine. And here we go, that is indeed the case. All of this still works as expected.

EF Core and The Repository Pattern

All right, so in the demos, we create repositories that work. But you may have heard that Entity Framework Core itself provides an implementation repository pattern, too. What's that about, and why should we create repositories around that then? Well, when you want to interact with data through the DbContext, you do that by defining DbSet on the DbContext. DbSet allow you to query entities of a specific DbContext. To stick with the DbSet here, we'd have a DbSet of Order and a DbSet of OrderLine. Such a DbSet represents a collection for a given entity. And it's through that, you eventually get to database operations against an entity. You can add entities to the set, you can update entities in the set with queries, and so on. Does that sound familiar? Well, it probably should. The DbSet of T is a generic representation. That, by the way, makes the DbContext a unit of work.

Real-world Use Cases

First, use case maybe a library management system. Imagine such a system where users could borrow them and return them. The Repository pattern can be used to abstract the details of data storage for patrons and transactions, so you'd have a book repository, a pay repository, and a transaction repository to handle corrupt operations for their respective entities. Deals with products, orders, and customers. The Repository pattern allows this logic to interact, in this case, a product repository, order repository, and customer, these repositories manage data persistence and storing customer information. And in fact, all applications that deal with n access can benefit from the repository pattern. I can view more, but I'm sure you can as well matter which type of data.

Summary

So, we learned that the Repository pattern provides a way to abstract and encapsulate your data access logic from the rest of your application. It aims to make the code safe for the repository implementation, persistence ignorant. It is useful for most applications that deal with data access. Generic and non-generic implementations are common, and don't be afraid to use both if needed. Entity Framework Core provides a repository implementation via the DbSet. But it is considered a bit leaky, so it's often encapsulated in another repository just as we did in the demos. And that's it for this module. But, we've currently got those SaveChanges called in our repositories, does such a method

really belong there? And what if we need to work with transactions across repositories? That is where the Unit of Work pattern comes in. Let's have a look at that one next.

Enterprise Pattern: Unit of Work

Coming Up

The Unit of Work pattern is another common pattern used in Enterprise software. It's used for managing transactions and coordinating multiple data operations across repositories. I'm Kevin, and I'll teach you how to use it. We'll start with detailing the Unit of Work pattern. What is it, and which problem does it solve? Then, we'll implement it in a demo. We will build upon what we ended up with in the previous module. But if you didn't watch that, no worries, I will quickly run through the starter solution first. We'll also look at how it relates to Entity Framework Core, and that is important because Entity Framework Core provides an implementation of the pattern out of the box. After that, we will also look into a few real-world use cases. Let's dive in.

Detailing the Unit of Work Pattern

In the previous module, we detailed the Repository pattern, which provides an abstraction layer between the data access layer, which contains the actual persistence logic and the rest of your application. When you start using the pattern, you will quickly run into an issue. What if you want to ensure that the data changes you want to persist across different repositories are handled in the same transaction? Take an Order and OrderLines repository as an example. You can create an order and persist it through the Order repository, and you can add and persist OrderLines through the OrderLineRepository, but that may mean you end up with inconsistent data. You may end up in a situation where saving the Order was successful, but saving the OrderLine, or Lines, failed. And that is where the Unit of Work pattern comes to the rescue, its primary goal is to manage transactions and coordinate multiple data operations within a single logical unit. Within this unit, multiple data

operations like inserts, updates, and leads of an Order and related OrderLines are grouped together. The Unit of Work then ensures that either all operations within the unit succeed or none of them do. If any operation fails, the entire unit of work is rolled back. All right, so that is what it does, but how do we implement something like that? Well, the first thing to do is create the UnitOfWork interface and base class. On it, we need to define a way to commit and rollback a transaction, so we define two methods for that. What I tend to do then is add an implementation of this which can serve as a base class. It depends on the DbContext. It's through this DbContext that we'll handle the transactions. We'll get to that in the demo. The third class is then the actual unit of work, for example, to create an Order with OrderLines. It derives from the base class, so it depends on the DbContext. And on top of that, it relies on one or more repositories through which changes can be applied. In our case, that's the Order and OrderLine repositories. The DbContext instance required by the different repositories is the same one. Registering it as a scope service, which is done by default, takes care of that in a web app. The method to effectively save the changes via the DbContext is moved from the repositories to the unit of work, that means that executing methods on the repositories don't save changes yet. It's only after all the methods on the different repositories have been executed that we call SaveChanges via the UnitOfWork, so via committing it through the UnitOfWork. These changes need to be wrapped in a transaction, and that is something you get out of the box with Entity Framework Core. By default, all changes on entities that are tracked by Entity Framework Core's change tracker are wrapped in a single transaction when saving the changes. So like that, we ensure that all changes are persisted or none at all. Let's have a look at a demo.

Demo: Implementing the Unit of Work Pattern

In this demo, we'll implement the Unit of Work pattern. In case you haven't watched the previous module in which I detailed the Repository pattern, let me take a few seconds to run through the startup solution here. This is a simple console app in which I set up dependency injection. You can see that I have a ConfigureServices method here, which is used to add services on the container and configure them so I can inject them when needed. Running the application, means getting a specific service and running that. Currently, that's still the RepositoryDemoService from the previous module. I'm going to change that to the UnitOfWorkDemoService, which is currently not implemented. Next to that, the demo scenario is pretty straightforward. I've got a SQLite database with Orders and related OrderLines

in it, which are exposed via an `DbContext`. Instead of directly calling into that `DbContext`, we work with the data through different repository implementations. So, if you have a look at the `Repositories` folder here, you can find different types of repositories implemented either generically or non-generically, and that's all you need to know to get started. But, what do we need for this demo? Well, we need methods to commit a unit of work and to roll it back. So, let me add a new folder here, `UnitsOfWork`, and let's add an interface to it. I'll name that interface `IUnitOfWork`, and I have it contain two methods, one to `Commit` changes and another one to `Rollback`. Then, let's implement this interface. I'm going to do that in a base class so our actual `UnitsOfWork` can derive from that. Like this, we avoid a bit of code duplication, so let me add a new class `UnitOfWork`. Let me clean that up a bit here. There we go. And I want to make this class abstract to start with. Like that, we indicate that we don't want this class to be used on its own. We need concrete implementations of this class in which additional behavior is defined. We want the class to implement the `IUnitOfWork` interface, that means we'll need to provide implementations for these two methods, `CommitAsync` and `Rollback`. To be able to do that, we need our `DbContext`. So, we inject it, so I inject an `DbContext`. I'll name it `dbContext` as variable name, and I'll store it in a readonly field. To commit changes, I need to call to `SaveChangesAsync` on the `dbContext`, so let's do that. It is an async method so I need to await it. And therefore, I also need this `CommitAsync` method to be marked with the `async` keyword. Like that, we can await methods inside of it. This call will automatically wrap all the changes that are being tracked on the context in a single transaction, thus ensuring that either all are saved or none at all. What do we do in the `Rollback` method? Well, we can rollback the changes we did on the context by clearing the change tracker. This means that none of the previously-applied changes are tracked anymore, resulting in nothing being saved and `SaveChangesAsync` is called. Now, what I often see is that a `Dispose` method is added in which `Dispose` is called on the `DbContext` instance. I'd say you mostly don't want to do that. Disposing of objects should be done by the component that created it, the component that is responsible for managing the lifetime of that object. There used to be a time where we did all of that ourselves. But today, most of the lifetime management of objects is actually done by your IoC container, it's thus the responsibility of the IoC container to dispose of objects that are no longer needed, so don't do this unless you're not using an IoC container. All right, let's implement this for one use case. We want to create an `Order` together with a set of `OrderLines` in one transaction. So, all operations should be executed correctly are null. For that, we're going to need two repositories, our

OrderLineRepository and an Order repository, either to GenericOrderRepository here or one via a GenericEFCoreRepository. So let's CreateOrderWithOrderLinesUnitOfWork, let's add it, and let's have it implement our abstract base class, UnitOfWork. To be able to do that, we need the context. So, we inject it in our CreateOrderWithOrderLinesUnitOfWorkClass. Let's name it, OrderDbContext. I don't need to store it because I don't need it in this class. I simply need to be able to pass it through to the underlying UnitOfWork class, so that is exactly what I do. Also, we do not create this context ourselves. We let the IoC container manage its lifetime. Like that, if this were a web app, we'd be sure that there would be only one instance of this context per request, as the DbContexts are registered as scoped by default. All right, the two other things I need here are our Order and OrderLine repositories. So, I'm going to inject these here as well, and I'm going to make these available as properties. Like that, wherever we use a CreateOrderWithOrderLinesUnitOfWork, we can easily get to the repositories related to this specific unit of work, so I create and assign an orderRepository and an orderLineRepository property. All right, that's it for the UnitOfWork, let's register this class so we can inject and test it. We register it on the IoC container in our Program class here. We do that by calling into AddScoped, passing through the type, and now let's open the UnitOfWorkDemoService and let's inject it. So we inject a CreateOrderWithOrderLinesUnitOfWork, and we name UnitOfWork. Let me store that in a field. To test this, we'll first create an Order, and then we can easily add it via the Order repository by accessing the OrderRepository property on our UnitOfWork instance. Then, we create an OrderLine for this repository. Here, too, we can easily add it via the OrderLineRepository property on the UnitOfWork, and now we persist the changes by calling to CommitAsync on our UnitOfWork. This is an async call so we await it. As explained before, these changes are automatically wrapped in a transaction. As you can see, there are no SaveChanges methods on the repository called anymore. I'm not going to remove them, as that would make our code from the repository demos stop compiling. But know that if you use the UnitOfWork pattern, you shouldn't have those SaveChanges methods on the repositories anymore. All right, let's make sure we run this code when starting the application. So, let's scroll down a bit here, and let's make sure we execute or RunAsync method on the UnitOfWorkDemoService. Let's give that a try. We should now see our two INSERT statements being executed. And as you can see, that is indeed the case. This was the result of that single CommitAsync call on our UnitOfWork, which is automatically wrapped in a transaction.

Alignment with EF Core

In the previous module, I mentioned that Entity Framework Core implements the Repository pattern through the DbSet, but that's not all it does. It also implements the UnitOfWork pattern, and I think you probably already figured that out by now. It's the DbContext in Entity Framework Core that acts as a UnitOfWork, as it manages the transaction scope. The DbContext tracks changes to entries, it handles database queries and persists changes when necessary, and when persisting the changes that's done in one transaction. In other words, either all changes are persisted or none, and that is the Unit Of Work pattern at work. Let's look into a few real-world use cases.

Real-world Use Cases

Let's start with a banking application. In financial systems, transactions involve multiple steps, deducting funds from one account and adding them to another, for example. The Unit Of Work pattern ensures that these operations execute within the same transaction boundary, so if any part of the transaction fails, for example, due to insufficient funds, the entire Unit Of Work is rolled back, and data remains consistent and correct. Another example could be an E-commerce checkout process. During an online purchase, several actions occur, updating the cart, reserving inventory, and recording the order. The Unit Of Work coordinates all these steps, ensuring that either all succeed or none. For example, if the inventory reservation fails, the entire checkout process is rolled back. And just like with the Repository pattern, almost all applications that deal with data access can benefit from the Unit Of Work pattern. Let's have a look at the module summary.

Summary

The primary goal of the Unit Of Work pattern is to manage transactions and coordinate multiple data operations within a single logical unit. It is an often-used pattern in applications that deal with data access. In fact, it is so common that Entity Framework Core implemented it. The DbContext is an implementation of the Unit Of Work pattern. And that is it for this lengthy course. I hope you learned something, and I hope you'll be able to apply these patterns in your day-to-day development life, as they can really help improve your code base. If you have a question, don't hesitate to ask it on the

Discussion step. Also, please consider rating the course, it gives us authors some feedback on whether we're doing this right or not. Just one more thing before I go, you're ready to be awesome.